

学生服务平台 (Server) 项目面试题库

基于 Spring Boot 3.5 + Redis + Kafka + WebSocket 的一站式校园生活服务平台

由易到难 · 九级难度分层 · 完整版

目录

1. 第一级：项目介绍与基础认知（热身题）
2. 第二级：Spring / Spring Boot 基础
3. 第三级：MyBatis 与数据库
4. 第四级：认证授权 —— JWT 与拦截器
5. 第五级：Redis 与缓存体系
6. 第六级：限流与 AOP
7. 第七级：秒杀系统核心链路（重点）
8. 第八级：Kafka 消息队列与最终一致性
9. 第九级：WebSocket 实时通信
10. 第十级：并发、JVM 与性能调优
11. 第十一级：系统设计开放题（高难度）
12. 第十二级：项目缺陷与"陷阱题"（面试官最爱问）

第一级：项目介绍与基础认知（热身题）

Q1.1 请用三分钟介绍一下你这个项目。

答题框架（STAR + 技术亮点）：

这是一个面向高校场景的**一站式校园生活服务平台**，后端基于 Spring Boot 3.5 构建，Java 17 编译。核心业务有四大块：

1. **用户体系**：注册、登录（JWT 无状态认证）、忘记密码（邮箱验证码）、头像上传、个人信息管理。
2. **选课系统**：课程条件筛选、选课/退课，采用"数据库乐观扣减 + Redis Hash 缓存"双写。
3. **商品秒杀（核心亮点）**：Redis + Lua 原子预扣库存，Kafka 异步削峰落库，配合 Caffeine 本地缓存、订单号分布式生成、事务提交后回调发消息等一整套高并发处理方案。
4. **实时在线统计**：WebSocket 长连接 + Redis 有序集合维护在线用户，配合心跳、僵尸连接清理、定时广播。

技术上值得说的点：双限流机制（AOP 注解式限流 + 拦截器式限流）、四级缓存（Caffeine → Redis → MySQL）、异步化（邮件全部走 Kafka）、事务与缓存的最终一致性补偿。

面试技巧：不要平铺直叙讲功能，要主动抛"钩子"。比如说完"秒杀用了 Redis + Lua"就停下，等面试官追问。面试官只会深挖你主动提到的技术点。

Q1.2 为什么项目里有两套 Controller（`control/` 和 `api/`）？

项目采用**前后端分离 + 多端适配**的分层设计：

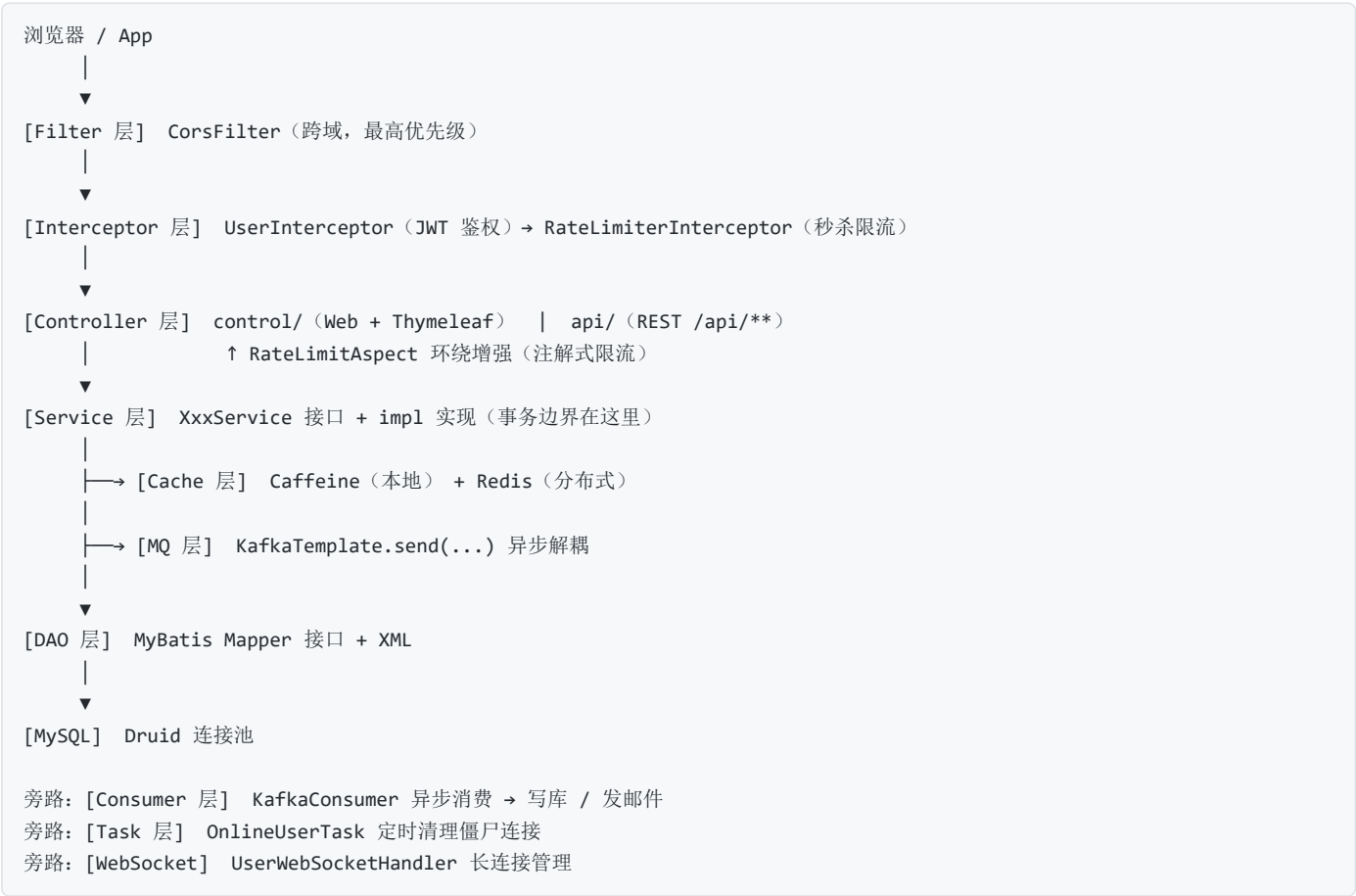
- `control/` 包同时服务 Web 端（Thymeleaf 服务端渲染页面，如 `login.html`、`person.html`）和部分 REST 接口；路径没有统一前缀。
- `api/` 包统一以 `/api/` 开头，专门给移动端 / 小程序 / 第三方调用，接口更精简。

两者共用同一个 **Service 层**，所以业务逻辑只写一遍，Controller 只做参数接收与视图适配。

这样设计的好处：业务逻辑复用、端侧差异化（Web 端返回重定向，API 端返回 JSON）、便于将来做网关级路由隔离（`/api/**` 单独限流、鉴权策略）。

可以主动补充的缺点：同一个业务在两处写了两遍参数接收代码，存在重复；更好的做法是在 `control` 层里把 Web 页面和 API 完全拆开，或者用 `@RequestMapping` 的 `produces` 做内容协商。目前这个设计属于“能用但不够优雅”。

Q1.3 整体分层架构是怎样的？请画出调用链路。



要点：要能说清楚 **Filter → Interceptor → AOP → Controller** 的执行顺序，这是面试官马上会追问的点。

Q1.4 项目用到了哪些技术栈？各自解决什么问题？

技术	版本	在本项目中的作用
Spring Boot	3.5.13	基础框架，自动配置、内嵌 Tomcat
Java	17 (编译目标 21)	语言版本
MyBatis	3.0.5 starter	ORM，手写 SQL 便于优化
MySQL	8.x (阿里云 RDS)	主数据存储
Druid	1.2.21	数据库连接池 + 监控 (<code>/druid/*</code>)
Redis	Spring Data Redis	缓存、秒杀库存、限流窗口、验证码、在线用户
Redisson	3.26.0	分布式锁、分布式原子自增 (订单号)
Caffeine	—	JVM 进程内本地缓存 (用户、商品)
Kafka	spring-kafka	异步削峰 (秒杀落库)、邮件解耦、验证码

JWT	0.12.6	JWT 生成与校验
Spring Security Crypto	—	只用了 BCrypt 密码加密，没引入完整 Security
WebSocket	spring-boot-starter-websocket	在线人数实时推送
Springdoc OpenAPI	2.8.6	Swagger 接口文档
PageHelper	1.4.7	评论分页
Thymeleaf	—	Web 端服务端渲染页面
Lombok	1.18.36	简化样板代码
fastjson2	2.0.32	JSON 序列化 (Kafka 消息体)

Q1.5 为什么项目里同时引入了 Spring Session Redis，但配置类里又把 @EnableRedisHttpSession 注释掉了？

这是一个架构演进的痕迹，也是很好的加分回答：

- 早期方案：**项目用传统 Session 维护登录态，为了保证多实例部署时 Session 共享，引入了 `spring-session-data-redis`，用 `@EnableRedisHttpSession` 把 Session 存进 Redis。
- 遇到的问题：**Session 方案下，用户信息以 `studentNum` 为 key 写进 Redis，导致缓存键混乱；同时 Session 是“有状态”的，在移动端、跨域、横向扩容场景都不友好。
- 最终方案：**改用 **JWT 无状态认证**，服务端不保存任何登录态。配置类里 `@EnableRedisHttpSession` 被注释，`SpringHttpSessionConfig` 变成了空类。

反面思考（面试官会追问）：既然不再用 Session，为什么 pom 里还留着 `spring-session-data-redis` 依赖？这是技术债，应该清理掉，否则会额外注册一堆 Session 相关的 Filter 和 Bean，增加启动耗时。

Q1.6 为什么选择 MyBatis 而不是 JPA / MyBatis-Plus？

选择 MyBatis 的理由：

- SQL 可控：**秒杀场景下 `update product set stock = stock - #{count} where id = ? and stock > 0` 这种“数据库层面保证不超卖”的写法，用 JPA 表达起来很别扭。SQL 掌握在自己手里，才能做针对性的优化。
- 解耦实体：**项目用 `dataobject`（DO，数据库映射）+ `model`（业务模型/对外 VO）分离，MyBatis 的 `resultMap` 天然支持这种手动映射。
- 学习成本低、团队熟悉度高。**

可以主动说的取舍：如果项目是典型的 CRUD 后台，MyBatis-Plus 能省掉大量重复的 XML，开发效率更高。本项目里有大量复杂 SQL（JSON_CONTAINS 查询、唯一索引幂等、乐观扣减），所以手写 XML 更合适。

Q1.7 DO / Model / VO 分了几层？为什么要这么分？

项目实际分了三对象：

类型	包	职责
XxxDO	<code>dataobject/</code>	与数据库表一一对应，含 <code>gmtCreated / gmtModified</code> ，实现 <code>Serializable</code> （要进 Redis）
Xxx	<code>model/</code>	业务模型 / 对外返回值，通过 <code>toModel()</code> 转换
XxxVO	<code>model/</code>	组合展示对象，如 <code>LoginVO{token, userInfo}</code>

为什么这么分：

- **安全**：UserDO 里有 password 字段，toModel() 里明确不拷贝密码，保证密码永不外泄。同时 User.password 上还加了 @JsonSerialize(using = NullSerializer.class) 作为第二道保险。
- **字段裁剪**：UserInfo 是 JWT 里携带的脱敏信息，只有 id/userName/studentNum/nickName/email，避免 token 过长。
- **解耦**：数据库加字段不影响对外接口。

追问：“为什么不用 MapStruct / BeanUtils？”项目用了 BeanUtils.copyProperties，缺点是反射性能差且字段名匹配靠约定、出错不报错（比如 userName 写成 username 会静默丢失）。生产项目更推荐 MapStruct（编译期生成代码）或手写映射。

Q1.8 数据库有哪几张表？关键索引怎么设计的？

表名	作用	关键约束
user	用户表	学号 student_num 唯一
course	课程表	grade / college / major 存 JSON 数组
user_course	选课关系表	user_id + course_id
product	秒杀商品表，含 stock	stock > 0 作为扣减守卫
snappedUser	抢购成功记录	唯一索引 uk_user_product(user_id, product_id) —— 一人一单的最终防线
order	订单表	唯一索引 uk_user_product(user_id, product_id) —— 防重复订单
comment	评论表，带 parent_id 支持多级回复	—

为什么 snappedUser 和 order 都要建唯一索引？

这是幂等性的兜底设计。Redis 层的 user:snapped:{userId}:{productId} 只能挡住“Redis 还活着的时候”的重复请求；一旦 Redis 数据丢失、key 过期，或者 Kafka 因重试导致消息重复投递，唯一索引就是最后一道防线——重复插入会抛 DuplicateKeyException，代码里显式捕获并 ack 掉，保证业务幂等。

追问：“一个用户能抢多个不同商品吗？”能。唯一索引是联合索引，限制的是 (user, product) 组合唯一，不是 user 唯一。

Q1.9 项目有哪些模块是你自己写的？哪些是参考资料？

诚实但突出亮点：这题没有标准答案，但绝对不能吹牛——面试官会顺着你吹的点深挖到细节，一旦答不上来整场就崩了。

推荐答法：先说清哪些是参考了教程/开源项目的骨架（如基础 CRUD、Thymeleaf 页面），再说哪些是自己独立设计并踩过坑的（如秒杀链路的 Lua 脚本、事务回滚补偿 Redis、限流切面的键设计、WebSocket 在线人数漂移校准）。

项目里 todo/ToDo.java 记录了完整的踩坑与修复过程，比如“解决由于事务回滚导致的 redis 数据扣减与数据库不一致”、“解决压测失效问题”、“解决 redis 里面意外出现的分布式 ID 键值对”。这些才是真正的加分项——说明你是真的跑了、真的压测了、真的遇到了问题并解决了。

第二级：Spring / Spring Boot 基础

Q2.1 @RequiredArgsConstructor + private final 的注入方式和 @Autowired 有什么区别？

项目里所有 Service、Controller、Config 都用了 Lombok 的 `@RequiredArgsConstructor` 配合 `private final` 字段，这实际上是构造器注入。

对比项	字段注入 <code>@Autowired</code>	构造器注入（本项目）
可变性	字段不能是 final，可被反射篡改	可声明 <code>final</code> ，不可变
依赖完整性	对象可能以“半初始化”状态被创建	构造完成即依赖齐全，天然避免 NPE
循环依赖	Spring 可用三级缓存“绕过”	启动直接报错，早暴露问题
可测试性	必须启动 Spring 容器或反射注入	直接 <code>new</code> ，纯单元测试
代码	每个字段一个注解	一个类注解搞定

Spring 官方推荐构造器注入，理由就是“依赖不可变 + 不为空 + 完全初始化”。本项目 `UserServiceImplLoginTest` 就是用 `@InjectMocks` 走构造器注入做纯单测的，完全不启动 Spring 容器。

陷阱题：“`@RequiredArgsConstructor` 生成的构造器有 `@Autowired` 吗？”

答案：**没有**。Spring 4.3 之后，如果一个类**只有一个构造器**，Spring 会自动把它当作注入点，不需要 `@Autowired`。但如果类里有多个构造器，就必须显式标注 `@Autowired`，否则 Spring 会用无参构造器，final 字段全为 null。

Q2.2 Spring Boot 自动配置的原理是什么？

核心是三个注解合成：

```
@SpringBootApplication
= @SpringBootConfiguration // 本质是 @Configuration
+ @ComponentScan           // 扫描启动类所在包及子包
+ @EnableAutoConfiguration // 自动配置的核心
```

`@EnableAutoConfiguration` 通过 `@Import(AutoConfigurationImportSelector.class)`，在 Spring Boot 3.x 中读取 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件（2.x 时代是 `spring.factories`），拿到几百个自动配置类的全限定名。

然后每个配置类上的条件注解决定是否生效：

- `@ConditionalOnClass`：类路径下存在某个类才生效。比如引入了 `spring-boot-starter-data-redis`，`RedisAutoConfiguration` 才生效。
- `@ConditionalOnMissingBean`：容器里没有这个 Bean 才创建。这就是为什么项目里自定义了 `RedisTemplate<String, Object>` 后，默认的那个就不会被创建（或者说被覆盖）。
- `@ConditionalOnProperty`：配置项开关控制。
- `@ConditionalOnWebApplication`：Web 环境才生效。

结合本项目举例：项目没有引入 `spring-boot-starter-security`，只引入了 `spring-security-crypto`，所以 `SecurityAutoConfiguration` 不会生效，也就不会出现默认的登录页和被拦截的问题，但依然可以用 `BCryptPasswordEncoder`。这是一个很典型的“按需引入”实践。

Q2.3 Bean 的生命周期讲一下。

完整流程（面试背这个）：

1. **实例化**：调用构造器创建对象（本项目里就是 Lombok 生成的构造器完成依赖注入）。
2. **属性填充**：@Autowired / @Value 注入，处理 @Value("\${jwt.secret}") 这类占位符。
3. **Aware 回调**：BeanNameAware → BeanFactoryAware → ApplicationContextAware。
4. **BeanPostProcessor.postProcessBeforeInitialization**（前置处理）。
5. **初始化**：@PostConstruct → InitializingBean.afterPropertiesSet() → @Bean(initMethod)。
本项目 SnappedUpServiceImpl.initStock() 和 UserWebSocketHandler.initOnlineCount() 都用了 @PostConstruct。
6. **BeanPostProcessor.postProcessAfterInitialization**（后置处理）——AOP 代理就是在这里生成的。
本项目 RateLimitAspect 切的是 Controller，所以 Controller Bean 在这一步被替换成代理对象。
7. **使用中**。
8. **销毁**：@PreDestroy → DisposableBean.destroy() → @Bean(destroyMethod)。

结合本项目的坑：SnappedUpServiceImpl.initStock() 在 @PostConstruct 里把数据库库存刷进 Redis。如果此时 Redis 不可用，整个应用会启动失败——因为 @PostConstruct 抛异常会导致 Bean 创建失败，进而容器启动失败。生产环境应该 try-catch 并降级，或者改成懒加载兜底。

Q2.4 解释一下 Spring 的三级缓存，本项目有循环依赖吗？

三级缓存：

- 一级 singletonObjects：成品 Bean（完全初始化好的）。
- 二级 earlySingletonObjects：半成品 Bean（实例化但未填充属性）。
- 三级 singletonFactories：Bean 工厂（ObjectFactory），用于在需要时生成提前暴露的引用（可能是 AOP 代理）。

为什么需要三级而不是两级？核心原因是 AOP 代理。如果 A 需要被切面增强，那么 B 注入的 A 必须是代理对象而不是原始对象。三级缓存的 ObjectFactory 在调用 getObject() 时才决定是返回原始对象还是代理对象，从而保证注入的是最终代理。

本项目的情况：

整体基本没有循环依赖，依赖方向是单向的：Controller → Service → DAO，符合分层规范。这得益于构造器注入——如果出现循环依赖，Spring 会在启动时直接抛 BeanCurrentlyInCreationException，而不是像字段注入那样“悄悄”解决掉，问题会被更早发现。

但有一处隐性循环风险的苗头：CommentServiceImpl 里 import 了 CommentController（虽然最终没实际使用），这种“Service 反向依赖 Controller”的写法一旦变成真正的依赖，就会形成 Controller ↔ Service 的循环。可以通过 @Lazy 打破，但正确做法是删掉这个反向 import。

Q2.5 @Transactional 的底层原理是什么？

原理：基于 AOP 的动态代理。

1. Spring 容器启动时，@EnableTransactionManagement（本项目在启动类显式加了）导入 TransactionAutoConfiguration，注册 InfrastructureAdvisorAutoProxyCreator。
2. 它扫描到标了 @Transactional 的 Bean，为其生成代理对象（有接口用 JDK 动态代理，无接口用 CGLIB）。
3. 代理对象里的 TransactionInterceptor 在方法调用前后做增强：

```
// 伪代码
TransactionInfo txInfo = createTransactionIfNecessary(...); // 开启事务
try {
    result = invocation.proceed(); // 执行真正的业务方法
```

```

        commitTransactionAfterReturning(txInfo);    // 正常返回 → 提交
    } catch (Throwable ex) {
        completeTransactionAfterThrowing(txInfo, ex); // 异常 → 回滚
        throw ex;
    } finally {
        cleanupTransactionInfo(txInfo);    // 清理 ThreadLocal
    }
}

```

关键：事务的元信息（连接、隔离级别等）存放在 `ThreadLocal` 里，所以事务是线程绑定的。这直接解释了“为什么 `@Transactional` 在异步线程 / Kafka 消费线程里会失效”——不同线程，`ThreadLocal` 拿不到同一个事务上下文。

本项目里的体现：`SnappedUpServiceImpl.doSnapBusiness()` 标了 `@Transactional(rollbackFor = Exception.class)`，它是在 **Kafka 消费线程**里被调用的，所以这个事务是独立于 Web 请求线程的——这是正确的，因为异步落库本来就该有自己的事务。

Q2.6 为什么 `doSnapBusiness` 要写 `rollbackFor = Exception.class` ?

因为 Spring 默认只对 `RuntimeException` 和 `Error` 回滚，对受检异常（`Exception` 的子类但不是 `RuntimeException`）不回滚。

也就是说，如果代码抛了一个 `IOException`，Spring 会照常提交事务，造成“半个业务写进去了”的脏数据。

加上 `rollbackFor = Exception.class` 后，所有异常都会触发回滚。这是生产代码的默认最佳实践，建议所有 `@Transactional` 都这么写。

可以补充：反过来还有一种情况——业务上“预料之中的失败”希望不回滚（例如记录一条失败日志后继续），可以用 `noRollbackFor`，或者用 `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly(false)`。

Q2.7 `@Transactional` 有哪些失效场景？（必背）

1. 方法不是 **public**：Spring 的事务拦截只作用于 **public** 方法（proxy 模式下）。

2. 同类内部方法自调用：`this.methodB()` 绕过了代理对象，注解不生效。

解法：注入自己（`@Lazy`）、用 `AopContext.currentProxy()`、或把方法拆到另一个 Bean。

3. 异常被 **catch 掉了没抛出**：事务拦截器根本感知不到异常，自然不会回滚。

本项目就这么干了！`KafkaConsumer.listenSnap()` 里 `catch (Exception e) { log.error(...); throw e; }` —— 这个是对的，它重新抛出了。

但 `listenOrder()`、`listenVerifyCode()`、`listenRegCode()` 里是 `catch (Exception e) { log.error(...); }` 然后吞掉了异常。这三个方法里没有 `@Transactional`，所以没造成事务问题，但如果将来在里面加了事务注解，这个 `catch` 就会导致事务静默失效。这是个需要警惕的隐患。

4. 异常类型不匹配：抛受检异常但没写 `rollbackFor`。

5. 方法被 **final / static** 修饰：CGLIB 无法重写。

6. 多线程调用：子线程的异常主线程感知不到，且 `ThreadLocal` 不共享。

7. 数据库引擎不支持事务（如 `MyISAM`）。

8. 事务传播行为设置成 `NOT_SUPPORTED` / `NEVER`。

Q2.8 `@Transactional` 的传播行为有哪些？本项目用到哪种？

传播行为	含义
<code>REQUIRED</code> （默认）	有事务就加入，没有就新建
<code>REQUIRES_NEW</code>	总是新建事务，挂起外层事务
<code>SUPPORTS</code>	有就加入，没有就以非事务方式执行

NOT_SUPPORTED	以非事务方式执行，挂起外层事务
MANDATORY	必须已有事务，否则抛异常
NEVER	必须无事务，否则抛异常
NESTED	嵌套事务，用 Savepoint 实现，内层回滚不影响外层

本项目用的是默认的 **REQUIRED**。`doSnapBusiness` 是 Kafka 消费线程的入口，没有外层事务，所以会新建一个事务。

可以主动引申的场景：如果将来要在秒杀落库失败时"记录一条失败日志但日志本身要保留"，就需要把日志方法标为 **REQUIRES_NEW**，让它开一个独立事务，不被主事务回滚。

Q2.9 Filter、Interceptor、AOP 三者有什么区别？执行顺序是怎样的？

维度	Filter	Interceptor	AOP
所属规范	Servlet 规范	Spring MVC 规范	Spring AOP
作用层次	最外层，进入 DispatcherServlet 之前	DispatcherServlet 内部，Controller 前后	任意 Bean 的方法调用
能否拿到 Bean	不能（需手动从容器取）	能（本身就是 Bean）	能
能否拿到方法信息	不能	能拿到 handler 但拿不到参数值	能拿到参数、返回值、注解
异常处理	自己做	自己做（response 写回）	抛异常，交给 <code>@RestControllerAdvice</code>
本项目实例	<code>CorsConfig</code> 的 <code>CorsFilter</code>	<code>UserInterceptor</code> 、 <code>RateLimiterInterceptor</code>	<code>RateLimitAspect</code>

执行顺序（重点）：

```

Filter.doFilter 前置
├─> Interceptor.preHandle
│   └─> AOP @Around 前置
│       └─> Controller 方法执行
│           └─> AOP @Around 后置
├─> Interceptor.postHandle
├─> Interceptor.afterCompletion
└─> Filter.doFilter 后置
  
```

本项目的实际顺序：`CorsFilter`（`Ordered.HIGHEST_PRECEDENCE`，最优先，保证 OPTIONS 预检请求先返回）→ `UserInterceptor`（鉴权）→ `RateLimiterInterceptor`（秒杀限流）→ `RateLimitAspect`（注解限流）→ Controller。

项目里 `ToDo.java` 记录了作者对这两个方案的对比总结，这个理解非常到位，可以背下来：

HandlerInterceptor：Servlet 层拦截、需手写 response、需在 `WebAppConfiguration` 注册、无法拦截 Service 方法。

AOP：方法调用层拦截、抛异常交给 `@RestControllerAdvice` 统一处理、只需 `@Aspect + @Component` 自动生效、可拦截任意 Bean 方法。

Q2.10 拦截器的 `preHandle` / `postHandle` / `afterCompletion` 分别在什么时机执行？

- `preHandle`：**Controller 方法执行之前。返回 `true` 放行，返回 `false` 中断请求（后续拦截器和 Controller 都不执行）。本项目用它做 JWT 校验，失败时设置 401 并 `writeValue` 写回 JSON，返回 `false`。

- `postHandle`：Controller 方法执行之后、视图渲染之前。可以修改 `ModelAndView`。**Controller 抛异常时不会执行。**
- `afterCompletion`：整个请求完成之后（包括视图渲染完毕）。**无论是否抛异常都会执行**，适合做资源清理、耗时统计、日志记录。

追问：“多个拦截器时 preHandle 的返回 false 怎么影响其他拦截器？”

`preHandle` 按注册顺序执行，`postHandle` 和 `afterCompletion` 按**逆序**执行。如果第 2 个拦截器的 `preHandle` 返回 false，则第 3 个及其后不再执行，而第 1 个拦截器的 `afterCompletion` **会被调用**（因为它已经成功 `preHandle` 过了）。

本项目的一个设计问题：`UserInterceptor` 里对 `OPTIONS` 请求直接返回 true 放行。这是为了配合 CORS 预检，但正确做法应该由 `CorsFilter` 处理预检（Spring 的 `CorsFilter` 内部会识别预检请求并直接返回 200）。这里属于“双保险”，不算错，但稍显冗余。

Q2.11 拦截器的排除路径是怎么配置的？为什么 WebSocket 和 Swagger 要排除？

在 `WebAppConfiguration.addInterceptors()` 里配置：

```
registry.addInterceptor(userInterceptor)
    .addPathPatterns("/**")           // 默认全拦
    .excludePathPatterns("/api/user/login") // 登录
    .excludePathPatterns("/user/login")
    .excludePathPatterns("/api/user/register") // 注册
    .excludePathPatterns("/user/register")
    .excludePathPatterns("/api/user/forgot") // 忘记密码
    .excludePathPatterns("/user/sendCode")
    .excludePathPatterns("/user/sendRegCode")
    .excludePathPatterns("/user/forgot")
    .excludePathPatterns("/uploads/**") // 静态资源（头像图片）
    .excludePathPatterns("/swagger-ui.html") // Swagger
    .excludePathPatterns("/swagger-ui/**")
    .excludePathPatterns("/v3/api-docs/**")
    .excludePathPatterns("/api-docs/**")
    .excludePathPatterns("/webjars/**")
    .excludePathPatterns("/ws/**") // WebSocket 握手
    .excludePathPatterns("/init"); // 数据初始化
```

为什么必须排除：

- **登录/注册/忘记密码：**用户此时还没有 token，如果拦截就死锁了——永远登不进去。
- **/uploads/**：**头像图片是 `` 直接访问的，浏览器不会带 `Authorization` 头。**如果不排除，头像永远加载不出来。**这是一个非常容易被忽略的实战坑。
- **Swagger 相关：**Swagger UI 是一堆静态 JS 在浏览器里跑，同样不带 token。
- **/ws/**：**WebSocket 握手请求无法自定义 Header，所以 token 是通过 URL 查询参数 `?token=xxx` 传的，由 `UserWebSocketHandler` 自己校验。拦截器校验 Header 的逻辑在这里不适用。
- **/init：**数据初始化接口，测试用。

面试官会挑刺的地方：`/init` 这个接口被排除了鉴权，意味着**任何人只要访问 `http://你的域名/init` 就能往数据库批量插入测试用户数据**。这是一个严重的安全漏洞。生产环境必须：① 删除这个接口；② 或者加管理员权限校验；③ 至少加一个密钥参数。同样，`/findByStudentNum`（查询用户信息）也没有排除也没有权限控制——虽然要登录才能访问，但普通用户可以查任意学号的信息。

Q2.12 @RestControllerAdvice 全局异常处理是怎么工作的？本项目为什么只处理了一种异常？

原理： `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`。它本质是一个 AOP 增强（通过 `ExceptionHandlerExceptionResolver` 实现），Spring MVC 在处理请求抛异常时，会遍历所有 `@ControllerAdvice` Bean，找到匹配 `@ExceptionHandler` 声明的方法来调用。

本项目 `GlobalExceptionHandler` 只处理了 `RateLimitException`：

```
@ExceptionHandler(RateLimitException.class)
public ResponseEntity<Result<Void>> handleRateLimit(RateLimitException e) {
    Result<Void> result = new Result<>();
    result.setSuccess(false);
    result.setMessage(e.getMessage());
    return ResponseEntity.status(429).body(result);    // HTTP 429 Too Many Requests
}
```

为什么不返回 200 而返回 429？ 语义正确。`429 Too Many Requests` 是 HTTP 标准状态码，前端/网关/监控都能据此识别限流，比“HTTP 200 + 业务码 500”更规范。

对照：而 `RateLimiterInterceptor` 里是手动写 `response.setStatus(429)` 然后 `return false`——因为它工作在拦截器层，还没有进入 Controller，抛出的异常不会被 `@ExceptionHandler` 捕获（`preHandle` 抛的异常由 `HandlerExceptionResolver` 处理，但 `@ControllerAdvice` 是否能捕获取决于具体版本和配置，这里作者选择了手动写回更保险）。

可以主动指出的缺陷：缺少兜底的 `@ExceptionHandler(Exception.class)`，意味着任何未预期的异常（比如 `NullPointerException`）都会直接把 Spring Boot 默认的 Whitelabel Error Page / 500 错误栈返回给前端，可能泄露内部信息。生产项目应该有兜底处理器，统一返回“系统繁忙”并记录完整堆栈。

Q2.13 CORS 跨域是怎么解决的？`allowCredentials(true)` 和 `allowedOriginPattern("*")` 能同时用吗？

本项目用 `CorsConfig` 注册了一个 `CorsFilter`，并设置 `bean.setOrder(Ordered.HIGHEST_PRECEDENCE)` 让它最先执行。

```
config.setAllowCredentials(true);
config.addAllowedOriginPattern("*");    // 注意：不是 addAllowedOrigin("*")
config.addAllowedHeader("*");
config.addAllowedMethod("*");
config.setMaxAge(Duration.ofDays(5));
```

关键知识点（高频点）：

不能同时用 `addAllowedOriginPattern("*")` 和 `setAllowCredentials(true)`。因为 CORS 规范规定：当响应里带 `Access-Control-Allow-Credentials: true` 时，`Access-Control-Allow-Origin` 必须是具体的源，不能用通配符 `*`。否则浏览器会直接拒绝，报错：

```
Access to XMLHttpRequest at '...' from origin '...' has been blocked by CORS policy:
The value of the 'Access-Control-Allow-Origin' header in the response
must not be the wildcard '*' when the request's credentials mode is 'include'.
```

正确做法就是用 `addAllowedOriginPattern("*")`（Spring 5.3+ 提供）——它会根据请求的 `Origin` 头动态回写具体的源，从而既支持任意域名又兼容 credentials。

为什么要设 `allowCredentials(true)`？因为前端需要携带 Cookie。`docker-compose.yml` 里也能看到对应配置：
`SERVER_SERVLET_SESSION_COOKIE_SAME_SITE=none`。

为什么 `MaxAge` 设 5 天？预检请求（OPTIONS）结果缓存 5 天，浏览器在这期间不再发预检，减少请求数。

安全隐患：`allowedOriginPattern("*")` + `allowCredentials(true)` 意味着任意网站都能带着用户的 Cookie 跨域调用你的接口。如果将来真用了 Cookie 存凭证，这就是一个 CSRF 漏洞。当前因为用的是 JWT + `Authorization` 头，安全性尚可（攻击

者站点无法读取用户 localStorage 里的 token)，但最佳实践仍应是**白名单具体域名**。

第三级：MyBatis 与数据库

Q3.1 `#{}` 和`${}` 有什么区别？项目中哪里用了哪个？

维度	<code>#{} </code>	<code>\${} </code>
实现	<code>PreparedStatement</code> 的 <code>?</code> 占位符	字符串直接拼接
SQL 注入	安全（参数被转义）	危险（可注入）
预编译	是，数据库可复用执行计划	否，每次都是新 SQL
适用场景	所有参数值	表名、列名、ORDER BY 字段等 SQL 结构部分

本项目全部使用 `#{}` ，例如：

```
<select id="findByKey" resultMap="userResultMap">
  select id, student_num, user_name, email from `user`
  where 1=1
  <if test="keyWord != null and keyWord != ''">
    and (student_num like concat('%', #{keyWord}, '%')
        or user_name like concat('%', #{keyWord}, '%'))
  </if>
</select>
```

注意这里的一个细节：模糊查询没有写成 `'%${keyWord}%'`（那样会注入），而是用 `concat('%', #{keyWord}, '%')`，让 `%` 作为 SQL 字面量参与拼接，参数本身仍走预编译。这是正确且安全的写法。

追问：“LIKE '%xxx%' 能用到索引吗？”

不能。前置通配符 `%xxx` 会导致索引失效，只能全表扫描。优化方案：① 如果只是后缀匹配用 `'xxx%'`；② 数据量大时用全文索引（MySQL 的 `FULLTEXT` 或 ElasticSearch）；③ 或者用搜索引擎（ES 的 ik 分词器）专门做搜索。

Q3.2 自定义 TypeHandler 是怎么实现的？项目里为什么需要它？

项目实现了两个 `TypeHandler`：

① `OrderStatusTypeHandler` —— 枚举 ↔ 数字互转

```
public class OrderStatusTypeHandler extends BaseTypeHandler<OrderStatus> {
    // 【入库】枚举 → 数字 code
    public void setNonNullParameter(PreparedStatement ps, int i, OrderStatus p, JdbcType t) {
        ps.setInt(i, p.getCode());
    }
    // 【出库】数字 → 枚举
    public OrderStatus getNullableResult(ResultSet rs, String columnName) {
        return OrderStatus.getByCode(rs.getInt(columnName));
    }
    // ... 还有两个重载
}
```

为什么需要：订单状态在 Java 侧是枚举 `OrderStatus`（`PENDING_PAYMENT(0)/PAID(1)/CANCELED(2)/COMPLETED(3)`），在数据库里是 `int`。MyBatis 默认不认识这个枚举，需要 `TypeHandler` 做桥接。

为什么不用 MyBatis 自带的 `EnumTypeHandler`？默认的 `EnumTypeHandler` 存的是枚举的 `name` 字符串

（“`PENDING_PAYMENT`”），而 `EnumOrdinalTypeHandler` 存的是 `ordinal` 序号（0,1,2,3）。后者的问题是顺序敏感——一旦

在枚举中间插入一个新值，所有历史数据的含义全乱。所以本项目自己实现了"存业务 code"的方案，与枚举声明顺序解耦，是最健壮的做法。

追问："枚举为什么不用 name 存储？"用 name 存可读性好，但一旦重命名枚举常量，历史数据就失配。用显式定义的 code 存，只要不改 code 值就永远稳定。**生产上更推荐存 code + 在库里加 CHECK 约束或注释。**

② `JsonTypeHandler` —— JSON 字符串 ↔ `List<String>`

```
@MappedJdbcTypes(JdbcType.VARCHAR)
@MappedTypes(List.class)
public class JsonTypeHandler extends BaseTypeHandler<List<String>> { ... }
```

为什么需要：课程表 `course` 的 `grade`（可选年级）、`college`（可选学院）、`major`（可选专业）是多值字段，一门课可能对多个年级开放。项目把它们存成 MySQL 的 JSON 数组，例如 `["2022", "2023"]`。

Java 侧需要的是 `List<String>`，所以用 `TypeHandler` 在 JDBC 层做转换。

配套的查询用了 MySQL 的 JSON 函数：

```
AND JSON_CONTAINS(grade, JSON_ARRAY(#{grade}))
```

面试官会追问的设计问题：

"为什么用 JSON 字段而不是建关联表？"

优点：① 一条 SQL 就能查，不用 JOIN；② 字段可扩展。

缺点：① `JSON_CONTAINS` 无法使用普通索引，数据量大时是全表扫描（MySQL 8.0.17+ 可以建多值索引 `CAST(grade AS CHAR(20) ARRAY)` 缓解）；② 无法做外键约束，数据一致性靠应用层保证；③ 无法按年级做聚合统计。

结论：如果只是"标签式筛选且数据量小"，JSON 字段是合理取舍；如果要做统计报表或数据量上万，应该拆成 `course_grade` 关联表。**能主动说出这个 trade-off 是加分项。**

Q3.3 `<bind>` 标签是干什么的？为什么要用它而不是用 `${}` 算？

项目里分页用了 `<bind>`：

```
<bind name="currentIndex" value="(pageNum - 1) * pageSize"/>
limit #{currentIndex}, #{pageSize}
```

`<bind>` 的作用是在 OGNL 表达式上下文中创建一个变量，后续可以直接用 `#{currentIndex}` 引用。

为什么不用 `limit ${ (pageNum-1)*pageSize}, ${pageSize}`？ 因为是 `${}` 就是字符串拼接，虽然这里参数是 int 相对安全，但一旦有人把 `pageSize` 改成 String 类型，立刻变成注入漏洞。`<bind>` 生成的变量走的是 `#{}` 预编译，安全。

更值得说的点：`limit offset, size` 深分页问题

当 offset 很大时（如 `limit 1000000, 10`），MySQL 会扫描并丢弃前 100 万行，性能极差。

优化方案：① 游标分页（推荐）——记住上一页最后一条的 id，写成 `where id > #{lastId} order by id limit 10`，走主键索引，O(1) 定位；② 延迟关联——`select * from t join (select id from t limit 1000000,10) x using(id)`，先在覆盖索引上分页再回表。

Q3.4 `resultMap` 里的 `association` 是干什么的？

`association` 用于一对一（或"多对一"）的嵌套映射，把 JOIN 出来的多表字段映射到一个嵌套对象里。

项目 `UserCourseDAO.xml` 里的用法：

```
<resultMap id="userCourseResultMap" type="...UserCourseDO">
    <id column="id" property="id"/>
```

```

<result column="user_id" property="userId"/>
<result column="course_id" property="courseId"/>

<association property="user" javaType="com.student.server.model.User">
    <id column="user_id" property="id"/>
    <result column="user_name" property="userName"/>
    <result column="student_num" property="studentNum"/>
</association>

<association property="course" javaType="com.student.server.model.Course">
    <id column="course_id" property="id"/>
    <result column="name" property="name"/>
    <result column="teacher" property="teacher"/>
    ...
</association>
</resultMap>

```

对应 SQL:

```

select uc.id, uc.user_id, uc.course_id,
       u.id, u.user_name, u.student_num,
       c.id, c.course_num, c.name, c.teacher, ...
from user_course uc
left join user u on uc.user_id = u.id
left join course c on uc.course_id = c.id
where uc.user_id = #{userId}

```

关键点:

- 用 `<association>` 时, 同一个 `<id>` 列不能在 `resultMap` 里出现两次, 否则 MyBatis 会混淆嵌套对象的标识。
- `<association>` 是"一对一", `<collection>` 是"一对多" (如 `UserDO.courseList`) 。
- 性能陷阱: 这个 SQL 用 `left join`, 如果 `user_course` 表数据量大, 且 `u.id` / `c.id` 上没索引, 就是三次全表扫描。应该确认 `user_course(user_id)` 和 `user_course(course_id)` 都有索引。

追问: " `association` 嵌套查询 (select 属性) 和 JOIN 哪个好? "

`<association select="...">` 会触发 **N+1 查询问题** (查 1 次主表 + N 次子查询)。JOIN 是一次查询。除非开启了 MyBatis 的 `lazyLoadingEnabled` 延迟加载, 否则优先用 JOIN。本项目用的是 JOIN, 是对的。

Q3.5 MyBatis 的一级缓存和二级缓存讲一下, 本项目有开启吗?

一级缓存 (SqlSession 级别) :

- 作用域: 同一个 `SqlSession` 内。默认开启, 无法关闭 (只能改作用域为 `STATEMENT`) 。
- 实现: `SqlSession` 里的一个 `HashMap` (`localCache`), key 是 `StatementId + SQL + 参数` 。
- 失效时机: 执行 `INSERT/UPDATE/DELETE` (任何写操作都会 `clearLocalCache()`) 、手动 `sqlSession.clearCache()` 、`SqlSession` 关闭、事务提交。

二级缓存 (Mapper 命名空间级别) :

- 作用域: 跨 `SqlSession`, 同一 namespace 共享。
- 需在 XML 里手动开启 `<cache/>` 或在 Mapper 接口上加 `@CacheNamespace` 。本项目没有开启。
- 存储: 默认 `PerpetualCache` (内存), 可换成 Redis / Ehcache。
- 实体类必须实现 `Serializable` 。本项目 `UserDO` 实现了 `Serializable` , 具备开启条件。

为什么本项目不开二级缓存? 三个原因:

1. 脏读风险大: 二级缓存是 namespace 级别的, 如果多表 JOIN 查询涉及的表被另一个 namespace 修改, 缓存不会失效。

2. **分布式环境下失效复杂**：多实例部署时，一个实例改了数据，其他实例的本地二级缓存不会刷新。
3. **已有 Redis 统一缓存**：项目已经用 Redis 做了分布式缓存（`course:all`、用户信息），再开 MyBatis 二级缓存会导致**两级缓存并存、失效逻辑复杂化**。

结论：分布式系统里，MyBatis 二级缓存基本被 Redis 替代，不建议开启。这是一个很好的加分回答。

Q3.6 项目里分页是怎么做的？为什么评论用了 PageHelper 而用户列表没用？

项目里**存在两套分页实现**：

① 手写 LIMIT 分页（用户列表、课程列表）

```
<bind name="currentIndex" value="(pageNum - 1) * pageSize"/>
limit #{currentIndex}, #{pageSize}
```

总数单独查一次：

```
<select id="countAll" resultType="java.lang.Integer">
    select count(*) from `user`
</select>
```

② PageHelper 物理分页（评论列表）

```
Page<Comment> page = PageHelper.startPage(1, 10)
    .doSelectPage(() -> commentService.findAllComments());
Paging paging = new Paging(page.getPageNum(), page.getPageSize(),
    page.getPages(), page.getTotal(), page.getResult());
```

PageHelper 的原理：

1. `startPage()` 把分页参数塞进当前线程的 `ThreadLocal`（`PageHelper` 内部的 `LOCAL_PAGE`）。
2. 拦截 MyBatis 的 `Executor`，在 SQL 执行前**改写 SQL**：先查 `count`，再在原 SQL 后拼 `limit ?, ?`。
3. 返回的 `Page` 对象继承了 `ArrayList`，额外携带 `total`、`pages` 等元信息。

PageHelper 的经典坑（高分点）：

1. **`startPage` 后必须紧跟查询**，中间不能插入其他 MyBatis 调用，否则分页参数会“错位”应用到别的 SQL 上。
2. **`ThreadLocal` 未清理**：如果 `startPage` 之后没有执行查询（比如提前 `return`），分页参数会残留在 `ThreadLocal` 里，污染**同一个线程后续的查询**。线程池复用线程时这就是灾难。
3. 本项目 `CommentController` 里就调用了**两次 `findAllComments()`**！

```
// 第一次：查询并构建树形结构（不需要分页）
Result<List<Comment>> result = commentService.findAllComments();
// 第二次：又被 PageHelper 包了一层再查一遍
Page<Comment> page = PageHelper.startPage(1, 10)
    .doSelectPage(() -> commentService.findAllComments());
```

同一个方法里**把评论查了两遍**：一次是完整数据（用于返回业务数据），一次是 `PageHelper` 包装的（只为了拿 `total` 分页信息）。这是**性能浪费**——评论全量查了两遍，其中一遍还只是为了统计总数。而且 `PageHelper` 的 `count` 查询统计的是**分页前的总数**，这里逻辑上还算对，但完全可以把点击分页信息从第一次查询里拿。

正确做法：要么统一用手写 `LIMIT`（和 `user / course` 一致），要么用 `PageHelper` 但只查一次。**能主动指出这个问题，面试官会认为你有 `code review` 的能力。**

Q3.7 选课的库存扣减是怎么防超卖的？和秒杀有什么不同？

选课用的是“数据库乐观扣减 + 条件守卫”：

```
<!-- CourseDAO.xml -->
<update id="increasedSelectedCount" parameterType="java.lang.Long">
    update course
    set selected_count = selected_count + 1,
        remaining = remaining - 1
    where id = #{courseId}
        and remaining > 0          <!-- 关键守卫 -->
</update>
```

Service 层的逻辑：

```
// 1. 先查选课记录，防重复选课
UserCourseDO exist = userCourseDAO.getByUserIdAndCourseId(userId, courseId);
if (exist != null) { return "你已选过该课程，不可重复选课"; }

// 2. 查课程，判断是否已满
CourseDO courseDO = courseDAO.getByCourseId(courseId);
if (courseDO.getSelectedCount() >= courseDO.getTotalCapacity()) { return "课程已满"; }

// 3. 插入选课记录
userCourseDAO.insertUserCourse(userId, courseId);

// 4. 更新人数（SQL 带 remaining > 0 守卫）
int updateRows = courseDAO.increasedSelectedCount(courseId);
if (updateRows <= 0) { return "选课成功，但更新人数失败"; }
```

防超卖的机制：第 4 步的 `and remaining > 0` 是数据库层面的原子守卫——多个并发请求同时执行 UPDATE 时，InnoDB 的行锁保证串行化，第一个把 `remaining` 减到 0 后，后续请求的 `where remaining > 0` 匹配不到行，影响行数为 0，就失败了。

但这套逻辑有 4 个严重问题（面试官一定会挖）：

- 没有事务！** `insertUserCourse` 和 `increasedSelectedCount` 是两个独立操作。如果第 3 步成功、第 4 步失败（比如课程从 `remaining=1` 被并发抢到 0），就出现选课记录写进去了但人数没加的不一致数据。
修复：整个 `insertUserCourse` 方法加 `@Transactional(rollbackFor = Exception.class)`，并在第 4 步失败时抛异常触发回滚。
- "先查后改"有竞态：**第 2 步查 `selectedCount >= totalCapacity` 判断“已满”，在两个请求之间状态可能已变。不过第 4 步的 SQL 守卫兜底了，所以不算致命——但这是典型的“检查后使用”（TOCTOU）反模式，更好的写法是只依赖 UPDATE 的返回值，不做前置查询。
- 重复选课的检查同样有竞态：**两个并发请求可能都通过了第 1 步的检查。根本解法是在 `user_course(user_id, course_id)` 上建唯一索引，让数据库兜底——就像秒杀表做的那样。项目里秒杀表建了唯一索引，选课表却没建，这是一个明显的不一致。
- Redis 缓存更新在事务之外且非原子：**后面更新 `course:all` 那段代码是在数据库操作之后执行的，如果这里抛异常，数据库已提交但缓存没更新。

和秒杀的核心差异：

维度	选课	秒杀
库存扣减	MySQL 行锁 + <code>remaining > 0</code>	Redis Lua 原子脚本
流量削峰	无	Kafka 异步落库

防重复	应用层查询（有竞态）	Redis key + 数据库唯一索引双保险
吞吐量	受数据库行锁限制，几百 QPS	Redis 内存操作，可达数万 QPS

加分总结："选课属于中低并发场景，直接用数据库行锁最简洁可靠；秒杀是瞬时高并发，必须把压力挡在 Redis 层，数据库只做最终落库。这说明技术选型要看**并发量级**，而不是无脑上复杂方案。"

Q3.8 秒杀扣库存的 SQL 为什么要写 `and stock > 0` ?

```
<update id="reduceStock">
  update product
  set gmt_modified = now(),
  <if test="count > 0">
    stock = stock - #{count}
  </if>
  where id = #{id,jdbcType=BIGINT} and stock > 0
</update>
```

`and stock > 0` 的作用：把"判断库存是否充足"和"扣减库存"合并成**一条原子 SQL**，由数据库的行锁保证原子性。

对比错误写法：

```
// 错误！两步之间有竞态窗口，会超卖
ProductDO p = productDAO.selectById(id);
if (p.getStock() > 0) {
    productDAO.reduceStock(id, 1);
}
```

应用层判定永远存在"检查—执行"之间的时间窗口。**把判断下推到 SQL 里，让数据库在一个原子的 UPDATE 语句内完成"检查 + 修改"，就不可能有窗口。**

Service 层对应：

```
int stockRows = productDAO.reduceStock(productId, 1);
if (stockRows <= 0) {
    throw new RuntimeException("库存扣减失败，商品库存不足或不存在");
}
```

用返回值（影响行数）判断成败，而不是"查一次再判断"，这是并发编程的核心思想。

面试官会挑的刺（项目 `ToDo.java` 里作者自己也记下了）：

`<if test="count > 0">` 这个判断是**隐患**。如果 `count <= 0`，SQL 会变成：

```
update product set gmt_modified = now() where id = ? and stock > 0
```

——**只更新时间，没扣库存**，但影响行数 `> 0`，代码会判定"扣减成功"。虽然当前调用方传的永远是 1 不会触发，但这是**脆弱的防御**，应该直接删掉 `<if>`，写成 `stock = stock - #{count}`，并在 SQL 里再加 `and stock >= #{count}` 保证不会扣成负数。

Q3.9 Druid 连接池的配置和优势是什么？

项目用 `druid-spring-boot-3-starter`，配置在 `application.yml`：

```
spring:
  datasource:
    druid:
      stat-view-servlet:
        enabled: true
        url-pattern: /druid/*
```

```
login-username: admin
login-password: admin
```

Druid 相对 HikariCP 的优势:

- **内置监控**: SQL 执行统计、慢 SQL 记录、连接泄漏检测, 访问 `/druid/*` 可视化查看。这是本项目选它的主因。
- **SQL 防火墙**: 可以拦 SQL 注入。
- **功能丰富**: 支持连接池预热、PSCache、RemoveAbandoned。

劣势: 性能略逊于 HikariCP (HikariCP 是 Spring Boot 2.x 起的默认连接池, 以极简和极致性能著称)。

本项目的严重安全问题: `login-username: admin / login-password: admin` 是**硬编码的弱口令**, 而且 Druid 控制台能看所有 SQL 和表名。生产环境必须: ① 关闭 `stat-view-servlet`; ② 或者用强密码 + IP 白名单 (`allow` 配置项, 项目里是空的)。

另外 `application.yml` 里数据库密码、Redis 密码、邮箱授权码全是明文——这是一个必须提的安全问题。生产环境的做法:

- ① 用环境变量 (项目 `docker-compose.yml` 里其实已经这么做了); ② 用 Jasypt 加密; ③ 上配置中心 (Nacos/Apollo)。

Q3.10 MyBatis 的 Mapper 接口没有实现类, 是怎么工作的?

原理: JDK 动态代理 + 注册中心。

1. 启动时, `@MapperScan` (或 `MybatisAutoConfiguration` 的 `AutoConfiguredMapperScannerRegistrar`) 扫描到 `@Mapper` 接口所在包, 即本项目 `com.student.server.dao`。
2. 用 `ClassPathMapperScanner` 把每个接口注册成一个 `MapperFactoryBean`。
3. 获取 Bean 时, `MapperFactoryBean.getObject()` 调用 `MapperProxyFactory.newInstance()`, 用 **JDK 动态代理**生成 `MapperProxy` 代理对象。
4. 调用接口方法时, `MapperProxy.invoke()` 拦截, 根据 接口全限定名 + 方法名 拼出 `StatementId` (如 `com.student.server.dao.UserDAO.findByStudentNum`), 去 `Configuration.mappedStatements` 里找到对应的 `MappedStatement`, 然后走 Executor 执行 SQL、用 resultMap 映射结果。

关键点: 这就是为什么 **Mapper XML 的 namespace 必须和接口全限定名完全一致**, 且 `<select id="xxx">` 的 id 必须和方法名一致。

追问: "如果接口方法和 XML 不对应会怎样?"

启动时不会报错 (懒加载), 但**调用时**抛 `BindingException: Invalid bound statement (not found)`。这通常是三个原因:

- ① `mybatis.mapper-locations` 路径配错 (本项目是 `classpath:com/student/server/dao/*.xml`); ② XML 的 namespace 写错; ③ 方法名和 id 不一致。

第四级：认证授权 —— JWT 与拦截器

Q4.1 JWT 的结构是什么？本项目生成了哪些 claim？

JWT = Header.Payload.Signature，三段用 `.` 分隔，Base64Url 编码。

1. **Header**: `{"alg": "HS256", "typ": "JWT"}`，说明签名算法。
2. **Payload**: 存放 claim（声明）。
3. **Signature**: `HMACSHA256(base64(header) + "." + base64(payload), secretKey)`，防止篡改。

本项目生成的 token：

```
Jwts.builder()
    .subject(userInfo.getStudentNum())      // sub 标准声明
    .claim("id", userInfo.getId())
    .claim("userName", userInfo.getUserName())
    .claim("studentNum", userInfo.getStudentNum())
    .claim("nickName", userInfo.getNickName())
    .claim("email", userInfo.getEmail())
    .issuedAt(now)                        // iat
    .expiration(expiryDate)              // exp
    .signWith(secretKey)
    .compact();
```

签名密钥构造： `Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8))`，配置里 `jwt.secret = leijiasheng-jwt-secret-key-2026-server-app-256bit-abcdef123456`。长度 ≥ 32 字节，满足 HS256 的最低要求，否则 jjwt 0.12.x 会直接抛 `WeakKeyException`。

面试官会问的安全问题：

1. 密钥硬编码在 yml 里且提交到了 Git。任何人拿到这个密钥都能伪造任意用户身份的 token。必须改成环境变量 + 定期轮换。
2. Payload 里带了 `email` 和 `nickName`，这些是用户隐私信息。虽然 JWT 是 Base64 编码不是加密，任何人都能解开看内容。而且这些信息会随用户修改而变化，但 token 里的是签发时刻的快照，会变旧——所以 `UserController.person()` 里才要重新查一次数据库来拿最新数据。更合理的设计是 token 里只放 id 和 studentNum，减少 payload 体积和隐私泄露。

Q4.2 JWT 和 Session 的区别？为什么选 JWT？

维度	Session	JWT
状态	服务端有状态（存 Session）	服务端无状态
存储位置	服务端内存/Redis，客户端只存 sessionId	全部信息在客户端
横向扩展	需要 Session 共享（Redis / 粘性会话）	天然支持，任意实例都能验签
跨域/多端	依赖 Cookie，跨域麻烦	放 Header，天然支持多端
主动失效	容易（服务端删 Session）	困难（签发即有效，只能等过期）
性能	每次要查 Redis	纯 CPU 验签，无 IO
体积	小（只是一个 id）	大（几百字节到几 KB）

本项目选 JWT 的理由：

- **多端支持**：项目有 Web 端（Thymeleaf）和移动端 API（`/api/**`）。移动端 App / 小程序不方便使用 Cookie，JWT 放 Header 更合适。
- **无状态易扩展**：将来多实例部署不用做 Session 共享。
- **减少 Redis 依赖**：源自项目演进——早期用 `@EnableRedisHttpSession` 存 Session，后来改成了 JWT（配置类里的注释就是历史痕迹）。

加分回答：“JWT 不是银弹。它的**最大痛点是无法主动失效**——用户改了密码、账号被封、主动登出，旧 token 在过期前依然有效。本项目也确实存在这个问题：`/user/logout` 接口**什么都没做**，只是返回了一句‘退出登录成功’，服务端没有任何状态变更，token 依然有效。（前端如果只是删掉 localStorage 里的 token，那这个接口就是一个空壳。）”

Q4.3 JWT 无法主动失效，如果需要“退出登录立刻生效”怎么办？

方案一：Redis 黑名单（最常用）

```
// 登出时把 token 的 jti 或整个 token 存入 Redis，TTL 设为 token 剩余有效期
redisTemplate.opsForValue().set("jwt:blacklist:" + token, "1",
    remainSeconds, TimeUnit.SECONDS);

// 拦截器里每次校验后多查一次黑名单
if (Boolean.TRUE.equals(redisTemplate.hasKey("jwt:blacklist:" + token))) {
    return 401;
}
```

优点：实现简单、能立即生效。**缺点**：每次请求多一次 Redis 查询，**牺牲了 JWT “无状态”的最大优势**——绕了一圈又回到了 Session 模式。

方案二：双 Token（Access Token + Refresh Token）

- **Access Token**：有效期短（如 15 分钟），用于业务请求。
- **Refresh Token**：有效期长（如 7 天），只用于换取新的 Access Token，存在服务端（Redis/DB）可控。
- 登出时服务端删除 Refresh Token，Access Token 最多再活 15 分钟，把“永不失效”降低为“最多 15 分钟窗口”。

这是业界主流方案，兼顾了安全性和无状态性。

方案三：版本号 / 时间戳校验

```
// Redis 里存 user:version:{userId}，改密码/封号时 +1
// token 里带 version，校验时不匹配就拒绝
```

可以做到“改密码后所有旧 token 立即失效”。

本项目现状：三种方案都没实现。`jwt.expiration = 86400000`（24 小时），意味着**token 泄露后有整整 24 小时的攻击窗口，且无法撤销**。这是一个应该主动承认并给出改进方案的短板。

Q4.4 `UserInterceptor` 的完整鉴权流程讲一下。

```
@Override
public boolean preHandle(HttpServletRequest req, HttpServletResponse resp, Object handler)
    throws Exception {

    // ① 放行 CORS 预检请求
    if ("OPTIONS".equalsIgnoreCase(req.getMethod())) return true;

    // ② 取 Authorization 头
    String authHeader = req.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
```

```

        writeUnauthorized(resp); // 写 401 JSON
        return false;
    }

    // ③ 去掉 "Bearer " 前缀 (7 个字符)
    String token = authHeader.substring(7);

    // ④ 验签 + 查过期
    if (!jwtUtil.validateToken(token)) {
        writeUnauthorized(resp);
        return false;
    }

    // ⑤ 解析出用户信息, 塞进 request 属性
    UserInfo userInfo = jwtUtil.getUserInfoFromToken(token);
    req.setAttribute("currentUser", userInfo);
    return true;
}

```

设计亮点: 用 `request.setAttribute("currentUser", ...)` 传递用户上下文, Controller 通过 `(UserInfo)` `request.getAttribute("currentUser")` 取用。这是无状态方案下的标准做法 (对比 ThreadLocal 方案的优势: 请求结束自动释放, 不像 ThreadLocal 那样在异步场景下可能丢失或泄漏)。

存在的问题:

1. 没有把 `UserInfo` 存到 `ThreadLocal`, 所以 Service 层拿不到当前用户, 只能靠 Controller 传参。如果 Service 里需要"当前用户", 方案就受限。
2. JWT 解析进行了两次: `validateToken(token)` 内部调用了 `parseToken`, 之后 `getUserInfoFromToken(token)` 又解析一次。两次 Base64 解码 + 两次验签, 纯属浪费。应该合并成一次解析:

```

// 优化: 一次解析, 异常即无效
try {
    Claims claims = jwtUtil.parseToken(token);
    UserInfo userInfo = jwtUtil.buildUserInfo(claims);
    req.setAttribute("currentUser", userInfo);
    return true;
} catch (JwtException | IllegalArgumentException e) {
    writeUnauthorized(resp);
    return false;
}

```

3. 异常处理用 `e.printStackTrace()` 风格: `validateToken` 里 `catch (Exception e) { return false; }` 吞掉了具体原因 (是签名错? 还是过期?), 不便于排障。至少应该区分 `ExpiredJwtException` (提示"登录已过期, 请重新登录") 和其他异常 (提示"非法凭证")。
4. 静态 `ObjectMapper` 手写 JSON: `OBJECT_MAPPER.writeValue(response.getWriter(), result)` 是手动序列化。更优雅的做法是抛出一个 `UnauthorizedException`, 交给 `@RestControllerAdvice` 统一处理, 和 `RateLimitException` 保持一致。

Q4.5 密码是怎么加密存储的? BCrypt 和 MD5 有什么区别?

项目用 `spring-security-crypto` 的 `BCryptPasswordEncoder`:

```

private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

private String encodePassword(String rawPassword) {
    return passwordEncoder.encode(rawPassword);
}

private boolean checkPassword(String rawPassword, String encodedPassword) {

```

```
return passwordEncoder.matches(rawPassword, encodedPassword);
}
```

BCrypt 的核心特性（必背三个）：

1. **自带随机盐 (salt)**。每次 `encode("123456")` 结果都不同，盐被编码在哈希串里（格式：`$2a$10$随机盐22字符+哈希31字符`）。所以同一个密码，两个用户存的值不一样，彩虹表直接失效。
2. **可调节计算强度 (cost factor)**。`BCryptPasswordEncoder()` 默认强度 10，即 $2^{10} = 1024$ 轮迭代。强度每 +1，耗时翻倍。这是“故意设计的慢”，让暴力破解的代价指数级上升。
3. **防时序攻击**。`matches()` 内部用常量时间比较，不会因为“前几位匹配”而提前返回，避免攻击者通过响应时间逐位推断密码。

MD5 为什么不能用：

- **速度快**：现代 GPU 每秒能算几十亿次 MD5，“故意慢”的设计正好相反。
- **无盐**：相同密码哈希值相同，彩虹表一查就出。项目测试代码里能看到旧方案：`DigestUtils.md5DigestAsHex((rawPassword + "ljs_zwy").getBytes()).toUpperCase()`——虽然加了固定后缀盐，但盐是全局固定的，一旦泄露（或从代码里看到）就失去意义。
- **已被证明不安全**：MD5 存在严重碰撞漏洞。

本项目的真实缺陷（面试官很可能会发现）：

`UserServiceImplLoginTest` 里仍在用 MD5 构造测试数据：

```
String expectedMd5 = DigestUtils.md5DigestAsHex(saltPwd.getBytes()).toUpperCase();
```

而生产代码已经改成 BCrypt。这个测试要么会失败，要么是通过 mock 绕过了真实校验——从代码看，`login()` 里用的是 `passwordEncoder.matches()`，用 MD5 值真实构造的 password 是匹配不上的。说明这个测试用例已经与实现脱节（属于需要更新但没更新的技术债）。如果面试官问“你的测试通过了吗”，这是必须诚实面对的问题。

其他密码存储建议：

- Argon2（2015 年密码哈希竞赛冠军，抗 GPU/ASIC，是当前的推荐首选）。
- PBKDF2（NIST/FIPS 认可，兼容性好）。
- scrypt（内存硬，抗硬件加速）。

BCrypt 的局限：输入截断到 72 字节，超过部分被忽略（意味着超长密码的后半段无效），且不支持密码长度超过 72 字节的严格校验。

Q4.6 注册和忘记密码的验证码流程是怎么设计的？

注册验证码流程（`buildRegCode`）：

- ① 校验学号是否已注册
 - 先查 Redis (`key = studentNum`)，没有再查数据库
 - 已存在 → 返回“该学号已注册”
- ② 生成验证码

```
Random random = new Random();
int code = random.nextInt(10000);
String codeNum = String.format("%04d", code); // 4 位补零
```
- ③ 写入 Redis，5 分钟过期

```
key = "register:code:{studentNum}"
redisTemplate.opsForValue().set(key, codeNum, 5, TimeUnit.MINUTES);
```
- ④ 通过 Kafka 异步发邮件

```
kafkaTemplate.send("regCode", email, codeNum);
```

注册时的校验：

```
String regCodeKey = RedisConstant.EMAIL_REGISTER_CODE_PREFIX + studentNum;
String regCode = (String) redisTemplate.opsForValue().get(regCodeKey);
if (!code.equals(regCode)) { return "验证码错误"; }
```

为什么用 Kafka 异步发邮件？发邮件是 **IO 密集 + 高延迟**操作（QQ SMTP 通常 1~3 秒）。如果同步发，用户点击“发送验证码”要等 3 秒；而且 SMTP 服务器不稳定时会导致整个接口超时。**异步化后接口立即返回，邮件在后台发送，用户体验和系统稳定性都大幅提升。**

面试官会挖的坑（一堆）：

1. 验证码用 **Random** 而不是 **SecureRandom**。`java.util.Random` 是**可预测的伪随机数**（线性同余法），如果能观测到几个输出值就能推算出种子和后续序列。验证码这种安全敏感场景应该用 `java.security.SecureRandom`。
2. **4 位数字验证码只有 10000 种可能**，配合“不限次数的验证尝试”就非常危险——攻击者可以用脚本在 5 分钟内爆破。虽然 `/user/forgot` 有 `@RateLimit(windowMs=60000, maxRequests=2)` 限制，但那是**针对发验证码的接口**，**校验验证码的接口限流策略需要单独评估**。
3. 验证码校验后没有删除！`register()` 和 `resetPwd()` 校验通过后，**Redis 里的验证码 key 没有被 delete**，在 5 分钟内可以**重复使用同一个验证码**。正确做法是校验成功后立即删除（或者用 Lua 脚本做“校验+删除”原子操作，防止并发重放）。
4. 注册校验里有 **NPE 风险**：`if (!code.equals(regCode))` —— 如果 `code` 参数为 null（虽然 `@RequestParam` 默认 `required=true` 会拦，但如果显式传空串），`code.equals(...)` 是安全的（不为 null 时），但 `regCode` 为 null 时会走 `equals` 返回 false，逻辑上没问题。**但反过来 `resetPwd` 里没有任何非空校验**，如果 `studentNum` 对应的用户不存在，`userDO.getPassword()` 会直接 **NPE**。
5. 忘记密码竟然是把密码重置为固定值 123456：

```
// UserServiceImpl.resetPwd()
if (checkPassword("123456", userDO.getPassword())) {
    result.setMessage("重置密码失败，密码已经为123456");
    return result;
}
String resetEncodedPwd = encodePassword("123456");
int setResult = userDao.updatePwd(resetEncodedPwd, studentNum);
```

这是极其危险的设计。“忘记密码”应该让用户**设置新密码**，而不是重置成一个所有人都知道的固定值。这意味着：**任何能拿到用户学号 + 邮箱的人（或猜到验证码），都能把账号密码改成 123456，然后登录进去。**这是一个必须修正的安全漏洞。**能主动承认并指出这一点，比试图掩饰要好得多。**

6. 验证码没有“尝试次数限制”。应该加一个计数器（如 `register:code:try:{studentNum}`），错误 5 次就作废验证码。

Q4.7 文件上传（头像）是怎么实现的？有什么安全隐患？

实现：

```
// ① 三重校验
if (file.isEmpty()) → "请选择头像文件"
String contentType = file.getContentType();
if (!jpeg && !png && !gif) → "仅支持 JPG、PNG、GIF 格式"
if (file.getSize() > 2 * 1024 * 1024) → "头像大小不能超过 2MB"

// ② 目录不存在则创建
String uploadDir = this.uploadDir + "/uploads/avatar/";
Path uploadPath = Paths.get(uploadDir);
if (!Files.exists(uploadPath)) Files.createDirectories(uploadPath);

// ③ 重命名：学号_随机8位.扩展名
```

```
String fileName = studentNum + "_" + UUID.randomUUID().toString().substring(0, 8) + ext;

// ④ 落盘
file.transferTo(new File(uploadDir + fileName));

// ⑤ 更新数据库 + 缓存
userService.updateAvatar(studentNum, "/uploads/avatar/" + fileName);
```

设计亮点：

- **重命名文件**：避免了原始文件名带来的风险（`../../etc/passwd` 路径穿越、同名覆盖、“中文名乱码”）。
- **限制大小**：2MB 上限，防止大文件打满磁盘。但这是应用层限制，攻击者仍可以在 TCP 层发超大请求撑爆内存——应该在 `application.yml` 里配 `spring.servlet.multipart.max-file-size` 和 `max-request-size`，让 Tomcat 在读取阶段就截断。
- **访问路径分离**：文件存在 `{user.dir}/uploads/avatar/`，通过 `addResourceHandlers` 把 `/uploads/**` 映射成静态资源，并在拦截器里排除了 `/uploads/**`，否则图片加载不出来。

安全隐患（面试官必挖）：

1. **getContentType() 来自客户端，完全可伪造**。攻击者可以把一个 `.jsp` 或 `.html` 文件的 Content-Type 伪造成 `image/png` 传上来。

后果：如果服务器配置了把 `/uploads/**` 当 JSP 解析（虽然 Spring Boot 默认不会），就是直接 **getshell**；即使用户打开一个恶意的 SVG/HTML，也可能引发 **存储型 XSS**。

正确做法：① 校验文件头魔数（读前几个字节，`FF D8 FF` = JPEG，`89 50 4E 47` = PNG）；② 用 `ImageIO.read()` 尝试解码，失败就是假图片；③ 用 `Files.probeContentType()` 按实际内容判断；④ 把上传目录放在 **Web 根目录之外**（本项目 `upload.dir` 默认是 `user.dir`，是一个外部目录，这点做得对）；⑤ 响应头加 `Content-Disposition: attachment` 或 `X-Content-Type-Options: nosniff`。

2. **扩展名从原始文件名截取，未做白名单**：`ext = originalName.substring(originalName.lastIndexOf("."))` 直接把用户给的后缀拼到新文件名上。如果传 `a.jsp`，新名字就变成 `2021001_ab12cd34.jsp`。**应该用白名单映射**：

```
// 推荐：根据校验后的真实类型决定扩展名，忽略用户输入
String safeExt = switch (detectedType) {
    case "image/jpeg" -> ".jpg";
    case "image/png" -> ".png";
    case "image/gif" -> ".gif";
    default -> throw new IllegalArgumentException("不支持的格式");
};
```

3. **没有清理旧头像**。用户每次上传都会生成新文件，旧文件永久留在磁盘上——**磁盘泄漏**。应该在更新数据库成功后删除旧头像文件。
4. **file.transferTo() 没有防目录穿越**。Spring 的 `StandardMultipartFile.transferTo()` 内部会做检查（`isAbsolute` 且非相对路径时会校验），但更稳妥的做法是显式用 `Paths.get(uploadDir).resolve(fileName).normalize()` 并断言结果仍在 `uploadDir` 下。
5. **没有做图片压缩/缩略图**。头像只需要 200×200，用户上传 2MB 的原图很浪费存储和带宽。应该用 Thumbnailator 或 ImageIO 压缩。
6. **单机存储无法横向扩展**。文件存在本地 `user.dir`，多实例部署时用户 A 的头像可能在实例 1 上，访问时负载均衡到实例 2 就 404。**正确做法是存对象存储（OSS / MinIO / S3）**。

Q4.8 为什么项目引入了 `spring-security-crypto` 却没引入完整版 Spring Security?

因为只需要密码加密功能，不需要 Security 的整套安全框架。

`spring-security-crypto` 是一个**零依赖的独立模块**，只包含密码编码器（BCrypt、Argon2、PBKDF2、SCrypt）和对称加密工具。可以单独使用，不引入 Filter 链。

如果引入了 `spring-boot-starter-security` 会怎样？

- 自动配置会开启**默认安全策略**：所有请求都需要认证，未登录会重定向到 `/login`（一个自动生成的登录页）。
- 会注册十几个 Filter（`UsernamePasswordAuthenticationFilter`、`BasicAuthenticationFilter` 等）。
- 会生成随机密码并打印到控制台。
- 需要额外写 `SecurityFilterChain` 配置全部放行，才能和现有的 `UserInterceptor` 共存。

所以项目的选择是对的：只取所需，避免了框架冲突和复杂度。这是一个很好的“技术选型克制”的例子。

但要主动承认局限：自研拦截器 + JWT 的方案，缺少 Spring Security 提供的能力——CSRF 防护、安全响应头（`X-Frame-Options`、`HSTS`）、方法级权限（`@PreAuthorize`）、会话固定攻击防护、认证事件监听。**如果项目要做成生产级系统，建议还是上 Spring Security + JWT 集成，用 `OncePerRequestFilter` 替换自定义拦截器。**

另一个代码细节问题：

```
private final BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();
```

这是**字段初始化**而不是 Spring 注入，而且在 `UserServiceImpl` 和 `InitController` 里各 **new 了一个**。问题：① 无法通过配置调整 cost factor；② 无法在测试中 mock 替换；③ 多实例浪费内存。**正确做法是注册为 Bean：**

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);    // 显式指定强度
}
```

然后构造器注入使用。

第五级：Redis 与缓存体系

Q5.1 项目里自定义了 `RedisTemplate`，默认的有什么问题？

`RedisTemplateInit` 里做了三件事：

```
@Bean
public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory) {
    RedisTemplate<String, Object> template = new RedisTemplate<>();
    template.setConnectionFactory(factory);

    // ① Key 用 String 序列化
    RedisSerializer<String> keySerializer = new StringRedisSerializer();
    template.setKeySerializer(keySerializer);
    template.setHashKeySerializer(keySerializer);

    // ② Value 用 Jackson JSON 序列化，并处理 Java 8 时间类型
    ObjectMapper om = new ObjectMapper();
    om.registerModule(new JavaTimeModule()); // 关键：支持 LocalDateTime
    om.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    om.activateDefaultTyping(
        LaissezFaireSubTypeValidator.instance,
        ObjectMapper.DefaultTyping.NON_FINAL,
        JsonTypeInfo.As.PROPERTY
    );
    GenericJackson2JsonRedisSerializer jsonSerializer = new GenericJackson2JsonRedisSerializer(om);

    template.setValueSerializer(jsonSerializer);
    template.setHashValueSerializer(jsonSerializer);
    template.afterPropertiesSet();
    return template;
}
```

为什么必须自定义？

Spring Boot 的 `RedisAutoConfiguration` 提供的默认 `RedisTemplate` 用的是 `JdkSerializationRedisSerializer`，它有三个致命问题：

1. **Key 是乱码**：JDK 序列化后 Redis 里存的 key 是一串 `\xac\xed\x00\x05t\x00\x06...` 的二进制。
后果：用 `redis-cli` 去看数据完全看不懂、无法排查问题、无法用 `KEYS / SCAN` 模式匹配，其他语言的服务也读不了。
2. **必须实现 `Serializable`**：所有存进去的对象都要实现序列化接口。
3. **体积大、性能差**：JDK 序列化会写入完整的类描述信息，比 JSON 大 2~3 倍。

自定义后的效果：Key 是可读字符串（`course:all`、`product:stock:1`），Value 是可读 JSON。

三个容易被追问的技术细节：

① `activateDefaultTyping` 是干什么的？为什么必须开？

它会在 JSON 里额外写入类型信息，例如：

```
{"@class":"com.student.server.dataobject.UserDO","id":1,"studentNum":"2021001",...}
```

为什么必须开：因为 `RedisTemplate<String, Object>` 的 value 是 `Object` 类型。反序列化时 Jackson 不知道要还原成什么类——如果不开 default typing，它只会返回一个 `LinkedHashMap`，那么这行代码就会直接抛 `ClassCastException`：

```
UserDO userDO = (UserDO) redisTemplate.opsForValue().get(studentNum); // 崩！
```

开了之后，Jackson 读到 `@class` 字段，就知道要还原成 `UserDO`，强转才能成功。

但这引入了一个著名的安全漏洞：Jackson 反序列化 RCE。

`activateDefaultTyping + LaissezFaireSubTypeValidator`（“放任不管”的校验器）= **允许反序列化任意类**。如果攻击者能往 Redis 写入一条 `{"@class":"com.sun.rowset.JdbcRowSetImpl", ...}` 的数据，就能触发 JNDI 注入，实现远程代码执行（这就是经典的 Fastjson/Jackson 反序列化漏洞家族）。

正确做法：

① 用 `BasicPolymorphicTypeValidator` 做**白名单**，只允许 `com.student.server` 包下的类：

```
om.activateDefaultTyping(BasicPolymorphicTypeValidator.builder().allowIfSubType("com.student.server.").build(), ...)
```

② 或者存 Redis 前**手动序列化**成 **String**，用 `RedisTemplate<String, String>` + 显式 `JSON.parseObject(json, UserDO.class)`，完全绕开 polymorphic typing。

③ **Redis 必须设置密码 + 禁止外网访问**（项目里 Redis 是 `47.99.86.70:6379` 的公网地址且密码明文写在 yml 里，风险很高）。

② 为什么要注册 `JavaTimeModule`？

因为 `UserDO`、`ProductDO` 里都有 `LocalDateTime` `gmtCreated`。Jackson 默认**不认识 Java 8 时间类型**，会抛：

```
Java 8 date/time type `java.time.LocalDateTime` not supported by default:
add Module "com.fasterxml.jackson.datatype:jackson-datatype-jsr310" to enable handling
```

注册 `JavaTimeModule` 后才支持。`disable(WRITE_DATES_AS_TIMESTAMPS)` 则是让时间序列化成 `"2026-09-28 12:00:00"` 而不是时间戳数组 `[2026,9,28,12,0,0]`，可读性更好。

还有个坑：`LocalDateTime` 在 Jackson 2.15+ 默认序列化成 ISO 格式 `"2026-09-28T12:00:00"`（带 T）。项目在 `UserDO` 的字段上加了 `@JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")` 来覆盖。但问题来了——这个注解是加在 **DO 上的**，而 Redis 序列化用的是自定义的 `ObjectMapper`（那个 `om` 是 `new` 出来的，不是 Spring 容器里配置了注解支持的那个）。实际上 Jackson 的 `@JsonFormat` 是靠 `JacksonAnnotationIntrospector` 生效的，默认就有，所以能生效。但如果两边配置不一致，就会出现“接口返回的时间格式和 Redis 里存的时间格式不一样”的诡异问题，这是分布式缓存项目里非常常见的坑。

③ 为什么 Key 用 `StringRedisSerializer` 而 Value 用 JSON？

Key 不存复杂对象，永远是字符串，用 `StringRedisSerializer` 最直观可读。

Value 可能是对象（`UserDO`、`Course`、数字库存），所以需要 JSON。

额外注意：项目里还有 `StringRedisTemplate`（如 `RateLimitAspect`、`UserWebSocketHandler`）和 `RedisTemplate<String, Object>`（如 `UserServiceImpl`）**并存使用**。

`StringRedisTemplate` 是 Spring Boot 自动配置的，Key 和 Value 都用 String 序列化。两种模板操作同一个 key 时，序列化方式必须一致，否则会出现“用 A 存的数据 B 读不到”的诡异问题。本项目里两者的 key 前缀是错开的（限流用 `rate:limiter:*`，业务用 `studentNum`），所以没踩坑，但这是需要警惕的设计。

Q5.2 缓存穿透、缓存击穿、缓存雪崩分别是什么？本项目怎么处理的？

① 缓存穿透：查一个不存在的数据

问题：请求查 `studentNum = "9999999"`（不存在），Redis 没有 → 查数据库也没有 → **不写缓存** → 下次请求还是打数据库。攻击者可以用随机学号疯狂刷，把数据库打挂。

本项目已经处理了！ `UserServiceImpl.login()` 里有这段：

```

userDO = userDAO.findByStudentNum(studentNum);
if (userDO == null) {
    // 缓存空对象，过期时间 6 分钟（短过期）
    redisTemplate.opsForValue().set(studentNum, new UserDO(), 6, TimeUnit.MINUTES);
    result.setMessage("登录账户不存在");
    return result;
}
// 后面用 id <= 0 判断空对象
if (userDO.getId() <= 0) {
    result.setMessage("登录账户不存在");
    return result;
}

```

这是标准的“缓存空对象”方案，而且过期时间用了 6 分钟而不是 6 小时——这是一个很讲究的细节：空对象只用于挡住短时间的攻击洪峰，一旦真实数据被创建，最多 6 分钟就能查到。

另一种方案是布隆过滤器（Bloom Filter）：把所有存在的学号预先放进一个位数组。查询时先过布隆过滤器，说不存在就一定不存在，直接返回。优点是不占用缓存空间，缺点是有误判率（说存在可能实际不存在）、无法删除元素。

② 缓存击穿：某个热点 key 突然过期

问题：秒杀商品库存 key 过期的一瞬间，成千上万请求同时发现缓存没了，全部去查数据库。

本项目的隐患就在这里：`initStock()` 在启动时预热库存，TTL 设为 `STOCK_EXPIRE_SEC = 21600`（6 小时），6 小时后 key 会全部过期。而此时如果活动还在进行，所有请求都会打到数据库。更糟的是，Lua 脚本里：

```

local stock = tonumber(redis.call('get', stockKey) or 0)
if stock <= 0 then return -1 end -- 返回"商品已抢光"

```

key 不存在时 `get` 返回 `nil`，`nil or 0` 得到 0，直接判定“已抢光”——用户会看到一个莫名其妙的“商品已抢光”，但实际库存还有很多。这是一个确实存在的 bug。

解决方案（项目 `ToDo.java` 里作者也想到了）：

```

-- Lua 脚本里加回源兜底
if redis.call('exists', stockKey) == 0 then
    redis.call('set', stockKey, loadFromDB(productId))
end

```

或者更完善的方案：互斥锁重建（只让一个线程去加载，其他线程等待）或逻辑过期（value 里存过期时间，不设 Redis TTL，发现逻辑过期后异步刷新，返回旧值）。

③ 缓存雪崩：大量 key 同时过期或 Redis 服务宕机

本项目有两个雪崩风险点：

1. `initStock()` 在启动时把所有商品库存一次性写入，TTL 全是 21600 秒——意味着 6 小时后所有库存 key 在同一秒钟集体过期，完美的雪崩条件。

修复：给 TTL 加随机扰动，例如 `21600 + Random.nextInt(600)`，把过期时间打散。

2. `course:all` 是一个 Hash 存了所有课程，TTL 3 小时。虽然它是一个 key 不会“雪崩”，但它是热点 key，一旦过期，所有课程查询都会去数据库全表扫描（`courseDAO.findAllCourse()`）。

修复：热点 key 加互斥锁重建，或者改成逻辑过期 + 异步刷新。

Redis 宕机的兜底：项目没有做任何降级。生产环境应该：① Redis 主从 + 哨兵/Cluster 保证高可用；② 本地缓存（Caffeine）作为二级兜底；③ 关键接口加熔断（Sentinel / Hystrix），Redis 挂了直接降级返回“系统繁忙”而不是拖垮数据库。

Q5.3 项目用了两级缓存（Caffeine + Redis），为什么？各自适合什么场景？

项目里 Caffeine 用在了两个地方：

缓存	位置	内容	配置
UserCache	cache/UserCache.java	Long userId → UserDO	maximumSize=10000 , expireAfterWrite=1h
UserCache.countCache	同上	总用户数	expireAfterWrite=10min
productCache	SnappedUpServiceImpl	Long productId → ProductDO	maximumSize=500 , expireAfterWrite=10min

Caffeine vs Redis 对比：

维度	Caffeine（本地）	Redis（分布式）
访问速度	纳秒级（JVM 堆内，无网络 IO）	毫秒级（网络往返 + 序列化）
容量	受 JVM 堆限制（MB 级）	可到 TB 级
一致性	多实例间不一致	全局一致
持久化	进程重启即丢失	支持 RDB/AOF
适用场景	极少变更的热点数据（商品信息、配置、字典）	需要共享、需要持久化的数据（用户会话、库存）

本项目为什么给商品加 Caffeine？

秒杀场景下，每次抢购都要查一次商品信息（拿价格）。如果每次都走 Redis：

序列化 → 网络往返 → 反序列化 ≈ 0.5 ~ 1ms。

在 1 万 QPS 下就是 1 万次 Redis 请求，Redis 直接成为瓶颈。

改用 Caffeine 后，访问耗时降到纳秒级（本地堆内存直接读），且完全不占用 Redis 连接。

但这里有一个必须指出的严重问题：

```
ProductDO productDO = productCache.get(productId, key -> productDAO.selectById(key));
String productJsonStr = JSON.toJSONString(productDO);
kafkaTemplate.send(Topics.KAFKA_SNAP_TOPIC, String.valueOf(userId), productJsonStr);
```

把 ProductDO（含 price）缓存 10 分钟并序列化发给 Kafka，意味着订单价格用的是最多 10 分钟前的旧价格。

后果：如果运营在活动期间调整了商品价格（从 99 元改成 199 元），接下来 10 分钟内的所有订单都会按 99 元成交，造成资金损失。这是一个真实的业务事故。

修复方案：

- 只缓存不变的字段：商品名、图片 URL 可以缓存，价格必须每次从 Redis 或数据库实时取。
- 价格改由服务端在结算时重新计算，不信任客户端传来的价格。
- 或者降级 Caffeine 的 expireAfterWrite 到秒级 + 加缓存失效通知（价格变更时主动 invalidate）。

多实例下的缓存一致性问题（进阶）：

Caffeine 是进程内缓存，实例 A 更新了数据，实例 B 的 Caffeine 不会知道。项目现在的做法是“写数据库后调 userCache.evict(userId)”，但这只能清掉当前实例的缓存。

生产级方案：① 用 Redis Pub/Sub 广播失效消息，所有实例订阅后清本地缓存；② 用 Redisson 的 RLocalCachedMap（分布式本地缓存映射，自动同步失效）；③ 或者干脆缩短本地缓存 TTL 到可接受的不一致窗口（如 30 秒）。

本项目 **UserCache** 的 **1 小时 TTL** 在多实例下会导致长达 **1 小时的数据不一致**——比如用户改了昵称，在线列表里可能还是旧昵称。

Q5.4 Caffeine 的底层原理是什么？它为什么比 Guava Cache 快？

核心：**W-TinyLFU 淘汰算法**。

LFU（最不经常使用）的问题：只看访问频次，会导致“历史热点”长期占据缓存。比如某个商品昨天被访问了 10 万次，今天已经不热了，但它的计数还是最高，赖着不走。

LRU（最近最少使用）的问题：只看最近访问时间，容易被“一次性扫描”污染——一次全表遍历会把所有热点数据挤出去（缓存污染）。

W-TinyLFU 的解法 = Window + TinyLFU：

- Window Cache (LRU, 占 1%)**：新来的元素先进这个小的 LRU 窗口，保证新元素有机会被访问到（解决“新元素刚进来就因为频次低被淘汰”的问题）。
- Main Cache (SLRU, 占 99%)**：分为 Probation（考察区 20%）和 Protected（保护区 80%）。
- TinyLFU 准入过滤器**：用 **Count-Min Sketch**（一种概率数据结构）统计频次。当 Window 满了要淘汰元素时，让“被淘汰者”和“Main 里的候选受害者”比频次，频次低的淘汰——所以叫 TinyLFU（“微小的 LFU 准入策略”）。
- Count-Min Sketch 的空间优化**：用 4-bit 计数器 + 定期衰减（reset）机制，把巨大的频次统计压缩到极小空间，同时解决了“老数据频次永不过期”的问题。

为什么比 Guava Cache 快：

- 无锁读**：Caffeine 用 **环形缓冲区 (Ring Buffer)** + **异步批处理**记录访问事件，读操作完全不加锁，写（淘汰）操作在独立线程里批量处理。
- 更优的淘汰命中率**：官方 benchmark 显示 W-TinyLFU 的命中率显著高于 LRU。
- 消除 GC 压力**：支持弱引用/软引用配置、异步加载。

本项目 API 用法：

```
Cache<Long, ProductDO> productCache = Caffeine.newBuilder()
    .maximumSize(500) // 容量上限（按条数）
    .expireAfterWrite(10, TimeUnit.MINUTES) // 写入后 10 分钟过期
    .build();

// get(key, mappingFunction): 不存在则用函数加载（类似 computeIfAbsent）
ProductDO productDO = productCache.get(productId, key -> productDAO.selectById(key));
```

三种过期策略的区别（高频追问）：

- expireAfterWrite**：写入后计时，期间即使被读也不重置。本项目用的是这个——适合“数据会变化，要保证一定时间内必然刷新”的场景。
- expireAfterAccess**：每次访问后重置计时，热点数据永不过期——适合纯读多写少的数据。
- refreshAfterWrite**：到期后异步刷新，刷新期间返回旧值。适合“不希望有请求等待加载”的场景（配合 `.buildAsync()`）。

Q5.5 项目用 Redis 缓存了哪些数据？key 是怎么设计的？

Key 模式	类型	内容	TTL	用途
{studentNum}	String	UserDO (JSON)	6 小时	用户信息缓存（正常）
{studentNum}	String	空 UserDO (id=0)	6 分钟	缓存穿透防护

<code>course:all</code>	Hash	<code>courseId</code> → <code>Course</code> JSON	3 小时	全量课程缓存
<code>product:stock:{id}</code>	String	剩余库存数字	21600 秒	秒杀库存
<code>user:snapped:{userId}:{productId}</code>	String	1	21600 秒	一人一单去重
<code>emailCode:{studentNum}</code>	String	4 位验证码	5 分钟	找回密码验证码
<code>register:code:{studentNum}</code>	String	4 位验证码	5 分钟	注册验证码
<code>online:users</code>	ZSet	<code>userId</code> , <code>score</code> = 心跳时间戳	无	在线用户
<code>rate:limiter:anno:{类}:{方法}:{userId/ip}</code>	ZSet	请求时间戳	窗口/1000+1 秒	AOP 注解限流
<code>rate:limiter:snapped:user:{userId}</code>	ZSet	请求时间戳	5 秒	秒杀拦截器限流
<code>order:auto:increment:{yyyyMMdd}</code>	String	自增序列	6 小时	订单号生成 (Redisson)
<code>jwt:blacklist:...</code>	—	未实现	—	登出黑名单 (待做)

一个明显的 key 设计问题：用户缓存直接用 `studentNum` 作为 key，没有命名空间前缀。

```
redisTemplate.opsForValue().set(studentNum, userDO, 6, TimeUnit.HOURS);
```

后果：

1. 与其他业务冲突：如果将来有个 key 也叫某个学号（比如 `course:all` 换成 `all`），就会撞车。
2. 无法批量管理：想清空所有用户缓存，只能用 `KEYS *`（性能杀手）或 `SCAN` 遍历。
3. 可读性差：在 `redis-cli` 里看到一堆 `2021001`、`2021002`，不知道是什么。

正确做法：`user:info:{studentNum}`。项目里其他 key 都遵守了 模块:实体:标识 的规范（`product:stock:`、`user:snapped:`），唯独用户缓存这个最重要的 key 破了例，应该统一。`RedisConstant` 里也没有为它定义常量，是硬编码的字符串。

追问："Redis key 命名的最佳实践？"

- ① 用冒号分隔，形成层级：业务:实体:id:字段；
- ② 控制长度（Redis key 本身也占内存，10 万 key 每个省 20 字节就很可观）；
- ③ 统一在常量类里定义（本项目已有 `RedisConstant`，但没贯彻到底）；
- ④ 永远不要用 `KEYS *`，线上用 `SCAN` 游标遍历；
- ⑤ 如果是 Redis Cluster，注意 `{}` 哈希标签保证相关 key 落在同一个槽（如 Lua 脚本操作的多个 key 必须在同一槽）。

Q5.6 课程缓存用的 Hash 结构，为什么不用 String？

```
String hashKey = "course:all";
Map<Object, Object> redisMap = redisTemplate.opsForHash().entries(hashKey);
List<Course> allCourses;

if (redisMap.isEmpty()) {
    // 缓存未命中，全量查库
    allCourses = courseDAO.findAllCourse().stream()...;

    Map<String, String> courseMap = allCourses.stream()
        .collect(Collectors.toMap(course -> String.valueOf(course.getId()), JSON::toJSONString));

    redisTemplate.opsForHash().putAll(hashKey, courseMap);
}
```

```

redisTemplate.expire(hashKey, 3, TimeUnit.HOURS);
} else {
    allCourses = redisMap.values().stream()
        .map(obj -> JSON.parseObject(obj.toString(), Course.class))
        .collect(Collectors.toList());
}

```

Hash 结构的好处:

- **可以按字段单独读写**: 选课时只需要更新某一门课的选课人数, 用 `HSET course:all {courseId} {json}`, 不用把整个课程列表读出来再整体写回。 `UserCourseServiceImpl` 正是这么做的:

```

redisTemplate.opsForHash().put(RedisConstant.COURSE_HASH_KEY, field, JSON.toJSONString(course));

```

- **节省序列化开销**: 如果是 String 存整个 List, 更新一门课要反序列化全部 N 门课、改一门、再全量序列化写回。N 越大越亏。
- **并发更安全**: 两个管理员同时改不同课程, Hash 结构下是 `HSET` 不同 field, 互不干扰; String 结构下就是"读-改-写", 必然丢失更新。

但本项目的实现有问题:

1. `entries(hashKey)` 会把整个 Hash 全量拉到客户端。如果课程有 5000 门, 每次查询都要传输几 MB 的 JSON 过来, 然后在 Java 内存里做过滤和分页 (`filter + skip + limit`)。
这是"把 Redis 当数据库用"的反模式。正确做法: ① 数据量小的时候直接在缓存里全量过滤 (当前做法, 能接受); ② 数据量大时应该把过滤下推到数据库, 用 `queryByCondition + JSON_CONTAINS` 做真正的分页查询, Redis 只缓存热点数据 (比如按年级/学院维度分别缓存)。
2. Hash 的 field 是 `String.valueOf(course.getId())`, 而 `UserCourseServiceImpl` 里用的也是 `String.valueOf(courseId)`——这一点是一致的, 很好。如果一处用 String、一处用 Long, 因为序列化器不同, 就会读写不到同一个 field。
3. `addCourse` 后没有失效 `course:all` 缓存!

```

// CourseServiceImpl.addCourse()
int insertRes = courseDAO.insertCourse(courseDO);
// ← 这里没有 redisTemplate.delete("course:all") 或 HSET 新课程

```

后果: 管理员添加了一门课, 但在最长 3 小时内, 所有用户查课程列表都看不到这门新课。这是一个典型的"缓存更新遗漏"bug。修复很简单: 添加课程后删除缓存 (或者直接 `HSET` 写入新课程)。

4. 课程缓存的更新逻辑也没考虑失败回滚: `UserCourseServiceImpl` 里先改数据库再改 Redis, 如果 Redis 操作抛异常, 数据库已经改了但缓存没改, 用户看到的是旧数据。

Q5.7 缓存和数据库的一致性怎么保证? 项目用了什么策略?

业界四种策略:

策略	做法	一致性	问题
先更新DB, 再更新缓存	UPDATE DB → SET Cache	差	并发下会写脏; 缓存值可能是多个请求交错更新的结果
先删缓存, 再更新DB	DEL Cache → UPDATE DB	差	删完后、更新前有请求进来, 会把旧值重新载入缓存
先更新DB, 再删缓存 (Cache Aside, 推荐)	UPDATE DB → DEL Cache	较好	极端并发下仍有小概率不一致, 但可接受

延迟双删	DEL Cache → UPDATE DB → sleep → DEL Cache	好	sleep 时间难定
------	--	---	------------

为什么"先更新DB再删缓存"是最优的？

考虑最坏情况：请求 A 更新数据库、请求 B 读数据。

- 如果 B 在 A 更新 DB 之后读，缓存已删（或即将删），B 会从 DB 读到新值并写回缓存 → 一致。
- 如果 B 在 A 更新 DB 之前读，B 读到旧值写进缓存，然后 A 删缓存 → 缓存被删了，下次读会重建 → 一致。

关键点：删除操作"晚"于读操作，就能覆盖掉脏数据。而"更新缓存"方案里，如果 A 的更新晚于 B 的写入，就会把 B 的旧值覆盖回去。

本项目实际用的是"更新DB + 更新/删除缓存"混合体：

```
// UserServiceImpl.updatePerMsg()
int result1 = userDAO.updateByStudentNum(userDO);
if (result1 > 0) {
    redisTemplate.opsForValue().set(studentNum, userDO, 6, TimeUnit.HOURS); // 更新 Redis
    userCache.evict(userDO.getId()); // 删除 Caffeine
    ...
}
```

这里其实做得还算对：Redis 是"写入新值"（因为 `userDO` 是刚从缓存/库里取出来改过的，值是对的），Caffeine 是"删除"（因为本地缓存无法精确知道有哪些 key 受影响，删了让它下次重建）。

但有几个问题：

1. **没有事务保护**：数据库更新和缓存更新不在同一个事务里。如果 `redisTemplate.set()` 抛异常（Redis 挂了），数据库已改但缓存是旧的，**用户看到自己的新昵称没生效**。
改进：① 用 `TransactionSynchronizationManager.registerSynchronization` 在 `afterCommit` 里再操作缓存（就像秒杀那样）；② 或者接受最终一致，靠 TTL 兜底。
2. **删除用户时删了 Redis，但没考虑"缓存重建"**：`deleteByStudentNum` 里 `redisTemplate.delete(studentNum)`，但如果此时有一个并发的登录请求刚刚读到旧值，它会把旧值重新写回缓存，导致**已删除的用户还在缓存里，还能登录**。**双重删除或加短 TTL 可以缓解**。
3. **选课/退课的缓存更新完全在事务外，且没有加锁**。
4. **Caffeine 多实例不一致**（前面 Q5.3 已详述）。

秒杀场景下的正确示范（本项目做得最好的地方）：

`doSnapBusiness` 里用 `TransactionSynchronizationManager.registerSynchronization`，在**事务真正提交后才发 Kafka**，并在**事务回滚时补偿 Redis**：

```
// ① 事务回滚时修复 Redis（注册在最前面，保证一定会被调用）
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
    @Override
    public void afterCompletion(int status) {
        if (status == STATUS_ROLLED_BACK) {
            redisTemplate.opsForValue().increment(stockKey, 1); // 库存加回去
            redisTemplate.delete(userKey); // 删掉去重标记
        }
    }
});

// ... 数据库操作 ...

// ② 事务提交成功后才发消息
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
    @Override
```

```

public void afterCommit() {
    kafkaTemplate.send(Topics.KAFKA_ORDER_TOPIC, ..., orderJsonStr);
}
});

```

这个“以数据库为准，失败则回滚缓存”的思路是本项目最值得讲的技术亮点。一定要能说清楚 `afterCommit`（只在成功时调用）和 `afterCompletion`（成功失败都调用，需自己判断 status）的区别。

Q5.8 Redisson 用来做什么？为什么不用 `setnx` 自己实现分布式锁？

项目里 Redisson 只用来做了一件事：**分布式原子自增**，生成订单号。

```

private String generateOrderNumber() {
    DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyyMMdd");
    String now = LocalDate.now().format(dateTimeFormatter);
    String redisKey = "order:auto:increment:" + now;

    RAtomicLong atomicLong = redissonClient.getAtomicLong(redisKey);
    atomicLong.expire(6, TimeUnit.HOURS);
    long number = atomicLong.incrementAndGet();
    return now + "-" + number;
}

```

生成的订单号：`20260928` + 自增数字，例如 `202609281`、`202609282`。

为什么用 `RAtomicLong` 而不是 `redisTemplate.opsForValue().increment()`？

其实 `increment()` 底层就是 `INCR`，本身也是原子的，功能上够用。用 Redisson 的意义在于：

- **统一封装**：项目已经引入了 Redisson，用它更顺手。
- **`expire()` 的原子性**：Redisson 的 `RAtomicLong.expire()` 会确保 key 存在并设置 TTL，避免“设了值但没设过期时间”导致内存泄漏。
- **可扩展性**：如果将来需要更复杂的原子操作（如 `compareAndSet`、`getAndAdd`），Redisson 的 API 更完整。

但这个订单号生成方案有问题：

1. **位数会变化**。`202609281`（9 位）和 `2026092810`（10 位）长度不同，如果下游系统按固定长度解析会出错。
2. **单日超过 10 位数怎么办**：没做兜底。
3. **`expire(6, TimeUnit.HOURS)` 会在每次调用时重置 TTL**。这意味着如果一直有订单产生，这个 key **永远不会过期**，Redis 内存持续增长（虽然每天一个 key，量不大，但逻辑上不对）。应该只在 key 首次创建时设置 TTL。
4. **Redis 是关键路径依赖**：Redis 挂了就生成不了订单号，整个秒杀链路断掉。应该有心跳降级（如本地雪花算法兜底）。
5. **更好的方案是 Snowflake 雪花算法**：本地生成、无网络 IO、趋势递增、每秒可产 409.6 万个 ID。缺点是依赖机器时钟（时钟回拨会重复），需要额外处理。

关于分布式锁（追问）：

项目引入了 Redisson，也 import 了 `RLock`，但实际上没有使用分布式锁（`RLock` 是未使用的 import）。这是个好问题——为什么秒杀不需要分布式锁？

因为 Redis + Lua 本身就是原子的。整个“检查库存 → 检查重复 → 扣减 → 标记用户”四个操作在一个 Lua 脚本里执行，Redis 单线程执行脚本，天然就是原子的。**加分布式锁反而会**：① 增加一次网络往返（加锁）+ 一次（解锁），性能下降；② 把并发变成串行，丧失 Redis 的吞吐优势；③ 引入锁超时、锁续期（看门狗）等复杂度。

如果非要用分布式锁，Redisson 的 `RLock` 相比手写 `setnx` 的优势：

- ① **原子加锁**：`SET key value NX PX 30000` 一条命令完成，避免“设了值再设过期时间”两步之间的宕机风险；
- ② **看门狗自动续期**：业务没执行完时自动延长锁的 TTL，避免“锁提前释放导致并发”（这是手写方案最容易踩的坑）；
- ③ **可重入**：同一个线程可多次加锁（用 Hash 存 线程ID → 重入次数）；

- ④ **安全的解锁**：用 Lua 脚本校验“锁是不是自己的”再删除，避免误删别人的锁；
- ⑤ **支持 RedLock**：多 Redis 实例投票加锁，容忍少数节点故障。

Q5.9 Redis 的持久化机制了解吗？项目需要怎么配置？

RDB (Redis Database) :

- 原理：**周期性快照**，把内存数据全量写入二进制文件 `dump.rdb`。
- 触发：`save`（阻塞） / `bgsave`（fork 子进程，写时复制 COW）。
- 优点：文件小、恢复快，适合备份和灾难恢复。
- 缺点：**会丢数据**——两次快照之间的写入全部丢失。fork 大内存实例时可能阻塞。

AOF (Append Only File) :

- 原理：**追加写命令日志**。每条写命令追加到 `appendonly.aof`。
- 刷盘策略：`always`（每条都 fsync，最安全最慢） / `everysec`（默认，每秒一次，最多丢 1 秒） / `no`（交给 OS）。
- 优点：数据安全性高。
- 缺点：文件大、恢复慢（要重放所有命令）。

Redis 4.0+ 混合持久化：RDB 全量 + AOF 增量。AOF 重写时，前半部分用 RDB 格式存快照，后半部分追加增量命令。**兼顾恢复速度和数据安全，是生产推荐配置。**

对本项目的意义（面试官可能追问）：

项目用 Redis 存了三类关键数据，对持久化的要求完全不同：

- 秒杀库存** `product:stock:{id}`：如果 Redis 重启丢失，`initStock()` 只在应用启动时执行，**Redis 重启而应用不重启，库存 key 就永久消失了**，Lua 脚本会因为 `stock or 0 = 0` 而判定“已抢光”。
所以库存必须开启 AOF (everysec) 或者至少 RDB 高频快照。更好的做法是加懒加载兜底（前面 Q5.2 提过）。
- 去重标记** `user:snapped:*`：丢失后果较轻——数据库唯一索引能兜底（会抛 `DuplicateKeyException` 并被捕获）。**这正好体现了“多层防护”的价值。**
- 验证码**：丢失只是用户要点一次“重新发送”，影响小。
- 在线用户** `online:users`：丢失没关系，WebSocket 重连后会重新注册。

还有一个重要点：Redis 的 key 都设了 TTL，但 TTL 不会被 RDB/AOF 完整保留——Redis 重启后，`expire` 时间的恢复依赖 RDB 里存储的绝对过期时间戳，过期了的 key 不会加载。这是正常的。

第六级：限流与 AOP

Q6.1 项目里为什么有两套限流机制？分别解决什么问题？

维度	AOP 注解式 (<code>RateLimitAspect</code>)	拦截器式 (<code>RateLimiterInterceptor</code>)
触发方式	<code>@RateLimit</code> 注解，声明式	路径匹配 <code>/api/product/snappedUp</code>
粒度	方法级，每个接口可配不同参数	路径级，只针对秒杀
限流维度	登录用户按 <code>userId</code> ，未登录按 <code>IP</code>	只按 <code>userId</code> ，未登录直接放行
参数	可配置 <code>windowMs</code> / <code>maxRequests</code> / <code>message</code>	常量写死在 <code>RedisConstant</code>
超限处理	抛 <code>RateLimitException</code> → <code>@RestControllerAdvice</code> → 429	手动写 <code>response</code> ，返回 429
执行顺序	后（进入 Controller 方法后）	先（进入 Controller 之前）
配置	加注解即可，自动生效	必须在 <code>WebAppConfiguration</code> 注册

为什么秒杀需要额外的拦截器？

秒杀接口的限流要求更严格、更靠前：`RATE_LIMIT_WINDOW_MS = 1000`（1 秒窗口）、`RATE_LIMIT_MAX_REQUESTS = 3`（最多 3 次）。

拦截器在 **Controller 之前** 执行，能在更早的阶段拒绝请求，节省后续所有处理的开销（参数绑定、AOP 代理调用等）。这在秒杀这种极端场景下是有意义的。

但当前实现有明显的设计缺陷：

1. 拦截器里未登录直接放行（`if (userInfo == null) return true;`）。但秒杀接口本身需要登录（`SnappedUpController.snappedUp` 直接 `userInfo.getId()` 会 NPE）。不过因为 `UserInterceptor` 注册在更前面，未登录请求已经被挡在 401 了，所以这里实际不会走到。这属于冗余的防御，逻辑上不够清晰。
2. 两套限流的 key 前缀不统一：AOP 用 `rate:limiter:anno:...`，拦截器用 `rate:limiter:snapped:user:...`。虽然不冲突，但缺乏统一规范。
3. 两套 Lua 脚本内容完全一样却写了两遍（`RateLimitAspect` 和 `RateLimiterInterceptor` 各有一份重复的脚本字符串）。应该抽成一个公共的限流工具类或把脚本外置到 `.lua` 文件。代码重复是维护隐患——改了一处忘了另一处。
4. 秒杀接口同时受两套限流：先过拦截器（1 秒 3 次），再进 Controller 过 AOP。但秒杀接口没有加 `@RateLimit` 注解，所以实际上只有拦截器生效。这个“双机制”的设计意图没有完整落地。

Q6.2 滑动窗口限流的 Lua 脚本详细讲一下。

```
local key = KEYS[1]          -- 限流 key
local now = tonumber(ARGV[1]) -- 当前时间戳（毫秒）
local windowStart = tonumber(ARGV[2]) -- 窗口开始时间 = now - windowMs
local maxRequests = tonumber(ARGV[3]) -- 窗口内最大请求数
local ttlSeconds = tonumber(ARGV[4]) -- key 过期时间
local member = ARGV[5]        -- 本次请求的唯一标识

-- ① 删除窗口外的旧数据（滑动窗口的灵魂）
redis.call('ZREMRANGEBYSCORE', key, 0, windowStart)

-- ② 统计当前窗口内的请求数
local count = redis.call('ZCARD', key)
```

```
-- ③ 超过阈值，拒绝
if count >= maxRequests then
    return 0
end

-- ④ 未超限，把本次请求加入窗口
redis.call('ZADD', key, now, member)

-- ⑤ 设置过期时间，防止内存泄漏
redis.call('EXPIRE', key, ttlSeconds)

return 1
```

数据结构：Redis Sorted Set (ZSet)

- `member` = `now + ":" + UUID`（必须唯一，否则同一毫秒的多次请求会覆盖）
- `score` = 请求时间戳

滑动窗口 vs 固定窗口：

算法	原理	问题
固定窗口	每 1 分钟一个 key， <code>INCR</code> 计数	临界问题： 限 100/分钟时，在 0:59 发 100 个 + 1:00 发 100 个， 2 秒内通过了 200 个请求
滑动窗口	ZSet 记录每个请求的时间戳，动态划定窗口	精确，但占内存（每个请求一条记录）
令牌桶	匀速生成令牌，请求取令牌	允许突发流量
漏桶	请求进桶，匀速流出	削峰但不允许突发

为什么这个脚本是原子且安全的？

因为 Redis 执行 Lua 脚本是原子的——脚本执行期间，Redis 会阻塞处理其他命令。这意味着“清理 → 计数 → 判断 → 写入”四步之间不可能有并发插入，避免了“两个请求同时读到 count=2（阈值 3）然后都放行”的竞态。

对比不用 Lua 的写法（错误）：

```
// 错误！四步之间有竞态
redisTemplate.opsForZSet().removeRangeByScore(key, 0, windowStart);
Long count = redisTemplate.opsForZSet().zCard(key);
if (count < maxRequests) {
    redisTemplate.opsForZSet().add(key, member, now);
}
```

这个脚本的问题（面试官会挖）：

- EXPIRE 每次请求都重置。**这没问题（滑动窗口本来就该一直往后延），但要注意：如果 key 的 TTL 小于窗口大小，会出现“数据提前被删、限流失效”。项目里 `ttlSeconds = windowMs / 1000 + 1`，比如窗口 1000ms → TTL = 2 秒。这是安全的，因为 TTL 略大于窗口。
- ZSet 内存占用：**每个请求存一条 member（约 50 字节）。如果窗口 1 分钟、限 10000 次，单 key 就是 500KB。高频场景下要评估。
优化方案：用“分片计数”（把窗口切成若干小格，每格一个计数器，用 Hash 存，内存降一个数量级，精度略有损失）。
- 没有考虑“同一毫秒内的多个请求”：**member 用 UUID 保证唯一，这点做对了。但如果两个请求的 `now` 完全相同，`ZADD` 的两个 member score 相同，仍会分别计数（ZSet 是 member 唯一，不是 score 唯一），**逻辑正确**。
- 没有降级策略：**如果 Redis 挂了，`stringRedisTemplate.execute()` 会抛异常，**整个接口 500**。生产环境应该 try-catch 降级为“放行”（可用性优先）或“拒绝”（安全优先），取决于业务。

5. **限流维度不够**：没有按"接口 + 用户 + IP"多维度组合。比如一个用户从多个 IP 攻击，或者一个 IP 上有多个用户。**生产级方案应该用 Sentinel / Redis-Cell (Redis 官方限流模块，基于令牌桶 + GCRA 算法)。**

Q6.3 @RateLimit 注解是怎么被拦截的？@Around("@annotation(rateLimit)") 的原理？

注解定义：

```
@Target(ElementType.METHOD)          // 只能标在方法上
@Retention(RetentionPolicy.RUNTIME) // 运行时保留（AOP 需要反射读取）
public @interface RateLimit {
    long windowMs() default 2000;        // 窗口大小
    int maxRequests() default 3;         // 最大请求数
    String message() default "操作太频繁，请稍后再试";
}
```

@Retention 的三个级别很重要：

- **SOURCE**：只在源码里，编译后丢弃（如 @Override、Lombok 的注解）。
- **CLASS**：保留在 class 文件里，但运行时不可见（默认值）。
- **RUNTIME**：保留到运行时，可通过反射读取——AOP 必须要这个。

切面定义：

```
@Aspect
@Component
public class RateLimitAspect {

    @Around("@annotation(rateLimit)") // 切所有标了 @RateLimit 的方法
    public Object doRateLimit(ProceedingJoinPoint pjp, RateLimit rateLimit) throws Throwable {
        // ...限流逻辑...
        if (result == 0L) throw new RateLimitException(rateLimit.message());
        return pjp.proceed(); // 放行，执行原方法
    }
}
```

@annotation(rateLimit) 的作用：它是一个切点表达式，匹配"所有标注了 @RateLimit 注解的方法"，并且 Spring 会自动把匹配到的注解实例注入到方法参数 rateLimit 上。所以方法签名里能直接写 RateLimit rateLimit 参数，拿到注解上的 windowMs()、maxRequests() 配置值。

底层实现链路：

1. Spring 启动时，@Aspect 类被 AnnotationAwareAspectJAutoProxyCreator（一个 BeanPostProcessor）识别为切面。
2. 对所有 Bean 检查方法是否匹配切点。匹配到的 Bean（本项目中是 UserController、UserAPI、CommentController 等）在 Bean 生命周期的 postProcessAfterInitialization 阶段被替换成代理对象（CGLIB，因为 Controller 没有接口）。
3. 调用代理方法时，ReflectiveMethodInvocation 链依次执行通知（Advice）：@Around → 目标方法。

限流的 key 设计（这段很值得讲）：

```
String className = pjp.getTarget().getClass().getSimpleName();
String methodName = pjp.getSignature().getName();

String key;
if (userInfo != null) {
    key = "rate:limiter:anno:" + className + ":" + methodName + ":" + userInfo.getId();
} else {
    key = "rate:limiter:anno:" + className + ":" + methodName + ":ip:" + getClientIp(request);
}
```

优点：

- 用"类名 + 方法名"作为 key 的一部分，使每个接口有独立的限流窗口。这样登录接口的限流不会影响注册接口。
- 已登录按 `userId`，未登录按 `IP`，兼顾了两种场景。
为什么未登录按 `IP`？因为登录/注册接口此时还没有用户身份，只能用 `IP`。为什么已登录不按 `IP`？因为同一办公网络（NAT）下很多用户共享一个出口 `IP`，按 `IP` 限流会误伤。
- 注解参数化：不同接口配不同阈值，如登录 5 次/分钟、发注册验证码 1 次/分钟、忘记密码 2 次/分钟。

问题：

1. `getClientIp` 直接信任 `X-Forwarded-For` 头：

```
String[] headers = {"X-Forwarded-For", "Proxy-Client-IP", "WL-Proxy-Client-IP"};
```

`X-Forwarded-For` 是客户端可以随意伪造的 HTTP 头！攻击者只要每次请求带一个不同的 `X-Forwarded-For` 值，就能无限绕过 `IP` 限流。

正确做法：① 在 Nginx/网关层就剥离客户端传来的 `X-Forwarded-For`，只保留真实 `IP`；② 只信任已知代理的 `IP`（维护一个可信代理列表）；③ 用 Spring Boot 的 `ForwardedHeaderFilter` 或 `server.forward-headers-strategy=native` 让容器统一处理。

- 2. `RequestContextHolder.currentRequestAttributes()` 可能抛异常：如果这个切面被非 HTTP 线程调用（比如定时任务、Kafka 消费者调用了带注解的方法），会抛 `IllegalStateException: No thread-bound request found`。应该加防护或改成可选的上下文获取。
- 3. 没有考虑注解在类上、或者组合注解的情况。`@annotation()` 只匹配方法级注解。
- 4. 限流失败时抛异常，但没有记录被限流用户的详细信息用于风控分析。

Q6.4 限流阈值是怎么设定的？登录 5 次/分钟合理吗？

项目里各接口的配置：

接口	windowMs	maxRequests	含义
<code>/user/register</code>	60000	5	1 分钟最多 5 次注册
<code>/user/login</code>	60000	5	1 分钟最多 5 次登录
<code>/user/forgot</code>	60000	2	1 分钟最多 2 次重置密码
<code>/user/update</code>	60000	3	1 分钟最多 3 次改信息
<code>/handleUpdatePwd</code>	60000	2	1 分钟最多 2 次改密码
<code>/user/uploadAvatar</code>	60000	5	1 分钟最多 5 次上传
<code>/user/sendRegCode</code>	60000	1	1 分钟最多 1 次发验证码
秒杀（拦截器）	1000	3	1 秒最多 3 次

阈值设定的考量：

- 登录 5 次/分钟：正常用户不会 1 分钟输错 5 次密码。这个值能挡住暴力破解的绝大部分尝试。但安全要求高的系统应该更严格（比如 5 次失败后锁定账号 15 分钟，而不是"1 分钟后自动恢复"）——因为攻击者可以"5 次/分钟 × 60 分钟 = 300 次/小时"，一天就是 7200 次，配合弱密码字典仍有可能成功。
- 发验证码 1 次/分钟：防止邮件轰炸（避免被用作垃圾邮件发送器，也避免 SMTP 账号被 QQ 封禁）。这个设计很必要。
- 秒杀 1 秒 3 次：正常用户不会连点 3 次以上，能挡住脚本批量刷。

但这套阈值设计有几个问题：

1. **限流维度是"单用户/单 IP"，没有全局限流。**如果 10 万个用户同时登录，每个用户都在阈值内，但系统整体已经被打垮。需要**有全局限流**（如整个 `/user/login` 接口 1 万 QPS 上限），这通常放在网关层（Nginx `limit_req` / Spring Cloud Gateway / Sentinel 集群限流）。
2. **没有"账号锁定"机制：**连续失败应该锁定账号，而不是只限制速率。
3. **阈值硬编码在注解里，无法动态调整。**生产环境应该把阈值放到配置中心，支持热更新（活动期间临时放宽秒杀限流）。
4. **没有区分成功/失败：**登录接口"5 次失败"和"5 次成功"都被限流。正常用户登录后还想再登录一次（换设备）就会被挡住。

Q6.5 AOP 有哪些通知类型？为什么这里必须用 `@Around` ？

通知类型	注解	执行时机	能力
前置	<code>@Before</code>	方法执行前	不能阻止执行（除非抛异常），拿不到返回值
环绕	<code>@Around</code>	包裹整个方法	可控制是否执行 <code>pjp.proceed()</code> 、可改参数、可改返回值
返回后	<code>@AfterReturning</code>	方法正常返回后	可拿到返回值，无法阻止
异常后	<code>@AfterThrowing</code>	方法抛异常后	可处理异常
最终	<code>@After</code>	方法结束后（含异常）	类似 <code>finally</code> ，无法阻止

为什么限流必须用 `@Around` ？

因为限流的语义是"条件性地不执行目标方法"。只有 `@Around` 能拿到 `ProceedingJoinPoint`，从而决定是否调用 `pjp.proceed()`：

```
if (result == 0L) {
    throw new RateLimitException(...); // 不调用 proceed，目标方法不执行
}
return pjp.proceed(); // 放行
```

如果用 `@Before`，虽然也能通过抛异常阻止执行，但：① 语义上不如 `@Around` 清晰；② 无法处理"放行后的逻辑"；③ 异常处理的链路不一样。

Spring AOP 的其他知识点（追问）：

- **只支持方法级连接点。**Spring AOP 基于动态代理，**不能拦截字段访问、构造器调用**（AspectJ 编译期织入可以，但配置复杂）。
- **只能拦截 Spring 容器管理的 Bean 的 public 方法。**
- **同类内部自调用失效**（同 `@Transactional` 的失效场景）。
- **性能开销：**CGLIB 代理的方法调用比直接调用慢约 10~30 纳秒，**在正常业务中可以忽略**，但在秒杀这种 QPS 上万的场景下，每次抢购都过一遍切面（要拿 `RequestContextHolder`、拼 key、走 Redis），累积开销不小。**这也是为什么秒杀选择了在拦截器层面做限流而不是 AOP。**

第七级：秒杀系统核心链路（重点）

Q7.1 完整讲一遍秒杀的流程，从用户点击到收到邮件。

阶段一：启动时库存预热

```
@PostConstruct
public void initStock() {
    List<ProductDO> productDOList = productDAO.getAll();
    for (ProductDO productDO : productDOList) {
        String stockKey = RedisConstant.PRODUCT_STOCK_PREFIX + productDO.getId();
        redisTemplate.opsForValue().set(stockKey, productDO.getStock(),
                                        RedisConstant.STOCK_EXPIRE_SEC, TimeUnit.SECONDS);
    }
    log.info("缓存预热成功！");
}
```

应用启动时把数据库库存全量加载到 Redis（`product:stock:{id}`，TTL 6 小时）。目的：让后续所有库存判断都在内存里完成，不碰数据库。

阶段二：用户请求抢购（同步链路，核心）

```
@GetMapping("/snappedUp")
public Result<Boolean> snappedUp(@RequestParam("productId") long productId,
                                HttpServletRequest request) {
    UserInfo userInfo = (UserInfo) request.getAttribute("currentUser");
    return snappedService.snappedUp(productId, userInfo.getId());
}
```

进入 `SnappedUpServiceImpl.snappedUp()`：

```
// ① 参数校验
if (productId <= 0 || userId <= 0) return "参数错误";

// ② 准备 key
String stockKey = "product:stock:" + productId;
String userKey = "user:snapped:" + userId + ":" + productId;

// ③ 执行 Lua 脚本（原子！）
Long execute = redisTemplate.execute(LUA_SCRIPT, Arrays.asList(stockKey, userKey));

// ④ 根据返回值判断结果
if (execute == -1) return "商品已抢光";
if (execute == -2) return "您已抢购过该商品";
if (execute != 1) return "抢购失败";

// ⑤ 拿商品信息（Caffeine 本地缓存）
ProductDO productDO = productCache.get(productId, key -> productDAO.selectById(key));

// ⑥ 发 Kafka 消息，异步落库
kafkaTemplate.send("snappedUp", String.valueOf(userId), JSON.toJSONString(productDO));

// ⑦ 立即返回（不等待落库结果）
return "若你抢购成功后会以邮件形式通知你。";
```

Lua 脚本（本项目的核心）：

```

local stockKey = KEYS[1]
local userKey = KEYS[2]

-- ① 幂等校验：该用户是否已抢过
if redis.call('exists', userKey) == 1 then return -2 end

-- ② 库存校验（注意 `or 0` 的兜底）
local stock = tonumber(redis.call('get', stockKey) or 0)
if stock <= 0 then return -1 end

-- ③ 扣减库存
redis.call('decr', stockKey)

-- ④ 标记用户已抢购，TTL 6 小时
redis.call('set', userKey, 1, 'EX', 21600)

return 1

```

返回值约定： **1** =成功， **-1** =库存不足， **-2** =重复抢购。

阶段三：Kafka 异步落库（`KafkaConsumer.listenSnap`）

```

@KafkaListener(topics = "snappedUp", concurrency = "3")
public void listenSnap(ConsumerRecord<String, String> record, Acknowledgment ack) {
    try {
        long userId = Long.parseLong(record.key());
        ProductDO productDO = JSON.parseObject(record.value(), ProductDO.class);
        snappedService.doSnapBusiness(userId, productDO.getId(), productDO.getPrice());
        ack.acknowledge();
    } catch (DuplicateKeyException e) {
        // 重复消费，跳过（幂等）
        ack.acknowledge();
    } catch (Exception e) {
        log.error("kafka消费失败", e);
        throw e;    // 不 ack，触发重试
    }
}

```

阶段四：事务内的数据库操作（`doSnapBusiness`）

```

@Transactional(rollbackFor = Exception.class)
public void doSnapBusiness(long userId, long productId, Double price) {

    // ① 注册事务回调（必须在最前面！）
    TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
        @Override
        public void afterCompletion(int status) {
            if (status == STATUS_ROLLED_BACK) {
                redisTemplate.opsForValue().increment("product:stock:" + productId, 1);
                redisTemplate.delete("user:snapped:" + userId + ":" + productId);
            }
        }
    });

    // ② 扣减数据库库存（SQL 带 stock > 0 守卫）
    int stockRows = productDAO.reduceStock(productId, 1);
    if (stockRows <= 0) throw new RuntimeException("库存扣减失败");

    // ③ 插入抢购记录（唯一索引兜底）
    snappedUserDAO.insert(snappedUserDO);
}

```

```
// ④ 创建订单
orderDAO.insertOrder(orderDO);

// ⑤ 注册提交后回调，发送邮件消息
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
    @Override
    public void afterCommit() {
        kafkaTemplate.send("order", String.valueOf(userId), JSON.toJSONString(orderDO));
    }
});
}
```

阶段五：发送订单邮件（`KafkaConsumer.listenOrder`）

```
@KafkaListener(topics = "order", concurrency = "2")
public void listenOrder(ConsumerRecord<?, ?> record, Acknowledgment ack) {
    try {
        UserDO userDO = userDAO.selectByUserId(userId);
        OrderDO orderDO = JSON.parseObject(record.jsonStr(), OrderDO.class);
        orderDO.setProduct(productDAO.selectById(...).toModel());
        orderDO.setUser(userDO.toModel());
        emailClient.sendOrderEmail(userDO.getEmail(), JSON.toJSONString(orderDO));
        ack.acknowledge();
    } catch (Exception e) {
        log.error("发送订单邮件失败", e);
        ack.acknowledge(); // 邮件失败也 ack，不阻塞
    }
}
```

整个链路的设计思想总结（这段要背熟）：

1. **层层过滤，越来越贵**：限流（拦截器，最便宜）→ Redis Lua（内存操作）→ Kafka（异步）→ 数据库（最贵。**绝大部分请求在 Redis 层就被挡掉了，只有真正抢到的才落库。**
2. **Redis 挡流量，数据库做最终一致。**
3. **异步化**：用户请求只需等 Redis 的几毫秒，不用等数据库和邮件。
4. **多级兜底**：Redis 去重 + 数据库唯一索引 + SQL 库存守卫，任何一层失效还有下一层。
5. **补偿机制**：事务回滚时把 Redis 扣掉的库存加回来。

Q7.2 为什么不用 Redis 事务（MULTI/EXEC）而要写 Lua 脚本？

Redis 事务的根本缺陷：不支持回滚、无法在事务中读取中间结果做判断。

Redis 事务的流程：`MULTI` → 命令入队 → `EXEC` 批量执行。它的特点：

- **命令是入队后批量执行，执行期间不能根据前面的结果做分支判断。**
我们的逻辑需要“**如果库存不足就返回，不执行扣减**”——这种**条件分支**在 Redis 事务里做不到，因为在 `MULTI` 之后、`EXEC` 之前，你拿不到任何中间值。
- **Redis 事务没有回滚。**如果 `EXEC` 时某条命令失败（比如类型错误），其他命令**照样执行**，Redis 不支持“撤销”。这和关系型数据库的“全部成功或全部失败”完全不同。
- **WATCH 乐观锁方案太重。**可以用 `WATCH stockKey` + `MULTI` 实现 CAS，但：① 需要在客户端监测 `EXEC` 返回值判断是否被打断，逻辑绕；② **高并发下冲突率极高**，大量请求要重试，性能反而更差（秒杀场景下每秒可能有上万人同时抢，WATCH 的冲突率接近 100%）。

Lua 脚本的优势：

- **真正的原子性**：Redis 单线程执行脚本，整个脚本执行期间不会插入其他命令。

- **支持条件判断和分支**：可以用 `if/else` 根据 `get` 的结果决定后续行为——这是 Redis 事务做不到的关键能力。
- **减少网络往返**：原本需要 4~5 次 RTT (`exists / get / decr / set`)，现在只要 1 次。在秒杀场景下这个优化非常显著——把 5 次网络往返压成 1 次。
- **可复用**：脚本会缓存在 Redis 端，用 SHA1 摘要调用 (`EVALSHA`)，不用每次传输脚本体。
项目里用的是 `redisTemplate.execute(script, keys, args...)`，Spring Data Redis 会自动先尝试 `EVALSHA`，失败再 `EVAL`。

Lua 脚本的注意事项（面试官会追问）：

1. **脚本不能太长、不能有耗时操作**。因为 Redis 单线程执行脚本，一个慢脚本会阻塞整个 Redis。项目里还配了 `lua-time-limit`（默认 5 秒）兜底，超时后 Redis 会开始报 BUSY 错误。
2. **不要在脚本里做循环大范围操作**（如遍历几十万个 key）。
3. **脚本里的操作不是事务性的回滚**——如果脚本执行到一半出错，前面已执行的命令不会回滚。所以脚本逻辑必须自己保证“要么全做，要么全不做”的顺序：本项目先检查再执行，检查通过后才修改，不会出现中间态。
4. **Redis Cluster 下，脚本涉及的所有 key 必须在同一个哈希槽**，否则报 `CROSSSLOT` 错误。本项目脚本涉及 `product:stock:{id}` 和 `user:snapped:{uid}:{pid}` 两个 key，在不同槽，所以这个脚本在 Redis Cluster 下会失败！
解法：用哈希标签 `{}` 强制同槽，例如把 key 设计成 `seckill:{productId}:stock` 和 `seckill:{productId}:user:{userId}`——花括号里的内容相同，就会落在同一个槽。
这是一个非常高频的进阶面试题。当前项目用的是单机 Redis，所以没暴露这个问题，但一旦上集群就会直接崩。
5. **脚本里访问不存在的 key 要小心**：`redis.call('get', stockKey) or 0` 会导致“库存 key 不存在 = 已抢光”，前面已经讨论过这个 bug。

Q7.3 为什么不用分布式锁来实现一人一单？

如果用分布式锁的方案（对比）：

```
RLock lock = redissonClient.getLock("lock:snap:" + userId + ":" + productId);
try {
    if (!lock.tryLock(1, 10, TimeUnit.SECONDS)) return "请求太频繁";
    // 查数据库是否已抢购
    if (snappedUserDAO.countByUserAndProduct(userId, productId) > 0) return "已抢购";
    // 扣库存
    // 插入记录
} finally {
    lock.unlock();
}
```

分布式锁方案的缺点：

1. **性能差**：每次抢购要“加锁 + 解锁”两次 Redis 往返，还要走数据库查询判断是否已抢。相比 Lua 一次原子操作，慢了数倍。
2. **锁的粒度问题**：如果按 `(userId, productId)` 加锁，粒度很细，锁的创建/销毁开销反而成了瓶颈。如果按 `productId` 加锁（更常见），就变成了串行化整个商品的抢购——1 万 QPS 直接降到几百，丧失了 Redis 的并发优势。
3. **锁的可靠性问题**：需要处理锁超时、看门狗续期、锁被误删、Redis 主从切换导致锁丢失（RedLock 才能缓解，但 RedLock 本身也有争议）。
4. **不解决库存超卖**：锁只能做“一人一单”，库存扣减还是要另想办法（或者在同一把锁里做，进一步降低并发度）。

Lua 方案的优势：

- 一次 RTT 完成“查重 + 查库存 + 扣库存 + 标记”四个操作，天然原子。
- 无锁化设计，并发度只受 Redis 单线程处理能力限制（Redis 单机可以轻松支撑 10 万 QPS 的简单命令）。
- 没有锁的生命周期管理复杂度。

什么场景才该用分布式锁？

- 操作不是"幂等的单条命令组合", 而是**跨越多个系统/多次数据库交互**的复杂业务 (如"扣库存 + 扣余额 + 记流水"跨三个服务)。
- 需要在业务层面做"互斥", 而不是"原子计数" (如防止同一个用户同时发起两个支付请求)。
- 操作的对象不支持原子命令 (如需要"读-计算-写"的复杂逻辑)。

总结一句话: 能用原子命令 (**INCR** /Lua) 解决的, 不要用锁。锁是最后的选择。

Q7.4 Redis 和数据库的库存怎么保证最终一致?

本项目的一致性链路分三段:

① 正常路径

```
Redis 扣减 (Lua, 原子)
  ↓ Kafka 异步消息
数据库扣减 (SQL 带 stock > 0 守卫)
  ↓
数据一致
```

Redis 是"预扣减", 真正生效以数据库为准。Redis 库存 ≤ 数据库库存 (Redis 先扣, 数据库后扣, 中间有短暂窗口)。

② 数据库失败, Redis 回滚 (事务回调补偿)

```
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
    @Override
    public void afterCompletion(int status) {
        if (status == STATUS_ROLLED_BACK) {
            redisTemplate.opsForValue().increment(stockKey, 1);    // 加回去
            redisTemplate.delete(userKey);                        // 允许重抢
        }
    }
});
```

这是本项目最核心的一致性保障。触发条件: 库存扣减返回 0 行、唯一索引冲突、订单插入失败等任何导致事务回滚的异常。

③ Kafka 消息丢失, Redis 无法回滚 (当前的漏洞)

```
// 当前代码
kafkaTemplate.send(Topics.KAFKA_SNAP_TOPIC, String.valueOf(userId), productJsonStr);
// ← 如果这里抛异常 (Kafka 挂了), Redis 已经扣了, 但消息没发出去
// ← 用户丢了抢购资格, Redis 库存少了一个, 数据库库存没动
```

这是一个真实的"资损"场景——Redis 库存被白白扣掉一个, 用户既没抢到也没收到通知, 而且这个库存槽位永久丢失 (除非重启应用或 key 过期回源)。

修复方案 (项目 **ToDo.java** 里作者也列出来了):

```
try {
    // 同步等待发送结果 (带超时), 确保消息真的进了 broker
    kafkaTemplate.send(Topics.KAFKA_SNAP_TOPIC, String.valueOf(userId), productJsonStr)
        .get(3, TimeUnit.SECONDS);
} catch (Exception e) {
    // 发送失败 → 回滚 Redis
    redisTemplate.opsForValue().increment(stockKey, 1);
    redisTemplate.delete(userKey);
    log.error("Kafka 发送失败, 已回滚 Redis 库存", e);
    result.setSuccess(false);
    result.setMessage("系统繁忙, 请重试");
}
```

```
return result;
}
```

代价：`.get()` 会阻塞当前线程等待 Kafka 确认，把“异步”变成了“同步”，每次抢购多等几毫秒到几十毫秒。这是一个可靠性 vs 性能的 trade-off。

更好的方案：

- **本地消息表（事务消息模式）**：先往数据库写一条“待发送消息”记录（和业务操作同一个事务），然后异步投递，成功后标记已发送。这样即使 Kafka 挂了，重启后也能重发。
- **Kafka 事务消息**：用 `transactional.id` 让 Kafka 消息和生产者的本地事务绑定。
- **定时对账补偿任务**：定期比对 Redis 库存、数据库库存、订单数量，发现差额自动修复（这是最终兜底，必须要有）。
- **用 Redis Stream 代替 Kafka**：把所有消息写在 Redis 里（同一个 Lua 脚本或 `XADD`），由消费者 `XREADGROUP` 消费。优点：Redis 里的消息和库存扣减在同一个存储上，可以在同一个 Lua 脚本里完成“扣库存 + 写消息”，彻底原子化。这是一个非常优雅的方案，值得在面试中主动提出。

④ 反向不一致：数据库有、Redis 没有（key 过期）

Redis 库存 key 6 小时后过期，此时若活动还在进行，Lua 会把“key 不存在”当成“库存 0”，返回“已抢光”。需要通过懒加载回源或定时刷新解决。

总结一句话回答：“Redis 是预扣减层，数据库是权威数据源。正常路径靠 Kafka 异步对齐；数据库失败靠事务回调把 Redis 加回来；Kafka 发送失败目前没有处理（这是我已知的改进点）；最终还应该有一个定时对账任务做兜底。”

Q7.5 TransactionSynchronizationManager 是什么？afterCommit 和 afterCompletion 有什么区别？

`TransactionSynchronizationManager` 是 Spring 事务管理的同步回调注册中心。它内部维护了一组 `ThreadLocal`，其中一个存放“事务同步回调列表”：

```
private static final ThreadLocal<Set<TransactionSynchronization>> synchronizations = ...;
```

通过 `registerSynchronization()` 注册的 `TransactionSynchronization` 会在事务生命周期的各个阶段被 Spring 回调。

完整的回调时机（按顺序）：

回调方法	时机	能否拿到事务状态
<code>beforeCommit</code>	提交前	可以，此时抛异常会导致回滚
<code>beforeCompletion</code>	提交/回滚前最后一步	可
<code>afterCommit</code>	仅提交成功后	可以读数据，但不能再改（改了不会提交）
<code>afterCompletion(int status)</code>	提交或回滚后都会调用	通过 <code>status</code> 参数判断

`status` 的取值：

- `STATUS_COMMITTED` (0)：已提交
- `STATUS_ROLLED_BACK` (1)：已回滚
- `STATUS_UNKNOWN` (2)：未知（如连接级问题）

为什么必须用 `afterCommit` 发邮件消息？

因为“提交成功”才代表订单真实存在。如果在方法体内、事务提交前就发 Kafka：

```
// 错误示范
orderDAO.insertOrder(orderDO);
kafkaTemplate.send("order", ...); // 消息发出去了
```

```
// ← 这里如果抛异常，事务回滚，订单不存在
// ← 但消息已经发出，消费者去查订单查不到 → 空指针或发了一封错误的邮件
```

用 `afterCommit` 保证“订单已落库”和“发送邮件消息”之间有明确的先后关系。

为什么回滚补偿要注册在“所有数据库操作之前”？

这是代码注释里明确写的：

```
// 注意要在所有数据库事务之前，否则来不及注册导致未执行
```

原因是：`registerSynchronization()` 必须在事务“活跃”期间调用。如果放在事务的最后一行，而它前面某行抛了异常，那么这行根本不会执行，补偿逻辑就失效了。

还有一个隐藏的坑（加分项）：

`TransactionSynchronizationManager.registerSynchronization()` 依赖 `ThreadLocal`。所以：

- 必须在事务所在的同一个线程里注册。如果在子线程里注册，`isSynchronizationActive()` 会是 `false`，直接抛 `IllegalStateException`。
- 如果方法没有实际开启事务（`@Transactional` 失效了），注册同步同样会抛异常。所以这段代码本身就是对“事务是否真的生效”的一种隐式校验——如果哪天 `@Transactional` 被误删，这个调用会立刻报错，算是意外的收获。

替代方案对比：

- `@TransactionalEventListener(phase = AFTER_COMMIT)`：更声明式的写法，本质是 `TransactionSynchronization` 的封装。项目没用这个，是因为需要“成功走一个逻辑、失败走另一个逻辑”的双分支，用同步回调更直观。

Q7.6 Kafka 消费者为什么能保证幂等？重复消费了会怎样？

三层幂等保障：

① Redis 层（第一道）

```
if redis.call('exists', userKey) == 1 then return -2 end
```

用户重复点击时，请求根本进不到 Kafka。但 Redis key 有 6 小时 TTL，过期后就失效了。

② 数据库唯一索引（第二道，最终防线）

```
ALTER TABLE snappedUser ADD UNIQUE INDEX uk_user_product (user_id, product_id);
ALTER TABLE `order` ADD UNIQUE INDEX uk_user_product (user_id, product_id);
```

重复插入时 MySQL 抛 `DuplicateKeyException`，被精确捕获：

```
} catch (DuplicateKeyException e) {
    log.warn("【重复消费，已跳过】用户{}已抢购该商品，无需重复处理", record.key());
    ack.acknowledge(); // 跳过，不再重试
}
```

为什么能捕获到？Spring 的 `MyBatisExceptionTranslator` 会把 `SQLIntegrityConstraintViolationException`（MySQL 错误码 1062）翻译成 Spring 的 `DuplicateKeyException`。

③ 事务保证（第三道）

因为 `doSnapBusiness` 是一个事务：`reduceStock` → `snappedUser.insert` → `order.insert`。如果第二步抛 `DuplicateKeyException`，整个事务回滚，第一步扣的数据库库存也会被撤销——这正是我们想要的（重复消息不应该再扣一次库存）。

但是！这里有一个微妙的问题（高分点）：

`DuplicateKeyException` 被 catch 后没有重新抛出，所以 `doSnapBusiness` 的事务是正常提交的，不，等等——异常是在方法内部抛出的，会传播到方法外（`@Transactional` 的方法），所以事务会回滚。

回滚会触发 `afterCompletion(STATUS_ROLLED_BACK)`，也就是会执行 Redis 补偿：把库存 +1、删掉用户标记。

这是一个 bug！场景还原：

1. 用户抢购成功，Redis 扣了 1 个库存。
2. Kafka 消息发出，消费者开始处理。
3. 由于某种原因（如网络抖动导致 `ack` 失败），Kafka 重投了同一条消息。
4. 第二次消费时，`snappedUser.insert` 抛 `DuplicateKeyException` → 事务回滚 → 触发 Redis 补偿，把已经正确扣减的库存加回来了！
5. 结果：Redis 库存比实际多 1 个，出现超卖。

正确做法：捕获 `DuplicateKeyException` 后，不应该让事务回滚。应该在 `doSnapBusiness` 内部捕获，或者用一个“只在真正的业务失败时回滚”的判断。

最简单可靠的修复：在补偿逻辑里加条件判断，只补偿“本次事务确实扣减了库存”的情况：

```
// 用 ThreadLocal 或局部变量标记“是否真的执行了扣减”
private void doSnapBusiness(...) {
    boolean deducted = false;
    ...
    int stockRows = productDAO.reduceStock(productId, 1);
    if (stockRows <= 0) throw new RuntimeException(...);
    deducted = true;    // 标记

    final boolean deductedFinal = deducted;
    TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
        public void afterCompletion(int status) {
            if (status == STATUS_ROLLED_BACK && deductedFinal) {
                // 只有真的扣了才补偿
            }
        }
    });
}
```

或者更彻底：把幂等检查前置到事务之前——在 Consumer 里先查一次 `countByUserAndProduct`，如果已存在直接 `ack` 跳过，根本不进事务。

这个分析如果能在面试中讲出来，是极强的加分项——说明你不只是会写代码，还能推演出并发边界场景下的错误。

Q7.7 Kafka 消息顺序性怎么保证？秒杀场景需要吗？

Kafka 的顺序性保证规则：

- 分区内有序：同一个 partition 内的消息按写入顺序消费。
- 跨分区无序：不同 partition 之间没有顺序保证。

本项目用的是 key 分区策略：

```
kafkaTemplate.send(Topics.KAFKA_SNAP_TOPIC, String.valueOf(userId), productJsonStr);
//                                topic                ↑ key = userId
```

Kafka 默认用 `hash(key) % numPartitions` 决定分区。所以同一个 `userId` 的消息会落到同一个分区，天然有序。

为什么用 `userId` 作为 key？

- **保证同一用户的操作有序**（如“抢购 → 取消订单”必须按顺序处理）。
- **保证分区均匀**：userId 分布均匀，各分区的信息量也均匀，避免热点。

秒杀场景需要严格的全局顺序吗？

不需要。因为：

- 不同用户的抢购是**完全独立**的，谁先谁后不影响业务正确性。
- 库存扣减的“顺序”由数据库的原子 UPDATE 保证（`where stock > 0`），**不依赖消息到达顺序**。
- 一人一单的幂等由唯一索引保证，也不依赖顺序。

如果真要全局有序会怎样？必须把 **partition 数设成 1** 或者所有消息用同一个 key——**这会让并发能力归零**（单个 partition 只能被一个消费者消费），秒杀场景绝对不能这么做。

但项目有一个顺序性隐患： `concurrency = "3"`

```
@KafkaListener(topics = "snappedUp", concurrency = "3")
```

`concurrency = 3` 意味着启动 **3 个消费线程**，每个线程消费不同的 partition。在**同一分区内仍然是有序的**（Kafka 保证一个 partition 同时只被一个消费者线程消费），**所以这个配置是安全的**。

但要注意：如果 topic 只有 1 个 partition，3 个线程里有 2 个是闲置的（Kafka 消费者线程数 > 分区数时，多余的线程空闲）。**所以 topic 的 partition 数应该 ≥ consumer 的 concurrency**。项目里没看到 topic 创建配置，如果 topic 是自动创建的（默认 1 个分区），那 `concurrency = 3` 就是浪费。

Q7.8 Kafka 的手动 ack 是怎么配置的？有什么坑？

配置：

```
spring:
  kafka:
    consumer:
      group-id: test-consumer-group
      enable-auto-commit: false           # 关闭自动提交
      auto-offset-reset: latest           # 无位点时从最新开始
      max-poll-records: 50                # 单次拉取最多 50 条
      properties:
        session.timeout.ms: 30000
        heartbeat.interval.ms: 10000
        max.poll.interval.ms: 300000
      listener:
        ack-mode: manual_immediate       # 手动立即确认
```

消费方法签名带 Acknowledgment：

```
@KafkaListener(topics = "...", concurrency = "3")
public void listenSnap(ConsumerRecord<String, String> record, Acknowledgment ack) {
    try {
        // 业务处理
        ack.acknowledge();    // 成功才提交位点
    } catch (DuplicateKeyException e) {
        ack.acknowledge();    // 幂等跳过，也算成功
    } catch (Exception e) {
        throw e;              // 不 ack → 位点不提交 → 下次重新拉取这条消息
    }
}
```

为什么用手动 ack 而不是自动提交？

自动提交（`enable-auto-commit=true`）是**定时批量提交**（默认每 5 秒），会有两个严重问题：

- 1. **消息丢失**：如果 offset 提交了但业务还没处理完（或处理失败），消费者重启后这条消息**永远丢失**。
- 2. **重复消费**：如果业务处理完了但 offset 还没提交就宕机，重启后会**重新处理**。

手动 ack 能做到“业务成功才提交位点”，把“至少一次”（at-least-once）语义落到实处。配合幂等设计，就得到了“effectively-once”的效果。

几个必须注意的坑：

- 1. **不 ack 会导致无限重试（毒丸消息问题）**。`listenSnap` 里 `catch (Exception e) { throw e; }`，如果这条消息因为**业务逻辑本身的 bug**永远处理不成功（比如 JSON 解析错误、数据库字段类型不匹配），那么每次拉取都会重新处理、再次失败——消费者会卡在这条消息上，永远消费不下去（消息堆积）。
正确做法：① 加重试次数限制（用 `RetryTemplate` 或记录重试次数到 Redis）；② 超过次数后**投递到死信队列（DLQ）**并 ack，人工介入处理；③ 用 Spring Kafka 的 `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`。
- 2. `listenOrder` / `listenVerifyCode` / `listenRegCode` 三个方法是 `catch (Exception e) { log.error(...); }` 然后 `ack.acknowledge()` —— **邮件发送失败就永久丢失了**。用户永远不会收到订单邮件/验证码。
这是“可靠 vs 可用”的取舍：邮件不是核心业务，失败不重试可以接受（避免阻塞）。但至少应该有**告警或重试队列**。而“验证码邮件丢失”会直接影响用户体验（用户永远收不到验证码，也无法重发）。
- 3. **三个方法没用 finally 块统一 ack**。前两个用 `catch` 里 ack，后两个用 `finally` 里 ack——**风格不统一**。用 `finally` 更安全（保证任何路径都会 ack）。
- 4. `auto-offset-reset: latest` 意味着**如果消费者组首次启动或位点丢失，会从最新消息开始消费，历史消息会被丢弃**。对于“订单邮件”这种业务，如果消费者停机一段时间，重启后会丢掉停机期间的**所有**订单邮件。**应该改成 `earliest`**（视业务而定）。
- 5. `max-poll-records: 50` 配合 `max.poll.interval.ms: 300000`（5 分钟）：如果 50 条消息 5 分钟处理不完，消费者会被踢出组触发 rebalance。本项目单条处理是“一次数据库更新 + 2 次插入 + 1 次发消息”，通常很快，这个配置是安全的。
- 6. **没有配置 `isolation.level`**：如果生产者用了事务，消费者默认是 `read_uncommitted`，可能读到未提交的消息。

Q7.9 生产者端配置 `acks=all` 和 `enable.idempotence=true`，意义是什么？

```
spring:
  kafka:
    producer:
      acks: all                # 所有 ISR 副本确认
      retries: 10              # 重试 10 次
      batch-size: 16384        # 16KB 批量
      buffer-memory: 33554432  # 32MB 缓冲
      compression-type: snappy # 压缩算法
      properties:
        enable.idempotence: true # 幂等生产者
        linger.ms: 5             # 等待 5ms 凑批
```

`acks` 的三个级别

值	含义	可靠性	性能
0	不等任何确认，发出去就算成功	可能丢消息	最高
1	等 leader 写入本地日志	leader 宕机可能丢	中
all / -1	等所有 ISR（同步副本）确认	最高	最低

`acks=all` + `min.insync.replicas≥2` 是**不丢消息的标准配置**。项目用了 `acks=all`，但没有配 `min.insync.replicas`（在 broker 端配置），如果 ISR 只剩 1 个副本，`acks=all` 也退化成 `acks=1` 的效果。

`enable.idempotence=true` 幂等生产者

解决的问题：网络抖动时生产者重试会导致消息重复。

生产者发出消息 → broker 写入成功 → 但 ACK 在网络中丢失
→ 生产者超时重试 → broker 又写入一遍 → 消息重复

实现原理：

- 生产者启动时从 broker 获取一个 `PID` (Producer ID)。
- 每条消息带一个单调递增的序列号 (Sequence Number)，按分区维护。
- broker 端为每个 `PID + 分区` 记录已接收的最大序列号。
- 收到序列号 $\leq$ 已记录值的消息 → 判定为重复，直接丢弃（但会返回成功 ACK，让生产者停止重试）。

开启幂等性的代价：

- 强制要求 `acks=all`、`retries > 0`、`max.in.flight.requests.per.connection ≤ 5`（否则无法保证顺序）。
- 有一定性能开销（多传 PID 和序列号）。
- 只保证单生产者会话内、单分区内的幂等。生产者重启会换新 PID，旧的去重信息失效。

`linger.ms: 5` 的作用

生产者不会一条一条发，而是等 5 毫秒凑一批再发。这样能显著提升吞吐量（减少网络请求数），代价是增加最多 5ms 延迟。

在秒杀场景下，这是一个很好的平衡点——5ms 的延迟用户完全感受不到，但批量化带来的吞吐提升是数量级的。

与 `batch-size: 16384` 配合：满足“攒够 16KB”或“等够 5ms”任一条件就发送。

`compression-type: snappy`

Snappy 的特点是压缩/解压速度极快，压缩率中等（相比 gzip 压缩率低但 CPU 开销小得多）。适合“吞吐优先、CPU 敏感”的场景，秒杀消息正好符合。

其他选项：`gzip`（压缩率高、慢）、`lz4`（比 snappy 更快、压缩率相近）、`zstd`（新，综合最优）。

Q7.10 秒杀接口当前有哪些设计缺陷？怎么改进？

这是项目 `ToDo.java` 里作者自己做的 code review，非常值得背下来，能主动说出自己的项目的缺陷是极强的加分项。

① 【严重】用 GET 做写操作

```
@GetMapping("/snappedUp") // ← 改状态的操作却用 GET
public Result<Boolean> snappedUp(@RequestParam("productId") long productId, ...)
```

问题：违反 REST 语义（GET 应该是幂等的、只读的）。更重要的是实际风险：

- 浏览器会预加载/预取链接。
- 搜索引擎爬虫会爬取并触发。
- 中间代理/CDN 可能缓存响应，导致后续用户直接拿到缓存的“成功”而没有真正抢购。
- 用户刷新页面会重复触发。

修复：`@PostMapping`。

② 【严重】Kafka 发送失败没有回滚 Redis

前面 Q7.4 已详述。这是最核心的一致性问题。

③ 【中等】Redis 库存无持久化/过期保障

前面 Q5.2 已详述。key 过期会导致“假性售罄”。

④ 【中等】用户反馈模糊

```
result.setMessage("若你抢购成功后会以邮件形式通知你。");
```

问题：这是个“薛定谔的抢购”——用户不知道到底成没成。而且如果后续 Kafka 消费/DB 写入失败了，用户永远等不到邮件，也没有任何提示，**只能干等**。

修复：① 提供“查询抢购结果”接口（轮询或 WebSocket 推送）；② 失败时也发一封“很遗憾没抢到”的邮件（项目里已经有 WebSocket 基础设施，完全可以用）；③ 至少在前端加个“查看我的订单”引导。

⑤ 【中等】Caffeine 缓存的商品价格可能是旧值

前面 Q5.3 已详述。**资金风险**。

⑥ 【轻微】`afterCommit` 里发 Kafka 没有错误处理

```
@Override
public void afterCommit() {
    kafkaTemplate.send(Topics.KAFKA_ORDER_TOPIC, ...);    // 失败了怎么办?
}
```

问题：事务已提交、订单已入库，但消息发送失败 → **邮件永不发送，且没有任何补偿机制**。

修复：① 加 try-catch + 重试 + 告警；② 落本地消息表，由定时任务补偿重发；③ 事务提交后写一条“待发邮件”记录，用独立的任务扫描发送。

⑦ 【轻微】SQL 的 `< if test="count > 0">` 隐患

前面 Q3.8 已详述。

⑧ 【补充：我们可以额外指出的】缓存预热依赖数据库全量扫描

```
@PostConstruct
public void initStock() {
    List<ProductDO> productDOList = productDAO.getAll();    // 全表查询!
    ...
}
```

问题：① 商品量大时启动慢；② 只预热“所有商品”，但实际只有少数参与秒杀活动，浪费 Redis 内存；③ 如果在活动进行中重启应用，**会把数据库的当前库存重新刷进 Redis，覆盖掉正在进行的秒杀状态**（可能导致超卖）。

修复：① 只预热“活动中的商品”（加 `where status = 1 and end_time > now()`）；② 用**懒加载 + 分布式锁**按需预热；③ 重启时判断 Redis 里是否已有库存值，有则保留。

Q7.11 如果让你把秒杀改造成能支撑 10 万 QPS，你会怎么做？

分层回答（这是系统设计题，要有层次感）：

第一层：前端 / 接入层

- 静态资源 CDN 化：**商品图片、秒杀页面的 HTML/JS 全部放 CDN，**让 90% 的流量根本不打到后端**。
- 前端限流 + 按钮置灰：**点击后立即禁用按钮，N 秒内不允许再点。这是最廉价有效的“第一道限流”。
- 答题/验证码前置：**秒杀前先做一次人机验证，**把脚本机器人挡在业务之外**（这是淘宝/京东的标配）。
- 秒杀 URL 动态化：**活动开始前才通过接口下发真实的秒杀 URL（带一次性 token），防止脚本提前拿到接口地址。

第二层：网关 / 负载均衡

- Nginx 层限流：**`limit_req_zone` + `limit_conn_zone`，按 IP 限流，**在流量进入应用之前就削掉大部分**。
- LVS + Keepalived** 做四层负载均衡。

3. **网关集群化**：Spring Cloud Gateway / Nginx 多实例，避免单点。

第三层：应用层

1. **服务无状态 + 水平扩容**：项目已经用 JWT 无状态了，直接加机器即可。**10 台 8 核机器大约能撑住 10 万 QPS 的简单接口。**
2. **本地缓存前置**：商品信息用 Caffeine，减少 Redis 访问。**但价格必须实时。**
3. **限流降级**：Sentinel / Hystrix 做熔断，防止级联故障。
4. **异步化**：Servlet 3.0 异步 / WebFlux / CompletableFuture，减少线程占用。10 万 QPS 下 Tomcat 默认 200 线程根本不够（每线程要处理 500 QPS，不可能）。

第四层：Redis 层（瓶颈所在）

1. **库存分片**：把一个商品的库存拆成 N 份（如 1000 个库存拆成 10 个 key，每个 100），**请求按 userId 哈希路由到不同的分片**。这样单个 key 的压力降低 N 倍。
代价：可能出现“某个分片抢光了但其他分片还有”的尴尬情况，需要更复杂的逻辑（如允许跨分片借调）。
2. **Redis Cluster / 主从读写分离**：秒杀场景是“读多写少”（大量请求读库存，少数写）。可以把读压力分散到从节点。
3. **多级缓存**：本地缓存标记“库存已售罄”，一旦售罄直接把后续请求挡在本地，完全不访问 Redis。**这是极其有效的优化——售罄后 Redis 压力归零。**

第五层：消息队列层

1. **Kafka 分区扩容**：秒杀 topic 设 10+ 分区，消费者 `concurrency` 相应提高。
2. **批量落库**：消费者攒一批消息（如 100 条）后**批量 INSERT**，把数据库写入从 10 万次降到 1000 次。
3. **削峰填谷**：Kafka 天然缓冲区，把瞬时 10 万 QPS 平滑成后端能承受的速率。

第六层：数据库层

1. **分库分表**：订单表按 userId 取模分片（ShardingSphere）。
2. **读写分离**：主库写、从库读。
3. **热点行优化**：单个商品的 `stock` 字段是热点行，InnoDB 行锁会严重串行化。**可以改成“多行库存”——把库存拆成 10 行，扣减时随机选一行**（这就是“库存分片”的数据库版本）。

第七层：兜底

1. **排队机制**：超过承载能力后，把请求**放进队列排队**，而不是直接失败（用户体验更好）。
2. **降级开关**：Redis 挂了直接降级为“活动暂不可用”，保护数据库。
3. **监控告警**：QPS、RT、错误率、Redis 内存、Kafka 堆积量，全链路监控（Prometheus + Grafana）。
4. **压测**：上线前用 JMeter / wrk / Gatling 做全链路压测，找出真实瓶颈。

项目 `ToDo.java` 里提到“解决压测失效问题”，说明作者是做过压测的。**如果面试官问“你压出了多少 QPS”，要能报出具体数字**（哪怕只是本地压测的 500 QPS，也比说“没测过”强）。

回答技巧：不要一股脑全说。先给出“分层削峰”的总体思路（**每一层都过滤掉一部分流量，越往后流量越小**），然后重点展开 2~3 个你认为最有效的点（本地缓存售罄标记、库存分片、批量落库），最后补充监控和压测。

第八级：Kafka 消息队列与最终一致性

Q8.1 项目用 Kafka 做了哪四件事？为什么每件都适合异步？

Topic	生产者	消费者	并发度	解决的问题
snappedUp	SnappedUpServiceImpl.snappedUp()	listenSnap()	3	削峰：把数据库写入从同步变异步，接口响应时间从几十 ms 降到几 ms
order	afterCommit 回调	listenOrder()	2	解耦：发邮件（SMTP 调用 1~3 秒）不阻塞主流程
verifyCode	UserServiceImpl.buildCode()	listenVerifyCode()	2	解耦：找回密码验证码邮件
regCode	UserServiceImpl.buildRegCode()	listenRegCode()	2	解耦：注册验证码邮件

为什么邮件场景特别适合异步？

- 1. 耗时：QQ SMTP 的一次完整调用（TLS 握手 + 认证 + 传输）通常 1~3 秒。同步发的话，用户点"发送验证码"要转圈 3 秒。
- 2. 不稳定：SMTP 服务器可能限流、超时、临时拒收。同步调用会把第三方的不稳定传递到自己的接口上——SMTP 挂了，注册功能就挂了。
- 3. 不是核心业务：邮件的成功与否不应该影响"注册成功"这个结论。
- 4. 可重试：Kafka 天然支持重试和重放。

为什么秒杀落库适合异步（削峰）？

秒杀的本质是"瞬时洪峰"。假设 10 万人在 1 秒内点击，但只有 1000 件库存：

- 同步方案：10 万个请求同时打数据库 → 数据库连接池打满 → 响应时间飙升 → 雪崩。
- 异步方案：Redis 挡掉 9.9 万个（库存不足），剩下 1000 个发 Kafka → 数据库只需要处理 1000 次写入，平滑分布在几秒内。

关键概念："削峰填谷"——Kafka 作为蓄水池，把瞬时的洪峰流量平滑成后端能承受的稳定流速。

Q8.2 为什么选 Kafka 而不是 RabbitMQ / RocketMQ？

维度	Kafka	RabbitMQ	RocketMQ
模型	发布订阅（Pull）	队列 / AMQP（Push）	发布订阅（Pull）
吞吐量	百万级/秒	万级/秒	十万级/秒
延迟	毫秒级（有 linger.ms）	微秒级	毫秒级
消息顺序	分区内有序	队列内有序	队列内有序
消息回溯	支持（按 offset 重放）	不支持（消费即删）	支持
事务消息	支持（较弱）	支持（AMQP 事务，性能差）	支持（效果好）
延迟消息	不支持（需自研）	支持（TTL + DLX）	原生支持
运维复杂度	高（依赖 ZK/KRaft）	低	中
适用场景	日志、大数据、高吞吐削峰	业务解耦、低延迟任务	电商交易、金融

本项目选 Kafka 的合理理由：

- **削峰场景需要高吞吐**。秒杀的消息量可能瞬时很高，Kafka 的批量化 + 顺序写磁盘 + 零拷贝，吞吐量远超 RabbitMQ。
- **持久化能力**：Kafka 的消息是**顺序追加写磁盘**（不是随机写），且默认保留 7 天，意外情况下可以重置 offset 重放。
- **消费位点可控**：手动 ack + 位点管理，精确控制"处理到哪了"。
- **生态与学习价值**：Kafka 是数据领域的事实标准，简历上有加分。

但要诚实地说不足之处：

1. 对"邮件"这种场景，Kafka 其实过重了。邮件量小（每天几百条）、需要延迟重试（5 分钟后重发）、需要死信队列。RabbitMQ 的 TTL + 死信队列更适合，或者干脆用更轻量的方案（数据库任务表 + 定时扫描）。
 2. 如果将来要做"订单 30 分钟未支付自动取消"，Kafka 不支持延迟消息，需要自己实现（用时间轮或定时扫描）。RocketMQ 原生支持 18 个延迟级别，这会是更优选择。
 3. 为了 4 个 topic 维护一套 Kafka + 消费位点管理，运维成本不低。如果团队小，用 RabbitMQ 可能更划算。
- 加分总结："选型要看场景。秒杀削峰选 Kafka 是对的；但如果只是为了发几种邮件，其实用 RabbitMQ 或者数据库任务表更合适。目前项目是'用一把锤子敲所有钉子'，这是一个可以优化点。"

Q8.3 Kafka 怎么保证消息不丢？

要从三个环节分别保证：

① 生产者不丢

- `acks=all`：等所有 ISR 副本确认。
- `retries=10`：失败自动重试。
- `enable.idempotence=true`：重试不会造成重复。
- 还要配 `min.insync.replicas ≥ 2`（broker 端），否则 ISR 缩到 1 时 `acks=all` 形同虚设。
- 项目缺失的：没有处理 `send()` 的返回值/异常。`kafkaTemplate.send()` 返回的是 `CompletableFuture`，不调用 `.get()` 或不加回调，发送失败会被完全忽略。

`KafkaTemplate` 有一个 `setProducerListener` 或可以注册 `Callback`：

```
kafkaTemplate.send(topic, key, value)
    .whenComplete((result, ex) -> {
        if (ex != null) {
            log.error("消息发送失败", ex);
            // 触发补偿：回滚 Redis / 落本地消息表
        }
    });
```

② Broker 不丢

- **副本因子 (replication.factor) ≥ 3**：每个 partition 有 3 个副本。
- `min.insync.replicas = 2`：至少 2 个副本写入成功才算成功。
- `unclean.leader.election.enable = false`：禁止非 ISR 副本被选为 leader（防止丢失数据）。
- **刷盘策略**：Kafka 默认依赖 OS 的页缓存刷盘（性能优先）。如果要强一致可以配 `log.flush.interval.messages=1`（性能暴跌，一般不用）。

③ 消费者不丢

- **手动 ack**：业务处理成功后才提交位点（项目已用）。
- **不要异步提交位点**：如果异步提交成功后业务才失败，消息就丢了。

本项目消费者端的丢失场景（关键）：

```
// listenOrder / listenVerifyCode / listenRegCode
} catch (Exception e) {
    log.error("发送订单邮件失败", e);
    ack.acknowledge(); // ← 邮件失败也 ack，消息永久丢失！
}
```

这三个方法在邮件发送失败时仍然 **ack**，意味着消息被永久丢弃，用户永远收不到邮件。这是一个明确的可靠性缺陷。

为什么作者这么写？大概率是故意的取舍——邮件失败重试容易陷入无限循环（比如邮箱地址格式错误，重试一万次也没用），而且邮件不重要。但正确的做法是“有限重试 + 死信队列 + 告警”，而不是静默丢弃。

面试中这样回答很加分：“我理解这里是为了避免毒丸消息阻塞消费，但更好的做法是引入重试上限和死信队列。我的项目里用的是简单方案，如果做生产级系统我会用 `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`。”

Q8.4 什么是“毒丸消息”？项目有防范吗？

毒丸消息 (Poison Pill)：一条永远无法被成功处理的消息。

本项目最可能出现的毒丸场景：

```
// listenSnap 里
long userId = Long.parseLong(record.key()); // ① key 不是数字 → NumberFormatException
ProductDO productDO = JSON.parseObject(record.value(), ProductDO.class); // ② JSON 格式错误 → JSONException
snappedService.doSnapBusiness(userId, productDO.getId(), productDO.getPrice()); // ③ 商品不存在 → NPE

} catch (Exception e) {
    log.error("kafka消费失败", e);
    throw e; // ← 不 ack，无限重试！
}
```

后果：这条消息会被无限次重新拉取、重新失败，消费者线程被永久占用，整个分区的信息不再前进（消息堆积）。

更糟的是：因为消费位点（offset）是分区级别的，一条毒丸消息会卡住整个分区，后面的所有消息都消费不了。如果这个分区里有 10 万个抢购消息，全部停滞——数据库库存永远不扣、订单永远不创建、邮件永远不发。

防范方案：

1. 重试次数限制 + 死信队列（推荐）：

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<String, String> template) {
    // 重试 3 次，每次间隔 1 秒
    FixedBackOff backOff = new FixedBackOff(1000L, 2);
    // 超过重试次数后，投递到死信 topic
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(template,
        (record, ex) -> new TopicPartition("snappedUp.DLT", record.partition()));
    DefaultErrorHandler handler = new DefaultErrorHandler(recoverer, backOff);
    // 明确指定哪些异常需要重试
    handler.addNotRetryableExceptions(NumberFormatException.class, JSONException.class);
    return handler;
}
```

核心思想：可重试的异常（如数据库临时不可用）重试，不可重试的异常（如数据格式错误）直接进死信队列。

2. 在 ack 之前先做参数校验，解析失败直接 ack + 告警，不抛异常：

```
long userId;
try {
```

```

    userId = Long.parseLong(record.key());
} catch (NumberFormatException e) {
    log.error("非法消息 key: {}", record.key());
    ack.acknowledge(); // 丢弃，不阻塞
    return;
}

```

3. 用 `@KafkaListener` 的 `errorHandler` 属性或全局 `CommonErrorHandler`。
4. 监控告警：consumer lag（堆积量）超过阈值就报警。

项目现状：没有任何防范。 `listenSnap` 的 `throw e` 就是典型的“毒丸放大器”。这是一个必须主动承认的缺陷。

Q8.5 Kafka 的 rebalance 是什么？对项目有什么影响？

Rebalance（再均衡）：消费者组内重新分配 partition 归属的过程。

触发条件：

- 消费者加入或离开（正常启停、宕机）。
- 消费者心跳超时（`session.timeout.ms`，本项目 30 秒）。
- 消费者处理超时（`max.poll.interval.ms`，本项目 5 分钟）——两次 `poll()` 间隔超过这个时间，会被踢出组。
- 订阅的 topic 分区数变化。

本项目的影响：

应用重启、扩容、或者某次消费卡住（如毒丸消息），都会触发 rebalance。

Rebalance 期间的问题：

1. 消费暂停：整个消费者组会短暂停止消费（Stop-The-World），消息堆积。
2. 重复消费：partition 从旧消费者转移到新消费者时，如果已消费的消息没有 ack，新消费者会从头消费——这正是本项目 `DuplicateKeyException` 处理逻辑要应对的场景。项目的幂等设计在这里发挥了作用。
3. 频繁 rebalance 会严重影响吞吐。Kafka 2.4+ 引入了增量协作再均衡（Cooperative Rebalancing），可以避免“全部停止”，但需要配 `partition.assignment.strategy=CooperativeStickyAssignor`。项目没配。

三个超时参数的关系（高频面试题）：

参数	含义	本项目
<code>heartbeat.interval.ms</code>	消费者向 coordinator 发心跳的间隔	10000 (10s)
<code>session.timeout.ms</code>	coordinator 多久收不到心跳判定消费者死亡	30000 (30s)
<code>max.poll.interval.ms</code>	两次 poll 的最大间隔（处理能力的上限）	300000 (5min)

关系：`heartbeat.interval.ms` 应该约为 `session.timeout.ms` 的 **1/3**（3 次心跳丢失才判定死亡，容忍 1~2 次网络抖动）。

注意：心跳是由**独立的线程**发送的，所以即使你的业务代码在阻塞处理，心跳仍在发——这也是为什么 `max.poll.interval.ms` 是必要的：它专门检测“消费者活着但处理太慢”的情况。

调优建议：本项目的单条消息处理很快（1 次 UPDATE + 2 次 INSERT），`max.poll.interval.ms=300000` 是安全的。`max-poll-records=50` 意味着一次最多处理 50 条， $50 \times$ 单条耗时 < 5 分钟即可。

Q8.6 事务提交后才发消息，会不会消息发不出去？怎么保证？

会。`afterCommit` 里的 `kafkaTemplate.send()` 如果抛异常或网络失败，消息就丢了，且没有任何补偿。

这是“本地事务 + 消息发送”的经典难题，业界有几种成熟方案：

① 本地消息表（最经典，推荐）

```
@Transactional
public void doSnapBusiness(...) {
    // ① 业务操作（扣库存、插订单）
    productDAO.reduceStock(productId, 1);
    orderDAO.insertOrder(orderDO);

    // ② 同一个事务里插入一条"待发送消息"记录
    messageDAO.insert(new LocalMessage(orderDO.getId(), "order",
                                         JSON.toJSONString(orderDO), PENDING));

    // 两个操作在同一个事务里 → 原子
}

// 独立线程/定时任务异步投递
@Scheduled(fixedDelay = 1000)
public void sendPendingMessages() {
    List<LocalMessage> messages = messageDAO.findPending(100);
    for (LocalMessage msg : messages) {
        try {
            kafkaTemplate.send(msg.getTopic(), msg.getPayload()).get(3, TimeUnit.SECONDS);
            messageDAO.markSent(msg.getId()); // 标记已发送
        } catch (Exception e) {
            messageDAO.incrementRetry(msg.getId()); // 重试计数
        }
    }
}
```

优点：彻底解决（业务和消息在同一事务里原子）。**缺点：**多一次数据库写入，且有延迟（最多 1 秒）。

② Kafka 事务消息

```
// 生产者开启事务
producer.beginTransaction();
try {
    // 本地数据库操作
    orderDAO.insertOrder(orderDO);
    // 发送 Kafka 消息（在事务里）
    kafkaTemplate.send("order", ...);
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
}
```

严格说，Kafka 事务无法和数据库事务原子绑定——它只能保证"多条 Kafka 消息"之间的原子性，不能保证"数据库操作 + Kafka 消息"的原子性（除非用 XA 分布式事务，那更重）。

③ 定时对账补偿（最终兜底）

```
@Scheduled(cron = "0 0/5 * * * ?") // 每 5 分钟
public void reconcileOrders() {
    // 找出所有"订单已创建但没有对应的邮件发送记录"的订单
    List<OrderDO> missed = orderDAO.findUnnotifiedOrders();
    for (OrderDO order : missed) {
        kafkaTemplate.send("order", ...); // 补发
    }
}
```

简单有效，是所有方案的兜底。缺点是"发现延迟"（最多 5 分钟）。

④ 项目当前的做法与改进

当前：afterCommit 里裸调 send()，无任何保护。

最小成本改进：

```
@Override
public void afterCommit() {
    try {
        kafkaTemplate.send(Topics.KAFKA_ORDER_TOPIC, String.valueOf(orderDO.getUserId()),
                           JSON.toJSONString(orderDO))
                           .get(3, TimeUnit.SECONDS);    // 同步等待结果
    } catch (Exception e) {
        // 消息发送失败 → 记录到失败表，由定时任务补偿
        log.error("订单消息发送失败，orderId={}", orderDO.getId(), e);
        failedMessageDAO.insert(orderDO.getId(), "order", JSON.toJSONString(orderDO));
    }
}
```

加上定时补偿任务，就能做到"至少一次"投递。配合消费者端的幂等（唯一索引），实现"effectively-once"。

Q8.7 邮件发送为什么不用线程池而用 Kafka?

这是一个很好的对比题。

维度	线程池 (@Async)	Kafka
实现复杂度	极简（一个注解）	需要部署 Kafka + 配置
可靠性	进程重启/宕机 → 任务丢失	消息持久化，重启后继续消费
重试	需自己实现	内置重试 + 位点管理
削峰	队列满了会拒绝或阻塞	可堆积 TB 级消息
多实例协作	每个实例自己消费自己的任务	消费者组自动分配分区，不重复消费
可观测性	差（线程池监控有限）	好（lag、offset 可视化）

结论：

- 如果邮件"丢了也无所谓"（比如营销邮件），线程池足够了，简单省事。
- 如果邮件"必须发出去"（订单确认、验证码），Kafka 的持久化能力是关键优势。用户注册时服务重启，用线程池的方案验证码就永远发不出去了。

另外，Kafka 还天然解决了"多实例重复发送"的问题：如果用线程池 + 定时扫描数据库任务表，两个实例都会扫到同一批待发邮件，导致重复发送。而 Kafka 的消费者组保证一条消息只被一个实例消费。

但要诚实承认：本项目用 Kafka 发邮件是"杀鸡用牛刀"。

- 邮件量小（每天几百封），Kafka 的吞吐优势完全没用上。
- 为了 2 个邮件 topic 引入一套 Kafka 集群，运维成本不成比例。
- 更合适的是：RabbitMQ（轻量、支持延迟队列和死信队列，正好适合"5 分钟后重发"的场景）或者 数据库任务表 + 分布式锁 + 定时扫描（零额外组件）。

面试时可以这样说："用 Kafka 发邮件在技术上是 对的（持久化 + 削峰 + 多实例不重复），但从架构经济性看是过度设计。如果重构，我会把邮件拆出来用更轻量的方案，把 Kafka 专注在秒杀削峰这个真正需要高吞吐的场景。"

第九级：WebSocket 实时通信

Q9.1 为什么要用 WebSocket？和轮询、SSE、长轮询比有什么优势？

方案	原理	实时性	服务器开销	适用场景
短轮询	客户端每 N 秒发一次 HTTP 请求	差（最多延迟 N 秒）	高（大量无效请求）	数据变化不频繁
长轮询	请求挂起，有数据才返回，然后立刻再发	较好	中（连接频繁建立/断开）	兼容性要求高
SSE	服务端单向推流（ <code>text/event-stream</code> ）	好	低	只需服务端推送（如股票行情）
WebSocket	HTTP 握手后升级为全双工 TCP 长连接	最好	低（连接常驻）	双向实时通信

本项目的需求：实时显示"当前在线人数"和"总注册用户数"，并且需要接收客户端心跳。

为什么 WebSocket 最合适：

- 真正的双向：**服务端可以主动推（在线人数变化），客户端也可以主动发（心跳）。SSE 只能单向。
- 低开销：**建立一次连接后，数据帧的头部只有 **2~10 字节**（相比 HTTP 每次都要带几百字节的头）。短轮询方案里，客户端每 5 秒发一次请求，光 HTTP 头就浪费几 KB。
- 低延迟：**无需等待轮询周期，服务端有变化立刻推。

项目的轮询历史：代码注释里能看到 `OnlineStats.vue` 每 5 秒执行 `GET /online/stats` ——说明前端同时存在轮询和 WebSocket。这是一个可以优化点：有了 WebSocket 推送，`/online/stats` 的轮询就应该停掉。

Q9.2 WebSocket 的握手过程是怎样的？为什么可以升级协议？

WebSocket 复用了 HTTP 的握手，然后"升级"协议：

```
① 客户端发送 HTTP 请求（带特殊的 Upgrade 头）
GET /ws/user?token=xxx HTTP/1.1
Host: localhost:8081
Upgrade: websocket           ← 关键
Connection: Upgrade         ← 关键
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ== ← 16 字节随机值 Base64
Sec-WebSocket-Version: 13

② 服务端返回 101 Switching Protocols
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o= ← 计算得出

③ 之后连接升级为全双工 TCP，不再使用 HTTP 协议
   双方通过 WebSocket 数据帧（Frame）通信
```

Sec-WebSocket-Accept 的计算：

```
Base64( SHA1( Sec-WebSocket-Key + "258EAF45-E914-47DA-95CA-C5AB0DC85B11" ) )
```

那个 GUID 是 RFC 6455 规定的固定字符串。**这个计算的意义：**① 证明服务端真的理解 WebSocket 协议（不是随便一个 HTTP 服务器）；② **防止被中间代理缓存**（因为 key 是随机的，每次响应的 Accept 都不同，缓存没有意义）。

本项目的配置：

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(userWebSocketHandler, "/ws/user")
            .setAllowedOrigins("*"); // 允许跨域
    }
}
```

关于 `setAllowedOrigins("*")`：

这是**必要的**——因为 WebSocket 握手也是 HTTP 请求，会受同源策略影响。如果前端部署在 `localhost:5173`（Vite 默认端口），后端在 `8081`，不配这个连不上。

但这是一个安全风险：允许任意域名建立 WebSocket 连接 = 允许跨站 WebSocket 劫持（CSWSH）。攻击者可以在自己的恶意网站上用受害者的 Cookie 建立 WebSocket 连接（如果认证靠 Cookie 的话）。

本项目因为用的是 URL 参数传 token，攻击者拿不到 token，所以风险较低。但生产环境仍应该配白名单域名。

另外，`/ws/**` 在拦截器里被排除了，因为 WebSocket 握手请求无法自定义 Header（浏览器的 WebSocket API 不支持设置 header），所以 token 只能通过 URL 参数传。这是 WebSocket 认证的经典限制。

但用 URL 传 token 有自己的问题：① token 会出现在服务器访问日志、Nginx 日志、浏览器历史里，泄露风险高；② 网络中间设备可能记录完整 URL。

更好的方案：① 用 `Sec-WebSocket-Protocol` 子协议头传 token（浏览器 API 支持这个）；② 或者先通过 HTTP 接口换取一个一次性的、短时效的 WebSocket ticket，用 ticket 建立连接。

Q9.3 在线用户是怎么统计的？ConcurrentHashMap 和 Redis ZSet 各起什么作用？

两个数据结构，职责不同：

① 内存：ConcurrentHashMap（连接管理）

```
// userId -> WebSocketSession（用于给用户推送消息）
private final ConcurrentHashMap<Long, WebSocketSession> sessions = new ConcurrentHashMap<>();

// sessionId -> userId（用于连接断开时反查用户）
private final ConcurrentHashMap<String, Long> sessionToUser = new ConcurrentHashMap<>();
```

为什么需要两个 Map？

- `sessions`：正向查找——要广播消息给某个用户时，需要 `userId → session`。
- `sessionToUser`：反向查找——连接断开时，Spring 只给你一个 `WebSocketSession`，你无法从 session 直接得知 `userId`（除非用 `session.getAttributes()` 存），所以要维护一个反向索引。

更好的做法：其实 Spring 的 `WebSocketSession.getAttributes()` 就是一个 Map，在握手时可以塞入 `userId`，不需要额外维护反向 Map。但当前实现用双 Map 也能工作，只是多占一份内存。

为什么用 `ConcurrentHashMap` 而不是 `HashMap`？

因为 WebSocket 的连接建立/断开/广播是在不同线程发生的（Tomcat 的 NIO 线程、Spring 的 `@Scheduled` 定时任务线程）。`HashMap` 在并发场景下会：① 数据不一致；② JDK 7 及以前并发扩容会导致链表成环、CPU 100%（JDK 8 改进了但仍不安全）。

`ConcurrentHashMap` 的保证：① 单个操作（put/get/remove）是原子的；② 但复合操作不是，本项目里：

```
// 这段是安全的：put 返回旧值，是原子的
WebSocketSession old = sessions.put(userId, session);
if (old != null && old.isOpen()) {
    sessionToUser.remove(old.getId());
    try { old.close(); } catch (IOException ignored) {}
}
```

因为 `ConcurrentHashMap.put()` 是原子的，同一个 `userId` 的两个并发连接，只有一个能拿到旧值，所以“挤掉旧连接”的逻辑不会重复执行。这是正确的用法。

② Redis: ZSet `online:users` (跨实例在线状态)

```
// member = userId, score = 最后心跳时间戳
stringRedisTemplate.opsForZSet().add("online:users", String.valueOf(userId), System.currentTimeMillis());
```

为什么还需要 Redis？内存里不就有吗？

- 多实例部署时，`sessions` 只在单个 JVM 内。实例 A 不知道实例 B 上有哪些用户在线。Redis 是跨实例的共享状态。
- 用 ZSet 而不是 Set 的关键原因：`score` 存心跳时间戳，可以按时间范围清理过期连接：

```
// 清理 2 分钟无心跳的记录
long threshold = System.currentTimeMillis() - 120_000;
stringRedisTemplate.opsForZSet().removeRangeByScore("online:users", 0, threshold);
```

如果用 Set，就只能存 `userId`，无法判断谁过期了——除非每个 `userId` 单独存一个带 TTL 的 key，那样又无法一次拿到全部在线用户。ZSet 在这里是最优选择。

查询在线人数：`ZCARD online:users`， $O(1)$ 复杂度。

查询在线列表：`ZREVRANGE online:users 0 499`，取 `score` 最大的 500 个（最近活跃的）， $O(\log N + M)$ 。

Q9.4 心跳机制是怎么设计的？为什么需要它？

问题：TCP 连接“看起来还活着”，但实际已经断了。

典型的“半开连接”场景：

- 用户手机走出 WiFi 范围、进入电梯、地铁隧道——网络断了，但客户端和服务端都不知道。
- 客户端没有发 FIN 包，服务端的 TCP 连接依然处于 ESTABLISHED 状态。TCP 的 `keepalive` 默认需要 2 小时才检测到 (Linux `tcp_keepalive_time`)。
- 结果：服务端一直认为这个用户在“在线”，实际他早就下线了。在线人数虚高。

本项目的解决方案：应用层心跳。

```
// 客户端每 15 秒发送一条消息（代码注释：OnlineTracker.vue 每十五秒发送消息更新心跳时间）
@Override
protected void handleMessage(WebSocketSession session, TextMessage message) {
    try {
        Long userId = sessionToUser.get(session.getId());
        if (userId != null) {
            // 更新 ZSet 里的 score 为当前时间戳 = 刷新“最后活跃时间”
            stringRedisTemplate.opsForZSet().add("online:users", String.valueOf(userId),
                System.currentTimeMillis());
        }
    } catch (Exception e) {
        log.warn("心跳处理异常，但不关闭连接", e);
    }
}
```

定时清理任务：

```
@Scheduled(fixedRate = 30000)    // 每 30 秒执行一次
public void cleanStaleOnlineUsers() {
    long threshold = System.currentTimeMillis() - 120_000;    // 2 分钟前
    Long removed = stringRedisTemplate.opsForZSet()
        .removeRangeByScore("online:users", 0, threshold);
    if (removed != null && removed > 0) {
        log.info("在线用户清理：移除 {} 个过期连接", removed);
    }
    userWebSocketHandler.setOnlineCountFromRedis();    // 同步内存计数器
}
```

参数设计分析（15s 心跳 / 30s 清理 / 2min 阈值）：

参数	值	推导
客户端心跳间隔	15 秒	足够频繁，能及时发现断线；又不会太频繁浪费资源
清理任务间隔	30 秒	= 2 倍心跳间隔，保证不会误杀（一个心跳周期内不会被清理）
过期阈值	120 秒	= 8 倍心跳间隔， 容忍连续 7 次心跳丢失 ，网络抖动不会误判

这个设计是合理的：容忍度足够（不会误杀），又能及时清理（最多 2 分半钟发现僵尸连接）。

但有几个问题：

1. Redis 里的记录被清理了，但 JVM 里的 `sessions` 和 `sessionToUser` 没有清理！

```
// OnlineUserTask 只做了这些：
stringRedisTemplate.opsForZSet().removeRangeByScore(ONLINE_USERS_KEY, 0, threshold);
userWebSocketHandler.setOnlineCountFromRedis();
// ← sessions 和 sessionToUser 里的僵尸连接依然存在！
```

后果：① 内存泄漏——`sessions` Map 里的 `WebSocketSession` 对象（包含缓冲区等资源）永远不被回收；② 广播消息时，`for (Long userId : sessions.keySet())` 会遍历到僵尸连接，`sendToUser` 里虽然检查了 `session.isOpen()` 会跳过，但僵尸连接会被一直遍历；③ 如果这个用户重连，`sessions.put()` 会返回旧 session 并尝试关它，虽然没大问题但语义混乱。

修复：清理任务里应该同时从 `sessions` 和 `sessionToUser` 中移除对应的条目。这是一个真实的 bug。

- 两处心跳逻辑重复：`UserWebSocketHandler.handleTextMessage()` 更新 ZSet 的 score，而 `OnlineUserTask` 清理它。但 `handleTextMessage` 里的 `sessions` 没有更新任何本地状态（比如“最后活跃时间”），导致清理任务只能查 Redis，无法知道本地哪些连接是僵尸的。这正是问题 ① 的根因。
- 没有服务端主动 ping。标准的 WebSocket 心跳是双向的：服务端也应该周期性发 `Ping` 帧，客户端回 `Pong`。这样即使客户端没有实现心跳逻辑（比如被恶意客户端绕过），服务端也能检测出死连接。当前实现完全依赖客户端的自觉——如果客户端不发心跳，这个连接永远不会被清理（除了 Redis 记录被删，但 JVM 里的 session 还在，见问题 ①）。
- `cleanStaleOnlineUsers` 没有分布式锁。多实例部署时，每个实例都在同时执行这个清理任务，做重复工作。虽然 `removeRangeByScore` 是幂等的（不会误删），但多个实例同时跑定时任务是一种浪费，且如果有其他副作用（如计数修正）就会冲突。应该用 Redisson 的分布式锁保证同一时刻只有一个实例执行。同理，`UserWebSocketHandler.scheduledBroadcast()` 也是每个实例都在跑。

Q9.5 在线人数广播的“内存计数器漂移”问题是怎么解决的？

问题背景：每次广播都查 `ZCARD online:users` 是有代价的（一次 Redis 往返）。但如果在内存里维护一个计数器 `AtomicInteger onlineCount`，它和 Redis 的真实值会逐渐不一致（漂移）。

漂移的来源：

1. **异常断开**: 连接因为网络问题断了, `afterConnectionClosed` 可能不被调用 (或者被调用了但 Redis 操作失败), 导致 `decrementAndGet()` 没执行。
2. **重复计数**: 同一用户重连时, `sessions.put()` 挤掉了旧连接 (覆盖, 尺寸不变), 但代码**无条件执行了** `onlineCount.incrementAndGet()` :

```
WebSocketSession old = sessions.put(userId, session); // 覆盖, sessions.size() 不变
if (old != null && old.isOpen()) { ...关闭旧连接... }
sessionToUser.put(session.getId(), userId);
stringRedisTemplate.opsForZSet().add(ONLINE_USERS_KEY, ...); // ZSet 里也是同一个 member, 不增加
onlineCount.incrementAndGet(); // ← 但计数器 +1 了! 漂移产生!
```

这是一个真实的 bug: 同一个用户重连 (比如页面刷新、网络切换), `onlineCount` 会 +1, 但 Redis ZSet 和 `sessions` 的实际数量都没变。**结果是在线人数虚高。**

3. **`afterConnectionClosed` 里也有一处 bug**:

```
@Override
public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
    Long userId = sessionToUser.remove(session.getId());
    if (userId != null) {
        sessions.remove(userId, session);
        stringRedisTemplate.opsForZSet().remove(ONLINE_USERS_KEY, String.valueOf(userId));
    }
    onlineCount.decrementAndGet(); // ← 无论 userId 是否为 null 都减!
}
```

如果 `userId == null` (比如连接在握手阶段就被拒绝了, 从未进入 `sessionToUser`), 计数器仍然会 -1。而那些被拒绝的连接从来没有 +1 过 (`afterConnectionEstablished` 里如果 token 无效直接 `closeSession` 并 return, 没有 increment)。这会导致计数器偏小, 甚至变成负数。

修复:

```
@Override
public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
    Long userId = sessionToUser.remove(session.getId());
    if (userId == null) return; // 从未注册的连接, 直接返回, 不计数
    sessions.remove(userId, session);
    stringRedisTemplate.opsForZSet().remove(ONLINE_USERS_KEY, String.valueOf(userId));
    onlineCount.decrementAndGet();
}
```

项目采用的"定期校准"方案

```
@Scheduled(fixedDelay = 2000)
public void scheduledBroadcast() {
    broadcastRound++;
    int current;

    // 每 10 轮 (20 秒) 用 Redis 真实值校准一次内存计数器
    if (broadcastRound % 10 == 0) {
        Long redisCount = stringRedisTemplate.opsForZSet().zCard(ONLINE_USERS_KEY);
        current = redisCount != null ? redisCount.intValue() : onlineCount.get();
        onlineCount.set(current); // ← 用 Redis 权威值覆盖内存值
    } else {
        current = onlineCount.get(); // 其他轮次用内存值 (省 Redis 查询)
    }

    if (current == lastBroadcastCount) return; // 值没变则不广播
    lastBroadcastCount = current;
}
```

```
String message = JSON.toJSONString(Map.of("type", "onlineCount", "count", current));
TextMessage textMessage = new TextMessage(message);
for (Long userId : sessions.keySet()) {
    sendToUser(userId, textMessage);
}
}
```

这是“内存计数器 + 定期校准”的经典设计模式：

- **性能**：19 次广播只读内存（0 次 Redis 往返），1 次读 Redis。相比每次都查，Redis 压力降低到 1/10。
- **准确性**：最多 20 秒的漂移窗口，然后被纠正。对于“在线人数”这种展示型指标，20 秒的误差完全可以接受。
- **避免无效广播**：`if (current == lastBroadcastCount) return;`——人数没变就不广播。这是一个很重要的优化：如果在线人数稳定在 100 人，那就完全不推送任何消息，节省了带宽和客户端处理。

还有一处细节做得很好——服务重启时从 Redis 恢复计数：

```
@PostConstruct
public void initOnlineCount() {
    Long count = stringRedisTemplate.opsForZSet().zCard(ONLINE_USERS_KEY);
    int c = count != null ? count.intValue() : 0;
    onlineCount.set(c);
    lastBroadcastCount = c;
    if (c > 0) log.info("从 Redis 恢复在线人数: {}", c);
}
```

为什么重要：如果服务重启，内存计数器归零，但 Redis 里的在线记录还在（用户还没断线，只是服务重启了，实际上连接已经断了但 Redis 记录还在 TTL 内）。不恢复的话，重启后在线人数会从 0 开始，和真实值差很多。

但这里还有个更深的问题：服务重启后，所有 WebSocket 连接其实都断了（TCP 连接随进程消失），但 Redis 里的记录还在（因为心跳机制要靠 2 分钟超时才能清理）。所以重启后 `initOnlineCount` 恢复到的是“重启前的在线人数”，而实际上是 0。在接下来的 2 分钟内，会显示一个虚高的在线人数，直到清理任务跑完。

更好的做法：重启时不恢复计数（直接用 0），或者给 `online:users` 加一个**实例标识**（每个实例维护自己的 ZSet key），重启时清掉自己那部分。

Q9.6 WebSocket 在多实例部署下有什么问题？怎么解决？

核心问题：`WebSocketSession` 是 JVM 内存对象，无法跨进程共享。

场景：用户 A 连在实例 1，用户 B 连在实例 2。现在要给用户 B 推送一条消息，但消息处理发生在实例 1——实例 1 的 `sessions` Map 里没有 B，推不过去。

本项目当前的影响：

- **在线人数统计是准确的**（因为用的是 Redis ZSet，跨实例共享）。✅
- **广播也是“准确的”（但低效）**：每个实例的 `scheduledBroadcast` 都在跑，各自检查自己的内存计数器，然后只广播给连在自己身上的用户。所以最终每个用户都能收到广播。
但代价是：N 个实例都在查 Redis（虽然只有 1/10 的频率）、都在做广播循环。这是 N 倍的浪费。
- `sendToUser(userId, message)` **无法跨实例定向推送**。目前项目里这个方法没有被使用（只有广播在用），所以问题没暴露。但一旦要实现“给某个用户发通知”，就会失败。

解决方案：

① Redis Pub/Sub（最常用，轻量）

```
// 发送方：往 Redis 频道发消息
redisTemplate.convertAndSend("ws:broadcast", JSON.toJSONString(message));
```

```
// 每个实例都订阅这个频道
@Bean
RedisMessageListenerContainer container(...) {
    container.addMessageListener((message, pattern) -> {
        // 收到消息后，推给自己的连接
        String body = new String(message.getBody());
        userWebSocketHandler.broadcastLocal(body);
    }, new ChannelTopic("ws:broadcast"));
    return container;
}
```

优点：简单、Redis 已经在用了、实时性好。**缺点：**Redis Pub/Sub **不保证可靠投递**（订阅者掉线期间的消息会丢、没有持久化）。对于“在线人数广播”这种可以丢的消息完全够用。

② Redis Stream（可靠的 Pub/Sub）

相比 Pub/Sub，Stream 有持久化、消费者组、ACK 机制，消息不会丢。但更重。

③ 专门的 MQ 广播

用 Kafka / RabbitMQ 的广播交换机，每个实例作为独立消费者订阅。项目已经有 Kafka 了，可以直接用。

④ 网关层粘性会话（最简单但不推荐）

用 `ip_hash` 让同一用户的请求固定路由到同一实例。**缺点：**① 实例扩容时大量重连；② 负载不均；③ 实例宕机时该实例上的用户全部掉线。**只适合临时方案。**

⑤ 专门的推送服务 / 网关

大规模系统（如微信、QQ）会把 **WebSocket 连接独立出来**做成“接入层”（Netty 集群 + 注册中心），业务服务通过 RPC 调用接入层推送。**这是终极方案，但复杂度高。**

本项目现状：完全没有处理多实例问题。因为当前是单实例部署（`docker-compose.yml` 里只有一个 backend 服务），所以问题没暴露。**面试官问“如果部署两个实例会怎样”，这就是标准答案。**

Q9.7 WebSocket 连接数有上限吗？单机能支持多少连接？

理论限制主要来自四个维度：

① 操作系统文件描述符

```
# Linux 默认单进程 1024，需要调大
ulimit -n                # 查看
ulimit -n 1000000        # 临时调整
# 永久调整：/etc/security/limits.conf
# * soft nofile 1000000
# * hard nofile 1000000
```

在 Java 中，`WebSocketSession` 每个连接会占用一个文件描述符。

② 端口数量

服务端只用一个监听端口（如 8081），所有连接通过**四元组（源 IP + 源端口 + 目标 IP + 目标端口）**区分。所以服务端端口不限制连接数，**限制在客户端**（同一台客户端机器最多 ~65535 个端口，所以单个客户端最多约 6 万个连接）。

③ JVM 内存

每个 `WebSocketSession` 对象（Tomcat 实现 + 缓冲区）大约占用 **10~50 KB**。

1 万连接 $\approx 100\text{ MB} \sim 500\text{ MB}$
10 万连接 $\approx 1\text{ GB} \sim 5\text{ GB}$

这是本项目最需要注意的：JVM 堆要按连接数预留。项目里 `sessions` 是 `ConcurrentHashMap<Long, WebSocketSession>`，如果僵尸连接不清理（前面 Q9.4 提到的问题 ①），内存会持续泄漏直到 OOM。

④ 线程模型

这是关键点。如果每个连接占一个线程（BIO 模型），1 万连接就要 1 万个线程——每线程 1MB 栈空间，就是 10 GB，直接 OOM。

Tomcat NIO 模型（Spring Boot 默认）：少量线程（`maxThreads` 默认 200）通过 **Selector 多路复用** 处理大量连接。只有真正有数据读写时才占用线程。所以 1 万连接只需要几百个线程。

但如果用了 `@Scheduled` 广播 + `synchronized (session)` 写消息，还是要小心：

```
public void sendToUser(Long userId, TextMessage message) {
    WebSocketSession session = sessions.get(userId);
    if (session == null || !session.isOpen()) return;
    synchronized (session) {           // ← 加锁保证同一 session 不并发写
        try {
            session.sendMessage(message);
        } catch (IOException e) { ... }
    }
}
```

为什么要 `synchronized (session)`？因为 Spring 的 `WebSocketSession` 不是线程安全的。如果多个线程同时调用 `sendMessage()`，会导致 `IllegalStateException: TEXT_PARTIAL_WRITING`（并发写状态冲突）。这是一个很常见的坑，项目里处理对了。

但 `synchronized (session)` 在高并发广播时是瓶颈：

```
for (Long userId : sessions.keySet()) {
    sendToUser(userId, textMessage); // 串行发送！逐个加锁、逐个 send
}
```

1 万个连接串行发送：即使每次只需 0.1ms，也要 1 秒。而广播是每 2 秒一次——意味着一轮广播还没发完，下一轮就开始了，形成积压。

优化方案：① 用 `ConcurrentWebSocketSessionDecorator`（Spring 提供），它内部用队列处理并发写，不用外部加锁；② 用**线程池并行发送**（但要控制并发度）；③ 或者用 Netty 的原生 `WebSocket`（性能更好）。

实际数据参考：

- 单机 8 核 16GB，Tomcat NIO，支撑 1~5 万个空闲 `WebSocket` 连接是可行的。
- 如果要 10 万+，需要用 **Netty 直接处理**（省去 Tomcat 的抽象开销），或者横向扩展多实例。

第十级：并发、JVM 与性能调优

Q10.1 项目的并发点有哪些？分别怎么处理的？

并发点	风险	项目方案	是否可靠
秒杀库存扣减	超卖	Redis Lua 原子脚本 + SQL <code>stock > 0</code> 守卫	✅ 可靠
秒杀一人一单	重复抢购	Redis key + 数据库唯一索引	✅ 可靠
选课人数扣减	超选	SQL <code>remaining > 0</code> 守卫	⚠️ 无事务，可能不一致
选课重复检查	重复选课	应用层查询（先查后插）	❌ 有竞态，应加唯一索引
订单号生成	重复	Redisson <code>RAAtomicLong</code> (INCR 原子)	✅ 可靠
限流计数	计数不准	Redis Lua 原子脚本	✅ 可靠
在线人数计数	漂移	AtomicInteger + 定期校准	⚠️ 重连时有 bug
WebSocket session 写	并发写冲突	<code>synchronized (session)</code>	✅ 可靠但低效
缓存更新	脏数据	无锁（依赖“更新 DB 后更新缓存”的顺序）	⚠️ 极端并发下会不一致
定时任务	多实例重复执行	无处理	❌ 多实例下重复执行

总结：项目在“秒杀”这条核心链路上的并发处理是可靠的（Redis 原子 + 数据库守卫 + 唯一索引，三层防护）；但在“选课”、“缓存更新”、“定时任务”这几个次要路径上存在并发缺陷。

面试时主动这样总结是加分项：“我把主要的并发防护力量放在了秒杀链路上，因为它是最核心、并发最高的场景；选课属于中低并发，我承认它的防护做得不够严谨（缺少唯一索引和事务），如果要做生产级系统我会补齐。”

Q10.2 AtomicInteger 和 volatile 在项目里怎么用的？

```
// UserWebSocketHandler
private final AtomicInteger onlineCount = new AtomicInteger(0);
private volatile int lastBroadcastCount = -1;
private int broadcastRound = 0;
```

AtomicInteger —— 原子性

作用：保证 `incrementAndGet()` / `decrementAndGet()` 是原子的。

原理：CAS (Compare-And-Swap) + volatile

```
// AtomicInteger.incrementAndGet() 底层
public final int incrementAndGet() {
    return U.getAndAddInt(this, VALUE, 1) + 1;    // Unsafe 的 CAS 循环
}

// 本质是自旋 CAS
do {
    oldValue = getIntVolatile(this, valueOffset);
    newValue = oldValue + 1;
} while (!compareAndSwapInt(this, valueOffset, oldValue, newValue));
```

为什么不用 `synchronized`？CAS 是无锁的，在低竞争下比锁快得多（不需要线程挂起/唤醒的系统调用）。在线人数增减这种“低竞争、高频率”的场景，CAS 是最优选择。

CAS 的三个问题：

1. **ABA 问题**：值从 A 改成 B 又改回 A，CAS 检查不出来。用 `AtomicStampedReference`（带版本号）解决。本项目只是计数器，ABA 无影响。
2. **自旋开销**：高竞争下大量线程空转 CPU。JDK 8 的 `LongAdder` / `DoubleAdder` 更好——它用“分段累加”（类似 `ConcurrentHashMap` 的分段思想），把热点分散到多个 cell，读的时候求和。在高并发计数场景下性能远优于 `AtomicInteger`。
本项目连接数在万级以下，`AtomicInteger` 完全够用；如果连接数达到百万级，应该换成 `LongAdder`。
3. **只能保证单个变量的原子性**，多个变量之间无法保证（需要锁）。

`volatile` —— 可见性

```
private volatile int lastBroadcastCount = -1;
```

作用：保证一个线程写入后，其他线程立刻能看到最新值。

原理（JMM）：

1. **禁用 CPU 缓存优化**：写 `volatile` 变量时，会把该变量从工作内存刷回主内存；读的时候直接从主内存读。
2. **禁止指令重排序**：`volatile` 写之前的操作不能被重排到写之后，`volatile` 读之后的操作不能被重排到读之前（内存屏障）。

为什么 `lastBroadcastCount` 需要 `volatile`？

它被两个线程访问：

- `scheduledBroadcast()`（`@Scheduled` 线程）——写入。
- `setOnlineCountFromRedis()`（`OnlineUserTask` 的定时线程）——写入 `-1`。

如果不加 `volatile`，`scheduledBroadcast` 可能读到缓存在自己工作内存里的旧值，导致“人数已经变了但判断为没变，不广播”或者反之。

但注意：`volatile` 不保证原子性！

```
lastBroadcastCount = current;    // 这是单个赋值，原子
if (current == lastBroadcastCount) return; // 这是“读-比较”，不是原子操作
```

如果两个线程同时执行这个“检查-赋值”序列，可能都通过检查、都执行广播（重复广播）。

本项目的场景下：`scheduledBroadcast` 只有一个调度线程（Spring 默认的 `TaskScheduler` 是单线程的），`setOnlineCountFromRedis` 也只有一个定时线程，所以实际不存在并发。但这是一个隐性依赖——如果将来 `@Scheduled` 配置了多线程池，就会出现这个问题。

`broadcastRound` 为什么不需要 `volatile`？

```
private int broadcastRound = 0;    // 普通 int
```

因为它只在 `scheduledBroadcast()` 这一个方法里读写，而该方法由单线程调度器调用，不存在多线程访问。这个判断是对的（虽然隐式依赖了调度的单线程性质）。

Q10.3 `ConcurrentHashMap` 为什么线程安全？和 `Hashtable` 有什么区别？

本项目在 `UserWebSocketHandler` 里用了两个 `ConcurrentHashMap`。

HashMap	✗ 不安全	数组 + 链表 + 红黑树	—
Hashtable	✓ 安全	所有方法加 synchronized	1 (全表锁)
Collections.synchronizedMap	✓ 安全	包装类, 方法加锁	1
ConcurrentHashMap	✓ 安全	CAS + synchronized 锁单个桶	数组长度 (粒度细)

JDK 8 的 ConcurrentHashMap 实现要点

1. put 时若桶为空, 用 CAS 直接插入 (无锁)。
2. 若桶不为空, 用 **synchronized** 锁住该桶的头节点 (只锁一个链表/红黑树, 不影响其他桶)。
3. 读操作完全无锁: `get()` 不做任何同步, 靠 **volatile** 保证可见性 (Node 的 **val** 和 **next** 是 volatile 的)。
4. `size()` 用 CounterCell 分段计数 (类似 LongAdder), 避免每次扩容都要全表统计。
5. JDK 7 用的是“分段锁 (Segment)”, 默认 16 段, 所以并发度上限是 16。JDK 8 改成了“锁单个桶”, 并发度提升到数组长度, 且内存占用更小。

本项目的具体用例分析

```
// 用例一: 连接建立 (挤掉旧连接)
WebSocketSession old = sessions.put(userId, session); // 原子操作
if (old != null && old.isOpen()) { ... }
```

为什么这是安全的? `put()` 返回被覆盖的旧值, 这个“替换并返回旧值”的语义在 **ConcurrentHashMap** 里是原子的。所以如果同一个 `userId` 有两个并发连接, 只有一个能拿到非 null 的 `old`, 不会重复执行“关闭旧连接”的逻辑。这是正确的用法。

```
// 用例二: 连接关闭 (条件删除)
sessions.remove(userId, session); // 两个参数的 remove: 只有 value 匹配才删除
```

`remove(key, value)` 为什么比 `remove(key)` 好?

因为存在这样的竞态:

- ① 用户 A 的旧连接要被关闭, 执行 `remove(userId)`
- ② 但在此之前, 用户 A 已经用新连接重新登录, `sessions.put(userId, newSession)`
- ③ 旧连接的 `remove(userId)` 执行 → 把新连接删掉了!
- ④ 结果: 新连接明明存在, 但 `sessions` 里没有, 无法推送消息

用 `remove(userId, session)` (只在 value 确实是自己时才删) 就避免了误删。项目里这点做得很好。

但 counter 的操作没有一起保护 (前面 Q9.5 分析过): `onlineCount.incrementAndGet()` 是独立于 `sessions.put()` 的, 所以“覆盖旧连接时计数器仍然 +1”的 bug 无法被 **ConcurrentHashMap** 的原子性拯救。这是设计层面的问题, 不是集合层面的问题。

Q10.4 项目用了哪些线程池? Kafka 消费者的 concurrency 是怎么回事?

① Kafka 消费者的并发: concurrency 参数

```
@KafkaListener(topics = "snappedUp", concurrency = "3")
@KafkaListener(topics = "order", concurrency = "2")
```

concurrency 的含义: 启动 N 个 **KafkaConsumer** 实例 (每个一个独立线程), 在同一消费者组内。Kafka 会自动把 topic 的 partition 分配给这些线程。

关键规则:

- 一个 partition 只能被一个消费者线程消费 (保证分区内有序)。

- 如果 `partition` 数 < `concurrency`，多余的线程闲置。

本项目 `snappedUp` 用 `concurrency=3`，但代码里没有看到 `topic` 的 `partition` 配置。如果 `topic` 是 Kafka 自动创建的（默认 `num.partitions=1`），那 3 个线程里有 2 个是纯浪费，而且并发度实际是 1（因为只有一个分区）——这直接影响秒杀削峰的效果！这是一个需要确认的关键配置。

Spring Kafka 的线程模型：

```
KafkaMessageListenerContainer
├─ KafkaMessageListenerContainer#0 (线程 1) → 消费 partition 0, 3, 6...
├─ KafkaMessageListenerContainer#1 (线程 2) → 消费 partition 1, 4, 7...
├─ KafkaMessageListenerContainer#2 (线程 3) → 消费 partition 2, 5, 8...
```

② Spring 的 `@Scheduled` 线程池

```
@Scheduled(fixedDelay = 2000)    // UserWebSocketHandler.scheduledBroadcast
@Scheduled(fixedRate = 30000)    // OnlineUserTask.cleanStaleOnlineUsers
```

默认配置：Spring Boot 的 `TaskSchedulingAutoConfiguration` 会创建一个单线程的 `ThreadPoolTaskScheduler`（`poolSize=1`）。

单线程带来的问题：

- 所有 `@Scheduled` 任务串行执行。如果 `scheduledBroadcast()` 因为要广播 1 万个连接而耗时 3 秒，那么 `cleanStaleOnlineUsers()` 会被阻塞（反之亦然）。
- `fixedRate` 的语义会失真：`fixedRate = 30000` 期望每 30 秒执行一次，但如果前一次执行耗时 40 秒，下一次会立即开始（不会并发，而是排队）。实际间隔变成 40 秒。
- `fixedDelay = 2000` 是“上次结束后等 2 秒再执行”，所以不会积压，但实际频率会低于每秒 0.5 次。

修复：配置 `spring.task.scheduling.pool.size=4`，或者用 `ThreadPoolTaskScheduler` 显式配置。

但要注意：如果改成多线程，本项目里那些“依赖单线程”的隐式假设就会崩！比如 `broadcastRound` 是普通 `int`（非线程安全），`lastBroadcastCount` 的“读-比较-写”不是原子的。这是一个典型的“牵一发动全身”——改配置前必须先修代码。

③ Tomcat 的请求处理线程池

```
server:
  tomcat:
    threads:
      max: 200          # 默认 200
      min-spare: 10
```

本项目没配，用的是默认值。200 个线程对于普通业务够用，但如果：

- 接口里有阻塞操作（如 `kafkaTemplate.send().get()` 的改进版），线程会被占用。
- 秒杀高并发时，200 个线程可能在 `synchronized` 或 Redis 上排队。

数据库连接池的大小应该和 Tomcat 线程数匹配：如果 Tomcat 有 200 线程而 Druid 只有 10 个连接，会有 190 个线程在等连接。项目里没有配置 Druid 的连接池大小（用默认值，Druid 默认 `maxActive=8`）——这是一个明显的配置缺失，8 个数据库连接在秒杀场景下会成为严重瓶颈。

线程池大小的经验公式（HikariCP 作者的公式）：

连接数 = CPU 核心数 × 2 + 有效磁盘数

对于 8 核机器、SSD，大约 $8 \times 2 + 1 = 17$ 个连接就够了。盲目调大反而有害（数据库的上下文切换开销）。但 8 个确实偏小，建议调到 20~50。

④ 项目里没用线程池的地方（可以优化的）

```
// 广播循环（串行）
for (Long userId : sessions.keySet()) {
    sendToUser(userId, textMessage);
}
```

这是纯串行的。虽然单次发送很快，但 1 万个连接就是 1 万次循环。可以用线程池并行化（分成 N 批，每批一个线程），但要注意：并发写 session 需要 `ConcurrentWebSocketSessionDecorator` 而不是普通 `synchronized`。

Q10.5 项目的 JVM 内存需要注意什么？

三个内存风险点：

① Caffeine 本地缓存（可控）

```
Cache<Long, UserDO> userCache      → maxSize = 10000
Cache<String, Integer> countCache → 无大小限制（只有 1 个 key）
Cache<Long, ProductDO> productCache → maxSize = 500
```

内存估算：

- `UserDO` 对象约 300~500 字节（含字段、对象头、引用）。
- 10000×500 字节 $\approx$ 5 MB。
- `ConcurrentHashMap` 的额外开销（Node、数组）约 1 倍 → 总共约 10 MB。

结论：这个配置是安全的。`maximumSize` 起到了硬性上限的作用——这正是 Caffeine 优于无界 Map 的地方。

但 `countCache` 用的是 `Caffeine.newBuilder().expireAfterWrite(10, MINUTES).build()`，没有 `maximumSize`！

```
this.countCache = Caffeine.newBuilder()
    .expireAfterWrite(10, TimeUnit.MINUTES)
    .build(); // ← 没有 maximumSize，理论上无界
```

风险评估：当前只用了 `get("total", ...)` 一个 key，所以无界也没关系。但如果将来有人往里塞动态 key（比如按学院统计用户数），就会内存泄漏。

最佳实践：Caffeine 缓存永远配 `maximumSize` 或 `maximumWeight`，作为内存的硬性保障。这是防御性编程。

② WebSocket 的 session（真正的风险点）

前面 Q9.7 算过：每个 session 10~50 KB。

关键问题：僵尸连接的泄漏。`OnlineUserTask` 只清理了 Redis，没有清理 `sessions` 和 `sessionToUser` 这两个 Map。

```
// 只做了：
stringRedisTemplate.opsForZSet().removeRangeByScore(ONLINE_USERS_KEY, 0, threshold);
userWebSocketHandler.setOnlineCountFromRedis();
// sessions / sessionToUser 完全没有清理！
```

泄漏速率估算：假设每天有 1000 个用户因为网络异常断线（`afterConnectionClosed` 未被触发），每个 session 占 30 KB：

$1000 \times 30 \text{ KB} = 30 \text{ MB/天} \rightarrow$ 一个月 900 MB。

对于一个默认堆大小（1/4 物理内存）的 JVM，1~2 个月就会 OOM。

修复方案：① 清理任务里同时移除 Map 条目；② 更根本的是实现**服务端主动 ping**，让 `afterConnectionClosed` 能可靠触发；③ 用 `ConcurrentWebSocketSessionDecorator` 包装 session（它内部有超时限制，慢客户端会被自动关闭）。

③ Kafka 生产者的缓冲区

```
buffer-memory: 33554432    # 32 MB
```

这是生产者的**异步缓冲池**。当 Kafka 不可用时，消息会积压在缓冲区，**满了之后 `send()` 会阻塞或抛异常**（取决于 `max.block.ms`，默认 60 秒）。

32 MB 的配置：因为项目没有调用 `.get()` 也没加回调，所以超时和异常完全被忽略。**也就是说，如果 Kafka 挂了，用户的抢购请求会得到“成功”的响应，但消息丢了。**（前面 Q7.4 分析过这个一致性问题。）

JVM 参数建议：

```
-Xms4g -Xmx4g                # 堆大小，固定避免动态扩缩
-XX:+UseG1GC                 # G1 收集器（大堆低延迟）
-XX:MaxGCPauseMillis=200     # 目标暂停时间
-XX:+HeapDumpOnOutOfMemoryError # OOM 时 dump，便于排查
-XX:HeapDumpPath=/app/logs/   # dump 路径
-Xlog:gc*:file=/app/logs/gc.log:time # GC 日志
```

项目 `docker-compose.yml` 里没有配置任何 JVM 参数——容器里跑 Java 会用默认值，且如果没有配 `-XX:MaxRAMPercentage`，旧版 JVM 可能不识别容器内存限制，导致堆大小超过容器 limit 被 OOM Killer 杀掉（JDK 8u191+ 和 JDK 10+ 已支持 `UseContainerSupport` 默认开启，本项目用 JDK 17 所以是安全的，但显式配置更稳妥）。

Q10.6 做过压测吗？怎么压的？结果如何？

项目 `ToDo.java` 里明确记录了“解决压测失效问题”，说明作者做过压测。这是非常好的加分项——很多学生项目根本没压测过。

压测工具选择

工具	特点	适用
JMeter	图形化、功能全、可分布式	复杂场景、参数化、断言
wrk / wrk2	命令行的 C 程序，极高并发	纯 HTTP 压测，追求极限 QPS
ab	Apache 自带，简单	快速验证
Gatling / Locust	代码化的压测脚本	复杂业务流

压测的关键问题：怎么解决“压测失效”？

“压测失效”通常指压测结果不可信。常见原因：

- 压测机成为瓶颈：**用一台机器同时跑 JMeter 和服务，CPU 被抢。**应该用独立的压测机。**
- 连接数限制：**JMeter 的 `httpClient4.max_conn_per_host` 默认值低；Linux 的文件描述符限制。**需要调大。**
- 没有预热 (JIT)：**JVM 前 1 万次请求都在解释执行，**结果会非常难看。**必须先跑一轮预热，等 JIT 编译完成、类加载完毕、连接池填充完毕再开始统计。
- 压测数据是假的：**比如所有请求都用同一个 `userId`，那“一人一单”的 Redis 检查会直接返回 -2，**根本没测到真实的扣减路径。**必须用参数化（不同的 `userId`、不同的商品）。
- 只压了单接口，没压全链路：**秒杀的真实压力来自“Redis + Kafka + 数据库”的整条链路，只压 HTTP 接口无法发现 Kafka 消费慢、数据库连接池不足的问题。
- 数据库里没有真实数据量：**在 100 条数据的表上压测和在 100 万条数据上压测，SQL 性能完全不同。

应该关注的指标（面试必答）

指标	含义	健康值
QPS / TPS	每秒处理的请求数 / 事务数	越高越好，但要结合 RT
RT（响应时间）	平均、P95、P99、Max	看 P99 而不是平均值（平均值会掩盖长尾）
错误率	非 2xx 的比例	< 0.1%
CPU / 内存	应用、Redis、MySQL 的负载	CPU < 70%
GC 次数与耗时	Full GC 频率	Full GC 很少或没有
DB 连接池等待	druid 的 WaitThreadCount	接近 0
Kafka lag	消息堆积量	持续接近 0

面试时的回答模板（即使压测结果不理想也要说）：

"我用 JMeter/wrk 做了压测。**压测过程中遇到了几个问题**：一是压测机和服务在同一台机器上，压测机成了瓶颈，后来拆开才拿到真实数据；二是没预热导致第一轮数据很差，加上预热后 QPS 提升了近一倍；三是发现所有请求用同一个 userId 导致根本没走到真实扣减路径，改成参数化后才测出真实性能。**最终单机在 X 核 Y GB 的机器上，秒杀接口达到了约 N QPS，P99 响应时间 M 毫秒**。瓶颈主要在 Redis（ZSet 操作）和数据库连接池（默认只有 8 个连接，后来调大了）。"

即使数字不高（比如几百 QPS），只要你能说出“瓶颈在哪、怎么发现的、怎么优化的”，就比一个漂亮的假数字更有说服力。

第十一级：系统设计开放题（高难度）

Q11.1 如果 Redis 整个挂了，你的秒杀系统还能用吗？怎么设计降级？

诚实回答："当前完全不能用。"因为：

```
// ① 限流：Redis 挂了，stringRedisTemplate.execute() 直接抛异常 → 接口 500
// ② 秒杀：Lua 脚本执行失败 → 抛异常
// ③ 登录：Redis 缓存查询失败 → 抛异常（虽然会 fallback 查库，但查询本身就是异常）
// ④ 在线统计：ZSet 操作失败
```

根本原因："Redis 变成了强依赖，而不是可降级的缓存。"

降级设计的三个层次

层次一：可用性优先（放行）

```
try {
    Long execute = redisTemplate.execute(LUA_SCRIPT, keys);
    ...
} catch (RedisConnectionFailureException e) {
    log.error("Redis 不可用，降级处理", e);
    // 降级方案 A：直接拒绝，保护数据库
    return Result.error("系统繁忙，请稍后重试");
    // 降级方案 B：改为直接走数据库（有超卖风险，需要 DB 层强校验）
    return fallbackToDatabase(productId, userId);
}
```

推荐方案 A——因为秒杀是“宁可少卖，不可超卖”的业务。降级到数据库会导致数据库被打穿（Redis 本来是挡流量的），最终整个系统雪崩。

层次二：本地缓存兜底

```
// 用 Caffeine 做二级库存缓存
Cache<Long, AtomicInteger> localStock = Caffeine.newBuilder()
    .expireAfterWrite(1, TimeUnit.SECONDS) // 极短 TTL，尽量保证一致
    .build();

// Redis 挂了，用本地库存做粗粒度判断
if (redisDown) {
    AtomicInteger stock = localStock.get(productId, k -> new AtomicInteger(0));
    if (stock.decrementAndGet() < 0) {
        stock.incrementAndGet();
        return Result.error("商品已抢光");
    }
    // 异步记录到本地队列，等 Redis 恢复后同步
    pendingQueue.offer(new SnapRecord(userId, productId));
}
```

问题：多实例下本地库存会超卖（每个实例都以为自己有 100 个）。只适合“允许一定超卖”的场景。

层次三（根本方案）：高可用架构

1. **Redis 主从 + 哨兵**：主节点挂了，哨兵自动选新主（秒级切换，但会丢少量数据）。
2. **Redis Cluster**：分片 + 副本，容忍部分节点故障。
3. **多机房部署**：同城双活。

4. 客户端熔断：用 Sentinel / Resilience4j 做熔断，Redis 挂了快速失败，不拖垮线程池。
5. 连接超时配置： `spring.data.redis.timeout=2s`，避免请求卡死。项目里没配，用的是默认值。

本项目还有一个隐患： `@PostConstruct initStock()` 里 Redis 不可用会导致应用启动失败。生产环境应该：

```
@PostConstruct
public void initStock() {
    try {
        List<ProductDO> productDOList = productDAO.getAll();
        for (ProductDO p : productDOList) {
            redisTemplate.opsForValue().set(...);
        }
    } catch (Exception e) {
        // 启动时不阻塞，记录告警，改为懒加载
        log.error("库存预热失败，将使用懒加载模式", e);
        warmupFailed = true;
    }
}
```

核心原则：预热是可选的优化，不应该成为启动的必要条件。

Q11.2 如果流量再涨 10 倍，你最先优化的地方是哪里？

答这题的关键是显示"定位瓶颈的思路"，而不是罗列优化手段。

第一步：先说"怎么找瓶颈"（这比答案更重要）

- 看监控：QPS、RT、错误率、CPU、内存，找出"哪个环节先到极限"。
- 看链路：把一次请求拆成"限流 → Redis → Kafka → DB → 邮件"，看每段的耗时占比。
- 看依赖：Redis 的 QPS、Kafka 的 lag、数据库的 QPS 和连接池等待数、慢 SQL 日志。
- 看单机资源： `top`、`jstack`、GC 日志、火焰图。

第二步：按"性价比"排序给出优化清单

优先级	优化项	预期收益	成本
P0	调大数据库连接池（Druid 默认 maxActive=8）	高	极低（改配置）
P0	确认 Kafka topic 分区数 ≥ 消费者 concurrency	高	极低
P0	加上连接超时、Kafka 发送超时配置	高（防雪崩）	极低
P1	修缓存穿透/过期回源（Lua 脚本加回源兜底）	高	低
P1	修 WebSocket session 泄漏	高（防 OOM）	低
P1	售罄本地标记（库存归零后用本地变量直接拒绝，不查 Redis）	高	低
P2	库存分片（一个商品拆成 N 个 key）	高	中（逻辑复杂）
P2	Kafka 生产者批量化 + 消费者批量落库	高	中
P2	静态资源 CDN 化 + 前端限流	很高	中（需运维配合）
P3	Servlet 异步 / WebFlux 改造	中	高（改架构）
P3	分库分表	中	很高

第三步：重点展开一个“最有效”的方案

我会首选“售罄本地标记”，因为它的性价比极高：

```
// 每个实例维护一个本地“已售罄”标记
private final Cache<Long, Boolean> soldOutCache = Caffeine.newBuilder()
    .expireAfterWrite(5, TimeUnit.SECONDS)
    .build();

public Result<Boolean> snappedUp(long productId, long userId) {
    // ① 本地先判断：已售罄 → 直接拒绝，不碰 Redis
    if (Boolean.TRUE.equals(soldOutCache.getIfPresent(productId))) {
        return Result.error("商品已抢光");
    }

    // ② 走正常的 Lua 脚本
    Long execute = redisTemplate.execute(LUA_SCRIPT, keys);
    if (execute == -1) { // Redis 说售罄了
        soldOutCache.put(productId, true); // 打上本地标记
        return Result.error("商品已抢光");
    }
    ...
}
```

为什么收益巨大：售罄之后，99% 的请求都在本地被挡掉，Redis 的 QPS 直接归零。而秒杀场景的特点是“开始瞬间流量巨大，售罄后流量依然巨大”（用户不断刷新）。

代价：5 秒的本地缓存，如果这期间补货了，会有最多 5 秒的延迟才能重新抢购。**这对秒杀场景完全可接受。**

为什么把“调大连接池”放在 P0？ 因为这是“零成本高收益”的修改——Druid 默认 `maxActive=8`，在秒杀场景下必然成为瓶颈（200 个 Tomcat 线程抢 8 个连接）。**这是一个“看一眼配置就能发现的问题”，体现的是“你会不会做配置审查”。**

Q11.3 怎么设计一个“订单 30 分钟未支付自动取消”的功能？

本项目没有这个功能，但秒杀生成了 `PENDING_PAYMENT` 状态的订单，所以这是必然的追问。

注意：`OrderStatus` 枚举里已经定义了 `PENDING_PAYMENT(0)`、`PAID(1)`、`CANCELED(2)`、`COMPLETED(3)`，说明作者想到了这个需求，但没实现。

方案一：定时任务扫描（最简单，最常用）

```
@Scheduled(cron = "0 */1 * * * ?") // 每分钟执行
public void cancelExpiredOrders() {
    // 查 30 分钟前创建的待支付订单
    List<OrderDO> expired = orderDAO.findExpired(
        OrderStatus.PENDING_PAYMENT, LocalDateTime.now().minusMinutes(30), 100);
    for (OrderDO order : expired) {
        try {
            orderService.cancelOrder(order.getId());
        } catch (Exception e) {
            log.error("取消订单失败 {}", order.getId(), e);
        }
    }
}

@Transactional
public void cancelOrder(String orderId) {
    // ① 乐观锁更新状态（防止并发取消 + 支付）
    int rows = orderDAO.updateStatus(orderId, OrderStatus.CANCELED, OrderStatus.PENDING_PAYMENT);
    if (rows == 0) return; // 状态已被改变（比如用户刚好支付了），放弃
}
```

```

// ② 回滚库存（数据库 + Redis）
productDAO.reduceStock(productId, -1); // 库存加回来
redisTemplate.opsForValue().increment("product:stock:" + productId, 1);

// ③ 删除 Redis 的一人一单标记，允许用户重新抢
redisTemplate.delete("user:snapped:" + userId + ":" + productId);
}

```

关键设计点：

1. **updateStatus 用条件更新（乐观锁）**：`where id = ? and status = PENDING_PAYMENT`。防止“用户正在支付”和“定时任务取消”的并发冲突——只有一个能成功。
2. **必须回滚库存**，否则库存永久丢失。
3. **必须删除 Redis 去重标记**，否则用户不能再抢同一个商品。
4. **分批处理（limit 100）**，避免一次查出几万条导致内存和数据库压力。
5. **索引**：`order(gmt_created, status)` 建联合索引，避免全表扫描。

缺点：① 时间不精确（最多延迟 1 分钟）；② 空扫描浪费（没订单时也在查）；③ 数据量大时单次扫描慢。

方案二：Redis ZSet 延迟队列

```

// 创建订单时，把“取消时间”作为 score 放进 ZSet
long cancelAt = System.currentTimeMillis() + 30 * 60 * 1000;
redisTemplate.opsForZSet().add("order:delay:cancel", orderId, cancelAt);

// 定时任务扫描到期的
@Scheduled(fixedDelay = 5000)
public void processDelayQueue() {
    Set<String> expired = redisTemplate.opsForZSet()
        .rangeByScore("order:delay:cancel", 0, System.currentTimeMillis(), 0, 100);
    for (String orderId : expired) {
        // 原子移除，防止多实例重复处理
        Long removed = redisTemplate.opsForZSet().remove("order:delay:cancel", orderId);
        if (removed != null && removed > 0) {
            orderService.cancelOrder(orderId);
        }
    }
}
}

```

优点：① 无需扫全表（只查到期的）；② Redis 操作极快；③ `remove` 的返回值天然实现了“多实例只有一个能处理”的幂等。

缺点：① Redis 重启会丢数据（需要配合补偿任务）；② 如果 Redis 挂了，所有订单永远不会被取消。

方案三：RocketMQ / RabbitMQ 延迟消息（最优雅）

```

// RocketMQ
msg.setDelayTimeLevel(16); // 30 分钟（RocketMQ 内置 18 个延迟级别）
rocketMQTemplate.syncSend("order-cancel", msg);

// 消费者
@RocketMQMessageListener(topic = "order-cancel", consumerGroup = "cancel-group")
public void onMessage(String orderId) {
    orderService.cancelOrder(orderId); // 幂等：只取消待支付的
}

```

优点：不用自己实现调度，MQ 保证消息不丢、不重复。

缺点：RocketMQ 的延迟级别是固定的（1s/5s/10s/30s/1m/.../2h），30 分钟正好在某一级别上；Kafka 不支持延迟消息，所以本项目如果要用 MQ 方案必须换 MQ 或自研消息中转。

方案四：时间轮（高性能场景）

Netty 的 `HashedWheelTimer` 或者 Kafka 内部用的时间轮算法。**O(1) 的插入和移除**，适合千万级的定时任务。**只适合单机**，分布式需要配合分片。

方案五：Redis 键过期事件（不推荐）

```
redisTemplate.opsForValue().set("order:expire:" + orderId, "1", 30, TimeUnit.MINUTES);
// 订阅 __keyevent@0__:expired
```

为什么不推荐：① Redis 的过期事件**不保证及时**（惰性删除 + 定期删除，可能延迟几分钟）；② **不保证可靠**（Redis 重启或没有订阅者时事件会丢）；③ 需要开 `notify-keyspace-events` 配置，有性能影响。

面试推荐回答："如果订单量不大（万级），我会用**定时任务扫描 + 数据库索引 + 乐观锁**，最简单可靠；如果订单量大（百万级），用 **Redis ZSet 延迟队列**，配合定时对账兜底；如果团队已经有 RocketMQ，直接用延迟消息最省事。**Kafka 不支持延迟消息，这是当前技术栈的一个限制。**"

Q11.4 如果要给“选课系统”做秒杀级的优化，你怎么设计？

选课和秒杀的业务相似度极高（都是“有限容量的资源 + 大量并发争抢 + 一人一单”），所以可以直接复用秒杀的那套方案。**这是一个很好的“举一反三”展示机会。**

现状的问题

```
// UserCourseServiceImpl.insertUserCourse
UserCourseDO exist = userCourseDAO.getByUserIdAndCourseId(userId, courseId); // ① 先查
if (exist != null) return "已选过";
CourseDO courseDO = courseDAO.getByCourseId(courseId); // ② 再查
if (courseDO.getSelectedCount() >= courseDO.getTotalCapacity()) return "已满";
userCourseDAO.insertUserCourse(userId, courseId); // ③ 插入
int updateRows = courseDAO.increasedSelectedCount(courseId); // ④ 更新人数
// ← 没有 @Transactional! ③④ 之间失败会造成不一致
```

改造方案（对标秒杀）

第一步：数据库层补强（最低成本，最先做）

```
-- ① 唯一索引防重复选课（目前缺失，是最大的漏洞）
ALTER TABLE user_course ADD UNIQUE INDEX uk_user_course (user_id, course_id);

-- ② 课程表的余量查询加索引
ALTER TABLE course ADD INDEX idx_remaining (id, remaining);
```

```
// ③ 加事务
@Transactional(rollbackFor = Exception.class)
public Result<UserCourse> insertUserCourse(long userId, long courseId) {
    // 直接依赖 SQL 的影响行数判断，不做前置查询
    int updateRows = courseDAO.increasedSelectedCount(courseId); // where remaining > 0
    if (updateRows <= 0) return Result.error("课程已满");

    try {
        userCourseDAO.insertUserCourse(userId, courseId);
    } catch (DuplicateKeyException e) {
        throw new BusinessException("你已选过该课程"); // 触发回滚，余量加回去
    }
    return Result.ok();
}
```

关键改动：① 先扣余量再插记录（反向顺序）；② 用 SQL 影响行数判断，不做“先查后改”；③ 靠唯一索引防重复，异常触发回滚。

第二步：Redis 预扣减（高并发时）

```
-- Lua 脚本，和秒杀几乎一样
local remainKey = KEYS[1]      -- course:remain:{courseId}
local userKey    = KEYS[2]      -- user:selected:{userId}:{courseId}
if redis.call('exists', userKey) == 1 then return -2 end
local remain = tonumber(redis.call('get', remainKey) or 0)
if remain <= 0 then return -1 end
redis.call('decr', remainKey)
redis.call('set', userKey, 1, 'EX', 86400)
return 1
```

然后通过 Kafka 异步落库，和秒杀完全一样的模式。

第三步：退课

```
@Transactional
public Result<Void> deleteUserCourse(long userId, long courseId) {
    int rows = userCourseDAO.deleteUserCourse(userId, courseId);
    if (rows <= 0) return Result.error("未找到选课记录");
    courseDAO.decreasedSelectedCount(courseId);    // remaining + 1
    // Redis 同步
    redisTemplate.opsForValue().increment("course:remain:" + courseId, 1);
    redisTemplate.delete("user:selected:" + userId + ":" + courseId);
    return Result.ok();
}
```

但要注意“选课”和“秒杀”的业务差异：

维度	秒杀	选课
并发量级	瞬时万级、秒级完成	开学时高峰、持续几小时
时间窗口	1 分钟内售罄	持续几天
退课需求	几乎没有	很常见（要退课换课）
容量语义	只减不增	有增有减（退课释放名额）
公平性要求	先到先得	可能需要抽签 / 分年级批次

结论：选课的并发远低于秒杀（持续时间长，压力被摊平），可能不需要 Kafka 削峰这么重的方案。但数据库层的补强（唯一索引 + 事务 + SQL 守卫）是必须的，因为这是正确性问题，不是性能问题。

面试金句：“秒杀方案可以复用，但不能照搬。选课有退课、有时间窗口、有公平性要求，这些是秒杀没有的。我的原则是：先保证正确性（唯一索引 + 事务），再考虑性能（Redis 预扣减）。很多系统的问题是正确性没做好，就去堆性能方案。”

Q11.5 这个项目如果要上生产，你会做哪些改造？

这是一个综合性的问题，按“安全 → 可靠 → 性能 → 可维护”四个维度回答。

一、安全（最高优先级，必须改）

问题	风险	修复
----	----	----

所有密码/密钥明文写在 yml 并提交 Git	极高 (数据库被拖库)	环境变量 / Jasypt 加密 / 配置中心; 已有的泄露密钥必须全部轮换
JWT 密钥硬编码	极高 (可伪造任意用户)	环境变量注入 + 定期轮换
Druid 控制台 admin/admin	高	关闭或强密码 + IP 白名单
忘记密码重置为固定 123456	极高 (任意账号可被接管)	改为用户自行设置新密码
验证码校验后不删除	中 (可重放)	校验成功后立即删除 (Lua 原子)
验证码用 Random	中 (可预测)	改用 SecureRandom
/init 接口无鉴权	高 (可批量注入数据)	删除或加管理员权限
文件上传只校验 Content-Type	高 (可传恶意文件)	校验文件头魔数 + 扩展名白名单
X-Forwarded-For 可伪造	中 (可绕过限流)	网关层剥离 + 可信代理列表
CORS allowedOriginPattern("*")	中	改成域名白名单
Redis Jackson 反序列化 LaissezFaireSubTypeValidator	极高 (RCE)	改成白名单 BasicPolymorphicTypeValidator
Redis 公网可访问	极高	内网部署 / 安全组白名单 / 只允许应用服务器 IP

二、可靠性 (必须改)

1. Kafka 发送失败回滚 Redis (Q7.4)。
2. 事务回滚补偿 Redis 的逻辑要加条件判断 (Q7.6 的重复消费导致超卖)。
3. Kafka 消费者加死信队列和重试上限 (Q8.4 毒丸消息)。
4. Redis 库存加懒加载回源 + TTL 随机化 (Q5.2)。
5. WebSocket session 泄漏修复 (Q9.4)。
6. 选课加事务 + 唯一索引 (Q3.7)。
7. 定时任务加分布式锁 (多实例重复执行)。
8. 补全全局异常处理 (@ExceptionHandler(Exception.class) 兜底)。
9. 加定时对账任务 (Redis 库存 vs 数据库库存 vs 订单数)。

三、性能 (按需改)

1. Druid 连接池大小 (默认 8 太小)。
2. 确认 Kafka topic 分区数 $\geq$ 消费者 concurrency。
3. 补全所有超时配置 (Redis、Kafka、HTTP 客户端)。
4. 售罄本地标记 (Q11.2)。
5. JVM 参数显式配置 + GC 日志。
6. 课程缓存的过滤下推到数据库 (Q5.6)。
7. 删除 CommentController 里的重复查询 (Q3.6)。

四、可维护性

1. 删除死代码: SpringHttpSessionConfig (空类)、pom.xml 里不再使用的 spring-session 依赖、CourseServiceImpl 里未使用的 kryo / jsqlparser import。

2. **抽取重复代码**：两份相同的限流 Lua 脚本、两份相同的邮件配置代码（`EmailClient` 里 4 个方法重复了 Session 创建逻辑）。
3. **补全单元测试**：目前只有 2 个测试类，且 `UserServiceImplLoginTest` 用的还是 MD5（已与实现脱节）。
4. **加 CI/CD**：GitHub Actions / Jenkins 自动构建测试。
5. **加监控**：Prometheus + Grafana + SkyWalking 链路追踪 + ELK 日志。
6. **加接口鉴权注解**：现在所有需要登录的接口都靠"拦截器全拦 + 排除清单"，**新增接口如果忘了加白名单就是"默认需要登录"（安全），但如果忘记加入某个公开接口就很麻烦**。更明确的写法是自定义 `@RequireLogin` / `@PublicApi` 注解，让每个接口的权限语义显式化。

最严重的问题排序（面试官最想听到的）：

1. **忘记密码重置为 123456** —— 直接导致任意账号可被接管
2. **Redis 公网暴露 + 密码明文在 Git** —— 数据库和缓存可被直接读取
3. **Jackson 反序列化漏洞（LaissezFaire）** —— 可 RCE
4. **JWT 密钥泄露** —— 可伪造任意用户身份
5. **Kafka 发送失败不回滚 Redis** —— 资损

Q11.6 怎么设计一个能支撑"全校 3 万学生同时选课"的系统？

先做容量估算（面试官喜欢看你有没有量化思维）：

全校学生数：30,000
选课开放时间：假设 8:00:00 开放
瞬时并发：如果 50% 的学生在开放后 10 秒内点击 $\rightarrow 15,000 / 10 = 1,500$ QPS
峰值可能更高（前 1 秒） $\rightarrow$ 可能达到 5,000~10,000 QPS

注意：这个量级其实没有"秒杀"那么极端（选课的热点集中在少数热门课程，不是所有课程同时爆）。所以不需要过度设计。

分层设计

① 前端层

- **排队机制（关键）**：开放瞬间，前端先请求"取号"接口，拿到一个排队号，按号依次放行（类似 12306）。这样能把瞬时峰值削成稳定流量。
- **前端限流**：提交后按钮立即置灰，防止重复提交。
- **选课列表静态化**：课程列表提前缓存在浏览器/CDN，不用实时查库。

② 接入层

- Nginx 限流（`limit_req`）+ 负载均衡。
- **按学号哈希做一致性路由**：同一个学生的请求总落到同一实例（对本地缓存友好）。

③ 应用层

- 无状态 + 水平扩容（JWT 已满足）。
- **热点识别**：把课程分成"热门"（需要 Redis 预扣减）和"普通"（直接走数据库）两类，避免所有课程都走复杂链路。

④ 缓存层（核心）

```
// 每个课程的余量放 Redis
course:remain:{courseId}  $\rightarrow$  剩余名额

// 用户的选课标记
user:selected:{userId}:{courseId}  $\rightarrow$  1
```

```
// Lua 脚本: 原子判断"是否已选 + 是否有余量 + 扣减 + 标记"
```

⑤ 数据库层

- `user_course(user_id, course_id)` **唯一索引** (防重复选课)。
- `course(id, remaining)` 索引 + `where remaining > 0` 守卫。
- 选课记录表按学期分表 (历史数据归档)。
- **异步落库**: Redis 扣减成功后, Kafka 异步写数据库。但选课对"实时性"要求比秒杀低, 可以接受几秒的延迟吗?
| 这里有个重要的业务差异: **选课需要"实时看到已选课程列表"**。如果异步落库, 用户选完课马上刷新"我的课表", 可能查不到。
所以选课的异步窗口必须很短, 或者"我的课表"接口要读 Redis 而不是数据库。

⑥ 特殊设计: 选课的公平性

秒杀是"先到先得", 但选课可能需要公平性:

- **分批次开放**: 大四先选、大三其次.....把并发压力天然分散到不同时间段 (这也是国内高校的普遍做法)。
- **抽签制**: 开放报名窗口 (几小时), 结束后统一抽签。这样完全没有瞬时并发压力, 只需要处理"报名写入"这一个简单操作。
- **志愿制**: 学生填志愿, 系统按绩点/优先级分配 (类似高考志愿)。技术上是最优雅的方案, 把在线实时并发问题转化成了离线批处理问题。

⑦ 兜底

- 降级: Redis 挂了直接返回"系统繁忙", 保护数据库。
- 对账: 定时核对 Redis 余量和数据库 `remaining`。
- 监控: 各课程 QPS、Redis 热点 key、数据库慢查询。

面试金句: "最好的性能优化是改变业务模型。如果能用抽签制或者分批次开放, 把瞬时并发问题转化成离线批处理问题, 那比任何技术优化都有效。技术方案要先问业务能不能避免这个并发场景。"

第十二级：项目缺陷与“陷阱题”（面试官最爱问）

这一章汇总了本项目所有真实存在的缺陷。面试官最喜欢问“你这个项目有什么问题/不足”，因为很多人会答“没什么问题”，这直接暴露了缺乏 code review 能力。能主动、系统地指出自己项目的缺陷，是最强的加分项。

Q12.1 你觉得你这个项目最大的问题是什么？

不要回答“没什么问题”或“性能还不够好”这种空话。要具体、要分层。

推荐的回答结构：

“我梳理了一下，问题分三类：

第一类是正确性 bug（最严重）：

1. Kafka 发送失败没有回滚 Redis。Redis 扣了库存但消息没发出去，用户丢了资格、库存永久少一个。这是资损问题。
2. 事务回滚补偿 Redis 的逻辑有缺陷。Kafka 重复消费时抛 `DuplicateKeyException`，事务回滚会触发补偿，把已经正确扣减的库存又加回来，导致超卖。
3. 选课没有事务、没有唯一索引。插入记录和更新人数是两步，中间失败会造成数据不一致。
4. WebSocket 僵尸连接的 session 没被清理。Redis 里的记录清了，但 JVM 的 `sessions` Map 没清，会内存泄漏直到 OOM。

第二类是安全漏洞：

1. 忘记密码重置为固定的 123456，等于任何人拿到学号就能接管账号。
2. Redis 反序列化用了 `LaissezFaireSubTypeValidator`，存在 Jackson 反序列化 RCE 风险。
3. 所有密码和密钥明文写在 yml 里并提交到了 Git。
4. 验证码校验后不删除，5 分钟内可重放；且用的是 `Random` 不是 `SecureRandom`。

第三类是工程规范问题：

1. 两套限流的 Lua 脚本完全重复，改一处忘一处的风险。
2. `CommentController` 里把评论查了两遍（一次给业务，一次给 `PageHelper` 取分页信息）。
3. `SpringHttpSessionConfig` 是空类、pom 里有未使用的依赖，属于技术债没清理。
4. 测试用例用的是 MD5，但实现已经改成 BCrypt，测试和实现脱节了。

如果重做，我会优先修第一类和第二类，第三类在做 code review 时顺手清理。”

为什么这样回答好？

- ① 分类清晰，说明有系统性的思考，不是零散地挑毛病；
- ② 排序合理，正确性 > 安全 > 规范，层次感强；
- ③ 每一项都具体到代码，说明是真的读过自己的代码；
- ④ 给出了重做时的优先级，体现工程判断力。

绝对不要做的：① 说“没什么大问题”（面试官会觉得你不知道自己写了什么）；② 把不属于问题的东西说成问题（如“我只用了 MySQL，没上分库分表”——这是过度设计，不是缺陷）；③ 编造一些不存在的亮点来掩盖缺陷。

Q12.2 你的秒杀接口用 GET，这样对吗？

这是一个“陷阱题”，正确回答是：不对，这是我在 code review 时发现的问题。

```
@GetMapping("/snappedUp")    // ← 改状态的操作用了 GET
public Result<Boolean> snappedUp(@RequestParam("productId") long productId, ...)
```

为什么这是错的（要说出实际危害，不能说“不符合规范”就完了）：

- 1. 浏览器的预加载/预取：Chrome 会预取页面里的链接，可能在用户没点击的情况下就触发了抢购。
- 2. 搜索引擎爬虫：会爬取并触发接口。虽然这个接口需要登录，但爬虫带的 Cookie 可能仍然有效。
- 3. 中间代理/CDN 缓存：如果代理把 GET 响应缓存了，后续用户拿到的可能是缓存里的“抢购成功”，但实际没有执行。
- 4. 用户刷新页面重复触发：GET 请求在浏览器里“重新加载”会重放。
- 5. 语义混乱：GET 应该幂等且无副作用。同一个 URL 请求两次结果应该一样，但抢购两次结果完全不同。

正确做法：

```
@PostMapping("/snappedUp")
public Result<Boolean> snappedUp(@RequestParam("productId") long productId, ...)
```

可以主动补充的知识点：

方法	语义	幂等	安全
GET	查询资源	✓	✓（不改变服务端状态）
POST	创建资源 / 执行操作	✗	✗
PUT	全量更新（不存在则创建）	✓	✗
PATCH	部分更新	一般✗	✗
DELETE	删除	✓	✗

“幂等”和“安全”是 HTTP 方法的两个重要属性：安全 = 不改变服务端状态；幂等 = 执行多次和执行一次效果相同。抢购既不安全也不幂等，所以必须是 POST。

还要补充：即使改成 POST，也不解决“重复提交”问题——用户连点两次还是两次 POST。防重复提交要靠：① 前端按钮置灰；② 后端 Redis 去重（项目已有）；③ 幂等 token（前端先请求一个一次性 token，提交时带上，服务端校验并删除）。

Q12.3 你的 resetPwd 为什么把密码重置成 123456?

这是个严重的安全漏洞，正确的回答是承认并给出修复方案，不要辩解。

```
@Override
public Result<User> resetPwd(String studentNum, String email, String code) {
    UserDO userDO = userDao.findByStudentNum(studentNum);
    // 校验验证码
    if (!code.equals(recCode)) return "验证码不正确";

    // ← 这里：把密码重置为固定的 123456
    if (checkPassword("123456", userDO.getPassword())) {
        result.setMessage("重置密码失败，密码已经为123456");
        return result;
    }
    String resetEncodedPwd = encodePassword("123456");
    userDao.updatePwd(resetEncodedPwd, studentNum);
}
```

为什么这是严重问题：

- 1. 任何知道用户学号 + 邮箱的人，只要能拿到验证码，就能把密码改成 123456 然后登录。
- 2. 更糟的是：密码是“每个人都一样”的。如果一个用户重置了密码后就忘了改，任何知道他是 123456 的人都能登录他的账号。
- 3. 攻击面被放大：攻击者不需要猜密码，只需要猜验证码（4 位数字，只有 1 万种可能）。

4. 验证码校验后不删除：同一个验证码 5 分钟内可重复使用，进一步降低了攻击成本。

正确做法：

```
// 方案一：让用户输入新密码（最标准）
@PostMapping("/user/forgot")
public Result<User> resetPwd(@RequestParam String studentNum,
                             @RequestParam String email,
                             @RequestParam String code,
                             @RequestParam String newPassword) {    // ← 新增参数

    // ① 校验验证码
    // ② 校验新密码强度（长度、复杂度）
    // ③ 校验新密码不能和旧密码相同
    // ④ 更新密码
    // ⑤ 【重要】删除验证码，防止重放
    // ⑥ 【重要】使用该用户所有已签发的 JWT 失效（版本号 / 黑名单）
}

// 方案二：生成一次性重置链接（更安全，业界主流）
// 邮件里发一个带 token 的链接，链接有效期 30 分钟且只能使用一次
// 用户点击后进入"设置新密码"页面
// 这样验证码不参与密码设置，攻击者拿到验证码也没用
```

方案二的优势：① 邮件是用户控制的（只有本人能看到）；② 一次性 token 比 4 位数字验证码安全得多；③ 用户体验更好（直接点链接）。

面试时的回答模板："这里确实有问题，我当时的设计是'重置为临时密码，让用户登录后再改'，但实现时直接固定成了 123456，这是个设计缺陷——**临时密码应该是随机的、且强制用户首次登录时必须修改**，而不是所有人共用同一个值。正确的做法是让用户直接设置新密码，或者发一次性重置链接。**这个我在重构时会优先改掉。**"

坦白 + 说出正确的设计 + 说明优先级，比试图辩解要好得多。

Q12.4 你的项目里有没有"看起来对但实际有坑"的代码？

这是高阶问题，考验的是"并发边界情况的推演能力"。以下都是本项目真实存在的"隐性 bug"：

坑一：缓存空对象导致"注册后不能登录"

```
// login() 里的缓存穿透防护
if (userDO == null) {
    redisTemplate.opsForValue().set(studentNum, new UserDO(), 6, TimeUnit.MINUTES);
    return "登录账户不存在";
}
```

场景：用户注册前，先试着登录了一次（或者在注册页面输入了一个学号点了"登录"）。此时缓存里写入了一个空对象，TTL 6 分钟。

然后用户**立刻完成注册**，但 `register()` 方法里：

```
int insertResult = userDAO.insert(userDO2);
if (insertResult > 0) {
    redisTemplate.opsForValue().set(studentNum, userDO2, 6, TimeUnit.HOURS);    // ← 覆盖了空对象
    ...
}
```

项目在这里覆盖了空对象，所以这个问题被规避了。✅ 但要注意：在 `buildRegCode` 里是这样查的：

```
UserDO userD01 = (UserDO) redisTemplate.opsForValue().get(studentNum);
if (userD01 == null) {
    userD01 = userDAO.findByStudentNum(studentNum);
}
```

```

}
if (userD01 != null && userD01.getId() > 0) {
    return "该学号已注册，注册失败";
}

```

这里的判断是 `userD01 != null && userD01.getId() > 0`，正确处理了空对象 (`id=0`) 的情况。✅ 这个细节做得很好——如果写成 `userD01 != null`，那么在缓存了空对象之后，用户就无法注册了（因为空对象非 `null`）。

但 `updatePerMsg`、`updatePwd`、`updateAvatar`、`findByStudentNum` 这些方法里：

```

UserDO userDO = (UserDO) redisTemplate.opsForValue().get(studentNum);
if (userDO == null) {
    userDO = userDAO.findByStudentNum(studentNum);
}
userDO.setNickName(nickName); // ← 如果 userDO 是空对象(id=0)，这里会更新一个不存在的用户

```

`updatePerMsg` 里没有校验 `userDO.getId() > 0`，如果命中缓存的空对象，会执行 `updateByStudentNum`（影响 0 行），返回“更新失败”。不会造成数据损坏，但错误提示不准确。而 `updatePwd` 里会先 `checkPassword(oldPwd, userDO.getPassword())`——空对象的 `password` 是 `null`，BCrypt 的 `matches()` 传 `null` 会抛 `IllegalArgumentException`，导致 500 错误。

坑二：OnlineController 接口需要登录

```

// WebAppConfiguration 的排除清单里没有 /online/*
.excludePathPatterns("/api/user/login")
.excludePathPatterns("/user/login")
...
// ← 没有 /online/count、/online/stats、/online/list

```

后果：在线人数接口需要登录才能访问。但“在线人数”通常是给所有访客看的（包括未登录用户），这会导致首页显示不出来。除非前端在未登录时也带了 token，或者这个接口只在内页使用。这是一个需要确认的设计问题。

坑三：CourseServiceImpl 的过滤逻辑用 contains 而不是 equals

```

filtered = allCourses.stream()
    .filter(c -> {
        boolean match = true;
        if (StringUtils.hasText(grade)) {
            match &= c.getGrade() != null && c.getGrade().contains(grade);
        }
        ...
    })

```

这里 `c.getGrade()` 是一个 `List<String>`（通过 `JsonTypeHandler` 映射），`contains("2022")` 是列表包含判断，语义正确。✅

但如果 `grade` 是 `"2"`，而列表里是 `["2022", "2023"]`，`contains("2")` 返回 `false`（列表元素是全等的，不做前缀匹配）。所以这个逻辑是精确匹配，没有问题。

但和数据库版本的 SQL 不一致！

```

-- CourseDAO.xml 的 queryByCondition 用的是 JSON_CONTAINS
AND JSON_CONTAINS(grade, JSON_ARRAY(#{grade}))

```

`JSON_CONTAINS(grade, JSON_ARRAY("2022"))` 也是精确匹配数组元素，所以两者语义一致。✅ 但注意：Redis 缓存路径和数据库路径的过滤逻辑必须保持同步，否则“缓存命中”和“缓存未命中”会返回不同结果——这是一类非常隐蔽的 bug。

坑四: `EmailClient` 里的静态 `Properties` 污染

```
public void sendOrderEmail(String messageAcceptor, String content) {
    Properties props = System.getProperties(); // ← 用的是 System.getProperties()!
    props.setProperty("mail.smtp.host", "smtp.qq.com");
    ...
}
```

`System.getProperties()` 返回的是全局的 `System Properties` 对象! 往里面 `setProperty` 会污染整个 JVM 的系统属性。

后果: ① 其他组件如果读 `System.getProperty("mail.smtp.host")` 会拿到这个值; ② 多次调用会重复写入 (虽然幂等); ③ 更重要的是, 这是个反模式——应该用 `new Properties()`。

注意: 另外三个方法 (`sendVerifyCodeEmail`、`sendRegVerifyCode`、`sendSuggestionEmail`) 用的是 `new Properties()`, 只有 `sendOrderEmail` 用了 `System.getProperties()` ——是不一致的历史遗留。

坑五: `UserDAO.xml` 的 `updateByStudentNum` 无法把字段更新为空串

```
<update id="updateByStudentNum">
  update `user`
  <set>
    <if test="nickName != null and nickName != ''">
      nick_name=#{nickName},
    </if>
    <if test="email != null and email != ''">
      email=#{email},
    </if>
    <if test="personSign != null and personSign != ''">
      person_sign=#{personSign},
    </if>
    ...
  </set>
</update>
```

后果: 用户无法把自己的个签 (`personSign`) 清空。用户在前端把个签删掉 (提交空串), `<if test="personSign != ''">` 判定为 `false`, 这个字段不会出现在 SQL 里, 数据库里的旧值保留。

这是一个经典的 MyBatis 动态 SQL 陷阱。

修复: 如果业务允许清空, 应该只判 `null` 不判空串 (`<if test="personSign != null">`); 或者用不同的字段区分“不更新”和“更新为空” (如引入 `Optional` 或单独的 `update` 方法)。

坑六: `@RequestParam` 缺少 `required = false` 导致的 400

```
// UserController.sendCode
public Result<String> sendCode(@Parameter String studentNum,
                               @Parameter String email) {
    // @RequestParam 默认 required = true
}
```

如果前端漏传参数, Spring 会直接返回 400 (`MissingServletRequestParameterException`), 而不是进入方法体做业务校验。虽然 `UserControllerLoginTest` 里专门测了这个行为 (`loginFailWhenStudentNumMissing` 期望 400), 但从用户体验看, 返回 400 的 HTML 错误页不如返回 JSON 的业务错误提示友好。应该加全局异常处理捕获这个异常并返回统一格式。

Q12.5 你项目里的测试写得好吗?

诚实回答: 规模不够, 而且有一个测试已经和实现脱节了。

现状

测试类	类型	测试内容
UserControllerLoginTest	@WebMvcTest (切片测试)	登录接口的 5 个用例
UserServiceLoginTest	@ExtendWith(MockitoExtension) (纯单测)	login 方法的 8 个用例
ServerApplicationTests	@SpringBootTest	仅 contextLoads()

做得好的地方（要主动说出来）：

- 1. 测试分层清晰：@WebMvcTest 只加载 Controller 层（用 @MockitoBean 替换 Service 和 JwtUtil），MockitoExtension 纯单测不启动 Spring 容器。这体现了对“测试金字塔”的理解——大部分应该是快速的单测，少量是集成测试。
- 2. 用 @DisplayName 写了中文描述，可读性好。
- 3. 覆盖了关键分支：成功、参数为空、用户不存在、密码错误、缓存穿透防护、缓存回写。特别是“缓存空对象时用 6 分钟而不是 6 小时”这个断言，说明测试是有意设计的。
- 4. 用 verify() 验证了交互行为（verify(valueOperations, times(1)).set(..., eq(6L), eq(TimeUnit.MINUTES))），而不只是断言返回值。这是高质量的测试。
- 5. 用 lenient() 处理了不必要的 stub，用 verifyNoInteractions() 验证了“参数为 null 时不去访问 Redis”——这是有经验的表现。

严重问题：UserServiceLoginTest 与实现脱节

```
// 测试里构造密码用的是 MD5
String saltPwd = rawPassword + "ljs_zwy";
String expectedMd5 = DigestUtils.md5DigestAsHex(saltPwd.getBytes()).toUpperCase();
UserDO mockUserDO = new UserDO();
mockUserDO.setPassword(expectedMd5);

when(valueOperations.get(studentNum)).thenReturn(mockUserDO);
Result<User> result = userService.login(studentNum, rawPassword);

assertTrue(result.isSuccess()); // ← 这个断言会失败！
```

为什么失败：login() 里用的是 BCryptPasswordEncoder.matches(rawPassword, encodedPassword)。第二个参数是 MD5 十六进制串，不是合法的 BCrypt 哈希。BCrypt 的 matches() 对格式非法的哈希会打印警告日志并返回 false。

所以这个测试用例应该是失败的（除非它根本没被执行）。这说明：① 实现从 MD5 迁移到 BCrypt 时，测试没有同步更新；② 可能 CI 里没有跑测试，或者测试被跳过了。

这是一个很好的自我批评点：“这说明我的测试维护没跟上代码演进。实际情况是项目早期用 MD5，后来为了安全改成 BCrypt，但测试里的构造数据没改。这正好说明了为什么要有 CI——如果每次提交都跑测试，这个问题在改动的当天就会被发现。”

缺失的测试（可以主动列举）

- 1. 秒杀核心链路的测试完全没有——这是最该测的地方（Lua 脚本逻辑、并发扣减、幂等、回滚补偿）。
- 2. 没有并发测试：用 CountDownLatch + 线程池模拟并发抢购，验证不超卖。这是验证秒杀正确性的关键测试。
- 3. 没有集成测试：@SpringBootTest 只测了 contextLoads()，没有端到端验证“抢购 → Kafka → 落库 → 邮件”。
- 4. 覆盖率低：项目有 60+ 个类，只有 2 个测试类。

应该怎么写并发测试（如果面试官让你现场说）

```
@Test
@DisplayName("并发抢购 - 100 个线程抢 10 个库存，应只有 10 个成功")
```

```

void concurrentSnapShouldNotOverSell() throws InterruptedException {
    int threadCount = 100;
    int stock = 10;
    // 初始化 Redis 库存为 10
    redisTemplate.opsForValue().set("product:stock:1", stock);

    CountDownLatch startLatch = new CountDownLatch(1);
    CountDownLatch endLatch = new CountDownLatch(threadCount);
    AtomicInteger successCount = new AtomicInteger(0);
    ExecutorService pool = Executors.newFixedThreadPool(threadCount);

    for (int i = 0; i < threadCount; i++) {
        final long userId = i + 1;
        pool.submit(() -> {
            try {
                startLatch.await(); // 所有线程同时开始
                Result<Boolean> r = snappedService.snappedUp(1L, userId);
                if (r.isSuccess()) successCount.incrementAndGet();
            } catch (Exception ignored) {
            } finally {
                endLatch.countDown();
            }
        });
    }

    startLatch.countDown(); // 发令枪
    endLatch.await(30, TimeUnit.SECONDS);
    pool.shutdown();

    // 关键断言：不能超卖
    assertEquals(stock, successCount.get(), "超卖了!");
    // 再断言 Redis 库存为 0
    assertEquals(0, redisTemplate.opsForValue().get("product:stock:1"));
}

```

这段代码如果能在面试中写出来，是非常强的加分项。关键点：① `CountDownLatch` 保证所有线程同时起跑（否则测不出真实并发）；② 断言“成功数 == 库存数”（不超卖）；③ 再查 Redis 验证余量为 0（不遗漏）。

Q12.6 如果让你重构这个项目，你会怎么做？

这题是“Q11.5（上生产改造）”的延伸，但更强调“代码结构和架构”。

第一阶段：修正确性（1 周）

1. Kafka 发送失败回滚 Redis（加 `.get(timeout)` + catch）。
2. 事务回滚补偿逻辑加条件判断（避免重复消费时误补）。
3. 选课加 `@Transactional` + `user_course` 唯一索引。
4. WebSocket session 泄漏修复。
5. 忘记密码改为用户设置新密码。

第二阶段：修安全（3 天，可与第一阶段并行）

1. 所有密钥改环境变量 + 轮换已泄露的。
2. Redis Jackson 改白名单 `BasicPolymorphicTypeValidator`。
3. 验证码用 `SecureRandom` + 校验后删除。
4. 文件上传加魔数校验 + 扩展名白名单。
5. 关闭 Druid 控制台 / `/init` 接口加权限。

第三阶段：结构重构（2 周）

① 抽取重复代码

```
// 现在：两份完全相同的限流 Lua 脚本在 RateLimitAspect 和 RateLimiterInterceptor
// 重构：抽成统一组件
@Component
public class SlidingWindowLimiter {
    private final StringRedisTemplate redisTemplate;
    private final DefaultRedisScript<Long> script;    // 从 .lua 文件加载

    public boolean tryAcquire(String key, long windowMs, int maxRequests) {
        Long result = redisTemplate.execute(script, List.of(key),
            String.valueOf(System.currentTimeMillis()),
            String.valueOf(System.currentTimeMillis() - windowMs),
            String.valueOf(maxRequests),
            String.valueOf(windowMs / 1000 + 1),
            UUID.randomUUID().toString());
        return result != null && result == 1L;
    }
}
```

然后 `RateLimitAspect` 和 `RateLimiterInterceptor` 都调用它。脚本外置到 `resources/scripts/sliding_window.lua`，便于维护和单独测试。

② 统一限流策略：既然有了 AOP 注解式限流，拦截器式的就可以删掉了——给秒杀接口加 `@RateLimit(windowMs = 1000, maxRequests = 3)` 即可。减少一套机制，代码更简单。

③ 统一异常处理

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(RateLimitException.class)
    public ResponseEntity<Result<Void>> handleRateLimit(RateLimitException e) { ... }

    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<Result<Void>> handleUnauthorized(UnauthorizedException e) { ... }

    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<Result<Void>> handleMissingParam(...) { ... }    // 替代 400 HTML

    @ExceptionHandler(Exception.class)    // 兜底
    public ResponseEntity<Result<Void>> handleAll(Exception e) {
        log.error("未预期异常", e);
        return ResponseEntity.status(500).body(Result.error("系统繁忙，请稍后重试"));
    }
}
```

④ 分离 Web 与 API 层：现在 `control/` 和 `api/` 有重复的参数接收代码。重构方案：把 `control/` 定位为“Web 页面 (Thymeleaf)”，`api/` 定位为“纯 REST 接口”，Web 页面的表单提交统一走 API 的 Service，避免两套参数接收逻辑。或者更彻底：Web 端改成前端渲染 (Vue/React) + 只保留 API 层，彻底消除两套 Controller。

⑤ 清理死代码

- 删除 `SpringHttpSessionConfig`（空类）和 pom 里的 `spring-session` 依赖。
- 删除 `CourseServiceImpl` 里未使用的 `kryo`、`jsqlparser` import。
- 删除 `CommentServiceImpl` 里对 `CommentController` 的 import。
- 删除 `SnappedUpServiceImpl` 里未使用的 `RLock` import。

⑥ 统一缓存 key 命名

```
// 现在：用户缓存直接用 studentNum，破坏命名规范
redisTemplate.opsForValue().set(studentNum, userDO, ...);
// 重构：
redisTemplate.opsForValue().set(RedisConstant.USER_INFO_PREFIX + studentNum, userDO, ...);
```

⑦ 清理 `CommentController` 的重复查询（Q3.6）。

⑧ 补测试：秒杀并发测试、幂等测试、集成测试。引入 JaCoCo 做覆盖率统计，在 CI 里设阈值（如 60%）。

第四阶段：架构优化（可选，看业务需要）

1. 引入 **Spring Security**：替换自研拦截器，获得 CSRF 防护、安全响应头、方法级权限。
2. 文件存储改 **OSS**：解决本地存储的扩容问题。
3. 引入配置中心（Nacos / Apollo）：配置热更新、多环境管理。
4. 引入监控（Prometheus + Grafana + SkyWalking）。
5. 引入分布式锁管理定时任务（或用 XXL-JOB 替代 `@Scheduled`）。
6. 考虑微服务拆分（如果业务继续膨胀）：用户服务、课程服务、秒杀服务、通知服务。但要说清楚：“目前这个体量，单体应用 + 模块化是最好的，过早微服务化是负优化。”

面试金句：“重构的优先级是**正确性 > 安全 > 可维护性 > 性能 > 架构**。很多人在项目里最喜欢讨论性能优化和微服务拆分，但**正确性 bug 和 安全漏洞才是真正会导致生产事故的东西**。我的重构会先把这两个修完，再谈结构。”

Q12.7 现场写代码：实现一个防重复提交的幂等 token

常见的手写代码题。目的是考察“幂等”这个概念的落地能力。

方案：一次性 token（前端先取号，提交时核销）

```
// ① 获取 token 接口
@GetMapping("/api/idempotent/token")
public Result<String> getToken() {
    String token = UUID.randomUUID().toString().replace("-", "");
    // 存 Redis, 5 分钟有效
    redisTemplate.opsForValue().set("idempotent:token:" + token, "1",
                                   5, TimeUnit.MINUTES);

    return Result.ok(token);
}

// ② 校验并核销（必须原子！）
private static final DefaultRedisScript<Long> CONSUME_SCRIPT = ...;
// Lua:
//   if redis.call('exists', KEYS[1]) == 1 then
//       redis.call('del', KEYS[1])
//       return 1
//   end
//   return 0

public boolean consumeToken(String token) {
    Long result = stringRedisTemplate.execute(CONSUME_SCRIPT,
                                              Collections.singletonList("idempotent:token:" + token));

    return result != null && result == 1L;
}

// ③ 在业务接口里使用
@PostMapping("/api/product/snappedUp")
public Result<Boolean> snappedUp(@RequestParam long productId,
                                @RequestHeader("Idempotent-Token") String token,
```

```

        HttpServletRequest request) {
    // 先核销 token，失败说明重复提交
    if (!consumeToken(token)) {
        return Result.error("请勿重复提交");
    }
    UserInfo user = (UserInfo) request.getAttribute("currentUser");
    return snappedService.snappedUp(productId, user.getId());
}

```

为什么必须用 Lua 脚本核销？

```

// 错误写法：两步之间有竞态
if (Boolean.TRUE.equals(redisTemplate.hasKey(key))) {    // 线程 A、B 同时通过这里
    redisTemplate.delete(key);                          // A 删了，B 也删，都返回 success
    return true;
}

```

两个并发请求会同时通过 `hasKey` 检查，然后都执行 `delete`，都认为是首次提交。必须用 Lua 把“检查 + 删除”合成原子操作。

这个方案和现有的 Redis 去重（`user:snapped:{userId}:{productId}`）有什么区别？

方案	粒度	解决的问题
用户+商品去重	业务级	一个用户只能抢一次这个商品（永久/6小时）
幂等 token	请求级	同一个请求只执行一次（5 分钟）

两者不可互相替代：

- 用户想抢 商品 A 和商品 B：用户去重不会拦（不同 productId），token 也不会拦（不同请求）。✅
- 用户对 商品 A 连点 3 次：用户去重会拦住后 2 次（返回“已抢购过”）✅；但如果用户第一次没抢到（库存不足），第二次想再试，用户去重不会拦（因为第一次没写 userKey），token 会拦（token 已被核销）——这是个问题！

所以幂等 token 的正确用法是：“失败时不核销”，只在业务成功后核销：

```

// 改进：先执行业务，成功后再核销
Result<Boolean> result = snappedService.snappedUp(productId, user.getId());
if (result.isSuccess()) {
    redisTemplate.delete(tokenKey);    // 只有成功才消耗 token
}
return result;

```

但这样又有个新问题：如果业务执行成功但 Redis 删除失败，token 依然有效，用户重复提交会再次执行。不过秒杀业务本身有 Redis 去重兜底，所以可以接受。

总结：幂等 token 适合“创建订单/支付”这类不能重复执行的操作；秒杀场景其实已经有更好的“业务去重”方案，token 是锦上添花。关键是要说清楚“选择哪种幂等方案取决于业务语义”。

Q12.8 你项目里有没有做过“技术选型”的取舍？举一个例子。

这是一个考察“工程判断力”的问题。要展示“你考虑了多个方案、比较了 trade-off、基于具体场景做了决定”。

例子一：秒杀用 Redis+Lua 而不是分布式锁

考虑过的方案：

- 分布式锁（Redisson `RLock`）
- 数据库乐观锁（版本号）

- 数据库悲观锁 (`select ... for update`)
- Redis + Lua (最终选择)

比较过程：

方案	吞吐	实现复杂度	结论
悲观锁	极低（行锁串行化）	低	✗ 无法支撑高并发
乐观锁	中（冲突多时大量重试）	低	✗ 秒杀冲突率接近 100%，重试会打垮数据库
分布式锁	低（加解锁两次 RTT + 串行）	高（锁超时、续期、误删）	✗ 把并发变串行，丧失优势
Redis+Lua	高（单次 RTT、无锁）	中（要写 Lua）	✓ 选择

关键决策理由："秒杀的操作集合是 '检查 + 修改'，完全可以用 Redis 的原子命令族 (`EXISTS` / `GET` / `DECR` / `SET`) 组合表达。Lua 脚本让这个组合变成原子操作。既然原子命令能解决，就不需要锁——锁是用来解决'无法用原子命令表达的复合逻辑'的。"

例子二：选课用数据库行锁而不是 Redis

决策理由："选课的并发量（开学时峰值可能几百 QPS）远低于秒杀（万级）。用 Redis 预扣减会引入额外的复杂度和一致性问题（缓存和数据库同步、Redis 挂了怎么办），而收益很小。数据库的行锁在这个量级完全够用，所以选择更简单、更可靠的方案。"

加分点：能说清楚"什么时候不该用高级方案"。

例子三：Kafka 而不是线程池发邮件

(Q8.7 已详述)

但要主动承认这次选型可能过度："用 Kafka 发邮件的技术理由（持久化、多实例不重复、重试）是成立的，但架构经济性上可能过度设计。邮件量很小，为了 2 个 topic 维护一套 Kafka 不划算。如果重构，我会把邮件拆出去用 RabbitMQ（更适合小消息量 + 延迟队列），把 Kafka 专注在秒杀削峰上。"

例子四：Caffeine + Redis 的两级缓存而不是只用 Redis

决策理由："秒杀时每次抢购都要读商品信息。纯 Redis 方案下，1 万 QPS 就是 1 万次 Redis 请求。加上 Caffeine 后，商品信息从本地堆内存读取，Redis 压力接近 0。代价是可能读到最多 10 分钟的旧数据。对于商品名/图片这类不变字段完全可接受，但对于价格不行——这也是我的项目里的一个缺陷（我把整个 ProductDO 都缓存了，包括 price）。"

例子五：JWT 而不是 Session

决策理由："项目有多端 (Web + 移动端 API) 需求，Session 依赖 Cookie，移动端不方便。JWT 放在 `Authorization` 头里，多端统一。代价是失去了服务端的'主动失效'能力——这也是我项目的短板 (`/logout` 接口目前是空的，token 在 24 小时内依然有效)。修复方案是引入 Redis 黑名单或双 Token 机制，但那会部分牺牲 JWT 的无状态优势，是一个需要权衡的取舍。"

面试金句（这题的核心）："技术选型的关键不是'哪个技术更先进'，而是'哪个技术最适合当前场景'。我的原则是：① 优先选团队熟悉、运维成本低的；② 优先选能解决问题的最简单方案；③ 只有当简单方案真的撑不住时，才上复杂方案。过早优化和过度设计，比性能不足更常见、更危险。"

Q12.9 你在项目里踩过最大的坑是什么？怎么解决的？

这是行为面试题 (Behavioral Question)，用 STAR 结构回答。项目 `ToDo.java` 里有大量真实的踩坑记录，可以挑几个讲。

推荐讲这个：事务回滚导致 Redis 与数据库不一致

(`ToDo.java` 里记录："解决由于事务回滚导致的 redis 数据扣减与数据库不一致")

S (情境)：秒杀上线压测后发现，Redis 库存和数据库库存对不上。Redis 显示扣了 100 个，数据库只扣了 80 个。

T (任务)：找出不一致的原因并修复。

A (行动)：

1. 定位问题：写了一个对账脚本，比对 `product:stock:{id}` (Redis) 和 `product.stock` (数据库)。发现 Redis 总是比数据库小——说明 Redis 扣了但数据库没扣成功。
2. 分析根因：Kafka 消费时，`doSnapBusiness` 是事务操作。如果库存扣减成功但插入订单失败（比如唯一索引冲突、或者任何异常），事务会回滚，但 Redis 里已经扣掉的库存没有加回来。因为 Redis 不在事务里，事务回滚不会影响它。
3. 设计解决方案：需要一种“事务失败时能感知到并补偿”的机制。想到了 Spring 的 `TransactionSynchronizationManager`——它提供了事务生命周期的回调。
4. 实现：

```
// 关键：必须在所有数据库操作之前注册
// 否则如果前面的操作抛异常，这行根本不会执行
TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
    @Override
    public void afterCompletion(int status) {
        if (status == STATUS_ROLLED_BACK) {
            redisTemplate.opsForValue().increment(stockKey, 1); // 库存加回
            redisTemplate.delete(userKey); // 删去重标记
        }
    }
});

int stockRows = productDAO.reduceStock(productId, 1);
if (stockRows <= 0) throw new RuntimeException("库存扣减失败");
snappedUserDAO.insert(snappedUserDO);
orderDAO.insertOrder(orderDO);
```

5. 踩到的第二个坑：一开始我把注册逻辑放在了所有数据库操作的后面，结果发现补偿从来没执行过。后来才想明白——如果前面某行抛异常，后面这行注册代码根本不会被执行到。移到最前面才生效。

R (结果)：修复后重新压测，Redis 和数据库的库存完全一致。并且我把这个经验也应用到了“提交成功后才发邮件”上——用 `afterCommit` 保证消息只在事务真正提交后才发出，避免“订单回滚了但邮件已经发出去”的问题。

总结收获：“这个坑让我理解了‘缓存和数据库的事务边界’这个核心问题——缓存操作不在事务里，所以任何‘先改缓存再改数据库’的代码都必须显式处理回滚。后来我在项目里统一了原则：以数据库为准，缓存失败靠补偿，补偿逻辑必须注册在事务开始处。”

其他可以讲的坑（从 `ToDo.java` 里挑）

- “解决 redis 里面意外出现的分布式 ID 键值对”——订单号生成的 Redis key (`order:auto:increment:{日期}`) 没有设 TTL 或者 TTL 设置时机不对，导致 Redis 里堆积了垃圾 key。修复是确保 `expire()` 一定被调用。
- “解决压测失效问题”——压测结果不可信。（详见 Q10.6）
- “解决删除数据库用户和缓存用户以及缓存里面的登录信息残留问题”——早期用 Session 时，删除用户后 Session 没清，用户依然“已登录”。改 JWT 后这个问题消失了（但也引入了新的问题：JWT 无法主动失效）。
- “解决评论系统子评论无法挂载到父评论”——`handleComments` 里的树形构建逻辑。原本可能是直接遍历挂载，但评论是按时间排序的，如果子评论先于父评论出现（理论上不会，但删除+新增可能造成），父评论还没在 Map 里就会挂载失败。修复方案是两趟遍历：第一趟建 Map，第二趟挂载——这正是当前代码的做法（`commentMap.put(0L, root)` 用 0 作为虚拟根节点，是个很巧妙的设计）。
- “解决输入内容转义问题”——评论内容的 XSS 防护，用 `StringEscapeUtils.escapeHtml4(content)` 转义 HTML。这是一个正确的安全实践（虽然更彻底的方案是在输出时转义 + 设置 `Content-Security-Policy`）。

回答行为面试题的原则：

- ① 必须是真实的问题（面试官会追问细节，编的会露馅）；
- ② 要有“定位过程”，不能只说“我发现...然后改了”——要讲怎么发现的（日志？对账脚本？压测？）；
- ③ 要有反思和提炼，从具体问题上升到通用原则；
- ④ 最好能说明“这个经验还应用到了别处”，体现迁移能力。

Q12.10 综合题：请指出下面这段代码的所有问题

面试官常给一段代码让你找问题。下面用本项目的真实代码作为例子。

```
@GetMapping("/snappedUp")
public Result<Boolean> snappedUp(@RequestParam("productId") long productId,
                                HttpServletRequest request) {
    UserInfo userInfo = (UserInfo) request.getAttribute("currentUser");
    long userId = userInfo.getId();
    return snappedService.snappedUp(productId, userId);
}
```

问题清单：

1. GET 做写操作——应该是 `@PostMapping`。
2. `userInfo` 可能为 null——虽然拦截器会拦未登录请求，但如果这个路径被加到了排除清单，或者在拦截器未注册的场景下（比如单测），就会 NPE。应该加防御：

```
if (userInfo == null) return Result.error("未登录");
```

3. 没有幂等 token 校验（可选，取决于设计）。
4. 缺少 `@RateLimit` 注解——限流靠另一个拦截器，两套机制不统一。
5. 缺少 Swagger 的 `@ApiResponse` 描述（小问题）。
6. 没有日志——抢购是关键业务，应该记录 `userId` + `productId`（用于排查和风控）。但要注意不能记录敏感信息。

再看 Service 层：

```
public Result<Boolean> snappedUp(long productId, long userId) {
    Result<Boolean> result = new Result<>();

    if (productId <= 0 || userId <= 0) {
        result.setSuccess(false);
        result.setMessage("参数错误");
        result.setCode("500");
        return result;
    }

    String stockKey = RedisConstant.PRODUCT_STOCK_PREFIX + productId;
    String userKey = RedisConstant.USER_SNAPPED_PREFIX + userId + ":" + productId;

    Long execute = (Long) redisTemplate.execute(LUA_SCRIPT, Arrays.asList(stockKey, userKey));

    if (execute == -1) { result.setMessage("商品已抢光"); ... return result; }
    if (execute == -2) { result.setMessage("您已抢购过该商品"); ... return result; }
    if (execute != 1) { result.setMessage("抢购失败"); ... return result; }

    ProductDO productDO = productCache.get(productId, key -> productDAO.selectById(key));
    String productJsonStr = JSON.toJSONString(productDO);
    kafkaTemplate.send(Topics.KAFKA_SNAP_TOPIC, String.valueOf(userId), productJsonStr);

    result.setSuccess(true);
}
```

```

result.setMessage("若你抢购成功后会以邮件形式通知你。");
result.setCode("200");
result.setData(true);
return result;
}

```

问题清单：

1. 业务错误返回 code "500"——"商品已抢光"、"已抢购过"都是正常的业务结果，不是服务端错误。用 500 会让前端和监控误以为是系统故障。应该用明确的业务码（如 4002 = 已抢光，4003 = 重复抢购）。
这是一个很常见的设计问题——HTTP 状态码和业务码混用。项目里 `Result.code` 既是 HTTP 语义又当业务码用，语义混乱。
2. `productCache.get()` 可能返回 null——如果 `productId` 不存在，`productDAO.selectById()` 返回 null，Caffeine 的 `get(key, mappingFunction)` 不会缓存 null，但会返回 null。然后 `JSON.toJSONString(null)` 返回 "null" 字符串，发给 Kafka，消费者 `JSON.parseObject("null", ProductDO.class)` 会返回 null，然后 `productDO.getId()` 直接 NPE。

```

// 修复
ProductDO productDO = productCache.get(productId, key -> productDAO.selectById(key));
if (productDO == null) {
    // 商品不存在，应该回滚 Redis 已经扣掉的库存！
    redisTemplate.opsForValue().increment(stockKey, 1);
    redisTemplate.delete(userKey);
    return Result.error("商品不存在");
}

```

注意这个 bug 的严重性：一个不存在的 `productId` 攻击——攻击者传入 `productId=99999`。

- Lua 脚本里 `redis.call('get', 'product:stock:99999') or 0` → 得到 0 → 返回 -1 → "商品已抢光"。

所以实际上抢不到，这个攻击路径被 "or 0" 意外挡住了。

但反过来：如果 Redis 里恰好有一个 key 而数据库里没有对应商品（比如商品被删了但 Redis 没清），就会出现"Redis 扣减成功 → 缓存查不到商品 → NPE → 库存被白白扣掉"。

3. 参数校验用 `productId <= 0` 但没校验上界，且错误码是 500。
4. Kafka 发送没处理失败 (Q7.4)。
5. 没有事务——Redis 扣减和 Kafka 发送之间没有原子性。
6. `productCache` 的缓存穿透：如果查询一个不存在的 `productId`，`productDAO.selectById` 会被反复调用（Caffeine 不缓存 null）。应该缓存空对象或加布隆过滤器。
7. 返回的 `data` 是 `true`，但这个 true 只代表"Redis 扣减成功"，不代表"抢购成功"（因为落库是异步的）。语义误导——前端如果根据 `data == true` 显示"抢购成功"，就是错误的。

这题的回答技巧：不要一口气全说出来，按严重程度排序，先说完最严重的 3 个，然后说"另外还有一些次要问题"。面试官更看重"你能不能区分严重程度"，而不是"你能不能找出所有问题"。

附录：面试速查表

A. 核心技术点速记

技术点	一句话要点
Redis + Lua 秒杀	Lua 脚本在 Redis 中原子执行，把"查重 + 查库存 + 扣减 + 标记"四步合成一次 RTT 的原子操作；集群下需用哈希标签保证同槽
事务回滚补偿缓存	<code>TransactionSynchronizationManager</code> 注册回调； <code>afterCompletion</code> 判断 <code>STATUS_ROLLED_BACK</code> 做补偿； <code>afterCommit</code> 只在成功时发消息；注册必须在事务最前面
滑动窗口限流	Redis ZSet 存请求时间戳， <code>ZREMRANGEBYSCORE</code> 清窗口外 + <code>ZCARD</code> 计数 + <code>ZADD</code> 记录，全部在 Lua 里原子完成
两级缓存	Caffeine（堆内、纳秒级、多实例不一致）+ Redis（分布式、毫秒级、一致）；热点不变数据用本地，共享数据用 Redis
缓存穿透防护	缓存空对象（短 TTL）+ 用 <code>id <= 0</code> 判断空对象，避免"注册后无法登录"
JWT 无状态认证	Header + Payload + Signature； <code>Keys.hmacShaKeyFor</code> 要求密钥 ≥ 32 字节；最大痛点无法主动失效（需黑名单/双 Token）
BCrypt	自带随机盐 + cost factor 可调（故意慢）+ 常量时间比较；输入截断 72 字节
Kafka 手动 ack	<code>enable-auto-commit=false</code> + <code>ack-mode: manual_immediate</code> ；失败不 ack 会无限重试（毒丸消息），需配死信队列
Kafka 幂等生产者	<code>enable.idempotence=true</code> ：PID + 分区级序列号，broker 去重；强制要求 <code>acks=all</code>
WebSocket 心跳	客户端 15s 发心跳 → 更新 ZSet score；定时任务 30s 清理 2 分钟无心跳的记录； <code>ConcurrentHashMap</code> 的 session 也要清，否则内存泄漏
WebSocket 并发写	<code>WebSocketSession</code> 非线程安全，必须 <code>synchronized (session)</code> 或用 <code>ConcurrentWebSocketSessionDecorator</code> ，否则 <code>TEXT_PARTIAL_WRITING</code>
MyBatis TypeHandler	枚举存业务 code 而非 ordinal（顺序敏感）或 name（重命名失效）；JSON 字段用 <code>JsonTypeHandler</code> 桥接 <code>List<String></code>
分页	手写 <code>limit #{offset}, #{size}</code> （ <code><bind></code> 算 offset，安全）；深分页用游标（ <code>where id > lastId</code> ）替代大 offset

B. 高危问题 Top 10（必须记住）

#	问题	位置	危害
1	忘记密码重置为固定 123456	<code>UserServiceImpl.resetPwd()</code>	任意账号可被接管
2	Redis Jackson <code>LaissezFaireSubTypeValidator</code>	<code>RedisTemplateInit</code>	反序列化 RCE
3	Redis 公网暴露 + 密码明文提交 Git	<code>application.yml</code>	缓存数据可被直接读取/篡改
4	JWT 密钥硬编码	<code>application.yml</code>	可伪造任意用户身份
5	Kafka 发送失败不回滚 Redis	<code>SnappedUpServiceImpl</code>	资损（库存白扣）
6	重复消费触发补偿导致超卖	<code>doSnapBusiness</code> 补偿逻辑	超卖
7	验证码校验后不删除 + 用 Random	<code>register / resetPwd</code>	验证码可重放/可预测

8	<code>/init</code> 接口无鉴权	<code>InitController</code> + 拦截器排除	数据可被批量注入
9	文件上传只校验 <code>Content-Type</code>	<code>UserController.uploadAvatar</code>	可传恶意文件
10	WebSocket session 未清理	<code>OnlineUserTask</code>	内存泄漏 OOM

C. 面试回答的通用原则

1. 先给结论，再给理由。例如“我认为这个设计有问题，原因是……”
2. 讲技术点时要结合项目。不要空谈理论，要说“我在项目的 X 文件里用了 Y，因为 Z”。
3. 主动暴露问题，但要有修复方案。“这里有问题”不如“这里有问题，我在重构时会用 X 方案修复”。
4. 量化。“性能提升了”不如“P99 从 800ms 降到 120ms”。
5. 说清楚 trade-off。任何方案都有代价，说出代价说明你思考过。
6. 不知道就说不知道，然后给出你的思路。“这个我没做过，但我会从 X 角度考虑，因为……”
7. 不要过度吹嘘。面试官会深挖，吹得越高摔得越惨。
8. 准备好三个故事：一个技术难点（秒杀一致性）、一个协作/学习经历、一个失败/踩坑经历。

D. 高频追问链（面试官会顺着往下问）

秒杀 → Redis 为什么能扛住？ → Lua 为什么原子？ → 集群下有什么问题？（CROSSSLOT）
 → 为什么不用分布式锁？ → 分布式锁怎么实现？ → Redisson 看门狗机制？
 → Redis 和数据库怎么一致？ → 事务回滚怎么补偿？ → afterCommit 和 afterCompletion 区别？
 → Kafka 发送失败了怎么办？ → 怎么保证不丢消息？ → 重复消费怎么幂等？
 → 超卖怎么防？ → 一人一单怎么防？ → 这三层防护有什么漏洞？

缓存 → 用的什么序列化？ → 默认的有什么问题？ → activateDefaultTyping 有什么用？
 → 有什么安全风险？ → 为什么需要 JavaTimeModule？
 → 缓存穿透/击穿/雪崩怎么处理？ → Caffeine 原理？ → 多实例一致性怎么办？

E. 最后一句话总结项目

“这是一个校园生活服务平台，技术上的核心亮点是秒杀链路：用 Redis + Lua 做原子预扣减把流量挡在数据库之前，用 Kafka 异步削峰落库，用事务回调实现‘数据库失败则补偿 Redis’的最终一致性，用数据库唯一索引做幂等的最后防线。我在这个过程中踩过‘服务重启库存丢失’、‘事务回滚缓存不一致’、‘压测数据不可信’这些真实的坑，也因此理解了缓存和数据库的事务边界、分布式并发下的正确性保障这些核心问题。”